NumPy

Knowledge-Base Reference

Version 2.5.4

Arrays, indexing, broadcasting, dtypes, ufuncs, I/O and performance, plus an API quick reference for the most-used functions.

Source: official documentation, https://numpy.org/doc/stable/
Content © 2005-2026 NumPy Developers (BSD-3-Clause)
Compiled October 29, 2026

Contents

About this reference 3

What is NumPy? 4
- Why is NumPy fast? 5
- Who else uses NumPy? 5

NumPy: the absolute basics 6
- How to import NumPy 6
- Reading the example code 6
- Why use NumPy? 6
- What is an ”array”? 7
- Array fundamentals 7
- Array attributes 9
- How to create a basic array 9
- Adding, removing, and sorting elements 10
- How do you know the shape and size of an array? 11
- Can you reshape an array? 11
- How to convert a 1D array into a 2D array (how to add a new axis to an array) 12
- Indexing and slicing 13
- How to create an array from existing data 15
- Basic array operations 17
- Broadcasting 18
- More useful array operations 19
- Creating matrices 20
- Generating random numbers 24
- How to get unique items and counts 24
- Transposing and reshaping a matrix 25
- How to reverse an array 26
- Reshaping and flattening multidimensional arrays 27
- How to access the docstring for more information 28
- Working with mathematical formulas 30
- How to save and load NumPy objects 32
- Importing and exporting a CSV 33
- Plotting arrays with Matplotlib 34

Array creation 36
- Introduction 36
- 1) Converting Python sequences to NumPy arrays 36
- 2) Intrinsic NumPy array creation functions 37
- 3) Replicating, joining, or mutating existing arrays 40
- 4) Reading arrays from disk, either from standard or custom formats 41
- 5) Creating arrays from raw bytes through the use of strings or buffers 41
- 6) Use of special library functions (e.g., SciPy, pandas, and OpenCV) 42

Indexing on ndarrays 43
- Basic indexing 43
- Advanced indexing 46
- Field access 52
- Flat iterator indexing 52

Assigning values to indexed arrays . . . 53
Dealing with variable numbers of indices within programs . . . 53
Detailed notes . . . 54

Data types **56**
Array types and conversions between types . . . 56
Relationship Between NumPy Data Types and C Data Types . . . 58
Array scalars . . . 59
Overflow errors . . . 59
Floating point precision . . . 59
Extended precision . . . 60

Broadcasting **61**
General broadcasting rules . . . 62
Broadcastable arrays . . . 62
A practical example: vector quantization . . . 65

Copies and views **67**
View . . . 67
Copy . . . 67
Indexing operations . . . 67
Other operations . . . 68
How to tell if the array is a view or a copy . . . 68

Universal functions (ufunc) basics **70**
Ufunc methods . . . 70
Output type determination . . . 71
Broadcasting . . . 72
Type casting rules . . . 72
Use of internal buffers . . . 74
Error handling . . . 74
Overriding ufunc behavior . . . 74

Writing performant NumPy code **75**
Introduction . . . 75
General concepts for multi-core processors in Python . . . 75
Using multi-core processors with Python standard libraries . . . 78
Third Party Libraries for Multi-Core Processing . . . 82

API quick reference **83**
Array creation . . . 83
Shape manipulation . . . 93
Math and statistics . . . 102
Logic, sorting and searching . . . 121
Linear algebra . . . 130
Random sampling (Generator) . . . 137
I/O . . . 141

About this reference

This knowledge-base reference collects core chapters of the official **NumPy 2.5.4** documentation (https://numpy.org/doc/stable/).

- **Chapters 2 onward** are converted from the project's own documentation source files (the reStructuredText files the website is built from), taken from the `v2.5.4` release tag of github.com/numpy/numpy. Text and code are the project's; Sphinx-only elements such as cross-reference links, rendered plot output and interactive widgets were simplified for print.
- **The API quick reference** at the end is generated from the docstrings of the installed 2.5.4 package, the same text the official API pages are built from. Each entry is trimmed to signature, parameters, returns, notes and a short excerpt of the notes and examples.

Built October 29, 2026 for a personal knowledge base. Selection and assembly by Claude; no content was rewritten.

License. NumPy and its documentation are distributed under the BSD 3-Clause License, Copyright (c) 2005-2026, NumPy Developers. Redistribution with this notice is permitted. Full text: https://numpy.org/doc/stable/license.html

Chapter sources

- What is NumPy?: https://numpy.org/doc/stable/user/whatisnumpy.html
- NumPy: the absolute basics: https://numpy.org/doc/stable/user/absolute_beginners.html
- Array creation: https://numpy.org/doc/stable/user/basics.creation.html
- Indexing on ndarrays: https://numpy.org/doc/stable/user/basics.indexing.html
- Data types: https://numpy.org/doc/stable/user/basics.types.html
- Broadcasting: https://numpy.org/doc/stable/user/basics.broadcasting.html
- Copies and views: https://numpy.org/doc/stable/user/basics.copies.html
- Universal functions (ufunc) basics: https://numpy.org/doc/stable/user/basics.ufuncs.html
- Writing performant NumPy code: https://numpy.org/doc/stable/user/basics.performant_code.html

What is NumPy?

Source: https://numpy.org/doc/stable/user/whatisnumpy.html

NumPy is the fundamental package for scientific computing in Python. It is a Python library that provides a multidimensional array object, various derived objects (such as masked arrays and matrices), and an assortment of routines for fast operations on arrays, including mathematical, logical, shape manipulation, sorting, selecting, I/O, discrete Fourier transforms, basic linear algebra, basic statistical operations, random simulation and much more.

At the core of the NumPy package, is the ~numpy.ndarray object. This encapsulates *n*-dimensional arrays of homogeneous data types, with many operations being performed in compiled code for performance. There are several important differences between NumPy arrays and the standard Python sequences:

- NumPy arrays have a fixed size at creation, unlike Python lists (which can grow dynamically). Changing the size of an ~numpy.ndarray will create a new array and delete the original.
- The elements in a NumPy array are all required to be of the same data type, and thus will be the same size in memory. The exception: one can have arrays of (Python, including NumPy) objects, thereby allowing for arrays of different sized elements.
- NumPy arrays facilitate advanced mathematical and other types of operations on large numbers of data. Typically, such operations are executed more efficiently and with less code than is possible using Python's built-in sequences.
- A growing plethora of scientific and mathematical Python-based packages are using NumPy arrays; though these typically support Python-sequence input, they convert such input to NumPy arrays prior to processing, and they often output NumPy arrays. In other words, in order to efficiently use much (perhaps even most) of today's scientific/mathematical Python-based software, just knowing how to use Python's built-in sequence types is insufficient - one also needs to know how to use NumPy arrays.

The points about sequence size and speed are particularly important in scientific computing. As a simple example, consider the case of multiplying each element in a 1-D sequence with the corresponding element in another sequence of the same length. If the data are stored in two Python lists, `a` and `b`, we could iterate over each element:

```
c = []
for i in range(len(a)):
    c.append(a[i]*b[i])
```

This produces the correct answer, but if `a` and `b` each contain millions of numbers, we will pay the price for the inefficiencies of looping in Python. We could accomplish the same task much more quickly in C by writing (for clarity we neglect variable declarations and initializations, memory allocation, etc.)

```
for (i = 0; i < rows; i++) {
  c[i] = a[i]*b[i];
}
```

This saves all the overhead involved in interpreting the Python code and manipulating Python objects, but at the expense of the benefits gained from coding in Python. Furthermore, the coding work required increases with the dimensionality of our data. In the case of a 2-D array, for example, the C code (abridged as before) expands to

```
for (i = 0; i < rows; i++) {
  for (j = 0; j < columns; j++) {
    c[i][j] = a[i][j]*b[i][j];
```

```
  }
}
```

NumPy gives us the best of both worlds: element-by-element operations are the ”default mode” when an ~numpy.ndarray is involved, but the element-by-element operation is speedily executed by pre-compiled C code. In NumPy

```
c = a * b
```

does what the earlier examples do, at near-C speeds, but with the code simplicity we expect from something based on Python. Indeed, the NumPy idiom is even simpler! This last example illustrates two of NumPy's features which are the basis of much of its power: vectorization and broadcasting.

Why is NumPy fast?

Vectorization describes the absence of any explicit looping, indexing, etc., in the code - these things are taking place, of course, just ”behind the scenes” in optimized, pre-compiled C code. Vectorized code has many advantages, among which are:

- vectorized code is more concise and easier to read
- fewer lines of code generally means fewer bugs
- the code more closely resembles standard mathematical notation (making it easier, typically, to correctly code mathematical constructs)
- vectorization results in more ”Pythonic” code. Without vectorization, our code would be littered with inefficient and difficult to read `for` loops.

Broadcasting is the term used to describe the implicit element-by-element behavior of operations; generally speaking, in NumPy all operations, not just arithmetic operations, but logical, bit-wise, functional, etc., behave in this implicit element-by-element fashion, i.e., they broadcast. Moreover, in the example above, `a` and `b` could be multidimensional arrays of the same shape, or a scalar and an array, or even two arrays with different shapes, provided that the smaller array is ”expandable” to the shape of the larger in such a way that the resulting broadcast is unambiguous. For detailed ”rules” of broadcasting see Broadcasting.

Who else uses NumPy?

NumPy fully supports an object-oriented approach, starting, once again, with ~numpy.ndarray. For example, ~numpy.ndarray is a class, possessing numerous methods and attributes. Many of its methods are mirrored by functions in the outer-most NumPy namespace, allowing the programmer to code in whichever paradigm they prefer. This flexibility has allowed the NumPy array dialect and NumPy ~numpy.ndarray class to become the *de-facto* language of multi-dimensional data interchange used in Python.

NumPy: the absolute basics

Source: https://numpy.org/doc/stable/user/absolute_beginners.html

Welcome to the absolute beginner's guide to NumPy!

NumPy (**Num**erical **Py**thon) is an open source Python library that's widely used in science and engineering. The NumPy library contains multidimensional array data structures, such as the homogeneous, N-dimensional `ndarray`, and a large library of functions that operate efficiently on these data structures. Learn more about NumPy at What is NumPy, and if you have comments or suggestions, please reach out!

How to import NumPy

After installing NumPy, it may be imported into Python code like:

```
import numpy as np
```

This widespread convention allows access to NumPy features with a short, recognizable prefix (`np.`) while distinguishing NumPy features from others that have the same name.

Reading the example code

Throughout the NumPy documentation, you will find blocks that look like:

```
>>> a = np.array([[1, 2, 3],
...               [4, 5, 6]])
>>> a.shape
(2, 3)
```

Text preceded by `>>>` or `...` is **input**, the code that you would enter in a script or at a Python prompt. Everything else is **output**, the results of running your code. Note that `>>>` and `...` are not part of the code and may cause an error if entered at a Python prompt.

To run the code in the examples, you can copy and paste it into a Python script or REPL, or use the experimental interactive examples in the browser provided in various locations in the documentation.

Why use NumPy?

Python lists are excellent, general-purpose containers. They can be ”heterogeneous”, meaning that they can contain elements of a variety of types, and they are quite fast when used to perform individual operations on a handful of elements.

Depending on the characteristics of the data and the types of operations that need to be performed, other containers may be more appropriate; by exploiting these characteristics, we can improve speed, reduce memory consumption, and offer a high-level syntax for performing a variety of common processing tasks. NumPy shines when there are large quantities of ”homogeneous” (same-type) data to be processed on the CPU.

What is an "array"?

In computer programming, an array is a structure for storing and retrieving data. We often talk about an array as if it were a grid in space, with each cell storing one element of the data. For instance, if each element of the data were a number, we might visualize a "one-dimensional" array like a list:

1	5	2	0

A two-dimensional array would be like a table:

1	5	2	0
8	3	6	1
1	7	2	9

A three-dimensional array would be like a set of tables, perhaps stacked as though they were printed on separate pages. In NumPy, this idea is generalized to an arbitrary number of dimensions, and so the fundamental array class is called `ndarray`: it represents an "N-dimensional array".

Most NumPy arrays have some restrictions. For instance:

- All elements of the array must be of the same type of data.
- Once created, the total size of the array can't change.
- The shape must be "rectangular", not "jagged"; e.g., each row of a two-dimensional array must have the same number of columns.

When these conditions are met, NumPy exploits these characteristics to make the array faster, more memory efficient, and more convenient to use than less restrictive data structures.

For the remainder of this document, we will use the word "array" to refer to an instance of `ndarray`.

Array fundamentals

One way to initialize an array is using a Python sequence, such as a list. For example:

```
>>> a = np.array([1, 2, 3, 4, 5, 6])
>>> a
array([1, 2, 3, 4, 5, 6])
```

Elements of an array can be accessed in `various ways <quickstart.indexing-slicing-and-iterating>`. For instance, we can access an individual element of this array as we would access an element in the original list: using the integer index of the element within square brackets.

```
>>> a[0]
1
```

> Note
> As with built-in Python sequences, NumPy arrays are "0-indexed": the first element of the array is accessed using index `0`, not `1`.

Like the original list, the array is mutable.

```
>>> a[0] = 10
>>> a
array([10,  2,  3,  4,  5,  6])
```

Also like the original list, Python slice notation can be used for indexing.

```
>>> a[:3]
array([10, 2, 3])
```

One major difference is that slice indexing of a list copies the elements into a new list, but slicing an array returns a *view*: an object that refers to the data in the original array. The original array can be mutated using the view.

```
>>> b = a[3:]
>>> b
array([4, 5, 6])
>>> b[0] = 40
>>> a
array([ 10,  2,  3, 40,  5,  6])
```

See basics.copies-and-views for a more comprehensive explanation of when array operations return views rather than copies.

Two- and higher-dimensional arrays can be initialized from nested Python sequences:

```
>>> a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
>>> a
array([[ 1,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

In NumPy, a dimension of an array is sometimes referred to as an ”axis”. This terminology may be useful to disambiguate between the dimensionality of an array and the dimensionality of the data represented by the array. For instance, the array `a` could represent three points, each lying within a four-dimensional space, but `a` has only two ”axes”.

Another difference between an array and a list of lists is that an element of the array can be accessed by specifying the index along each axis within a *single* set of square brackets, separated by commas. For instance, the element `8` is in row `1` and column `3`:

```
>>> a[1, 3]
8
```

> Note
> It is familiar practice in mathematics to refer to elements of a matrix by the row index first and the column index second. This happens to be true for two-dimensional arrays, but a better mental model is to think of the column index as coming *last* and the row index as *second to last*. This generalizes to arrays with *any* number of dimensions.

> Note
> You might hear of a 0-D (zero-dimensional) array referred to as a ”scalar”, a 1-D (one-dimensional) array as a ”vector”, a 2-D (two-dimensional) array as a ”matrix”, or an N-D (N-dimensional, where ”N” is typically an integer greater than 2) array as a ”tensor”. For clarity, it is best to avoid the mathematical terms when referring to an array because the mathematical objects with these names behave differently than arrays (e.g. ”matrix” multiplication is fundamentally different from ”array” multiplication), and there are other objects in the scientific Python ecosystem that have these names (e.g. the fundamental data structure of PyTorch is the ”tensor”).

Array attributes

This section covers the `ndim`, `shape`, `size`, *and* `dtype` *attributes of an array.*

The number of dimensions of an array is contained in the `ndim` attribute.

```
>>> a.ndim
2
```

The shape of an array is a tuple of non-negative integers that specify the number of elements along each dimension.

```
>>> a.shape
(3, 4)
>>> len(a.shape) == a.ndim
True
```

The fixed, total number of elements in array is contained in the `size` attribute.

```
>>> a.size
12
>>> import math
>>> a.size == math.prod(a.shape)
True
```

Arrays are typically ”homogeneous”, meaning that they contain elements of only one ”data type”. The data type is recorded in the `dtype` attribute.

```
>>> a.dtype
dtype('int64')  # "int" for integer, "64" for 64-bit
```

Read more about array attributes here and learn about array objects here.

How to create a basic array

This section covers `np.zeros()`, `np.ones()`, `np.empty()`, `np.arange()`, `np.linspace()`

Besides creating an array from a sequence of elements, you can easily create an array filled with `0`'s:

```
>>> np.zeros(2)
array([0., 0.])
```

Or an array filled with `1`'s:

```
>>> np.ones(2)
array([1., 1.])
```

Or even an empty array! The function `empty` creates an array whose initial content is random and depends on the state of the memory. The reason to use `empty` over `zeros` (or something similar) is speed - just make sure to fill every element afterwards! :

```
>>> # Create an empty array with 2 elements
>>> np.empty(2)
array([3.14, 42.  ])  # may vary
```

You can create an array with a range of elements:

```
>>> np.arange(4)
array([0, 1, 2, 3])
```

And even an array that contains a range of evenly spaced intervals. To do this, you will specify the **first number**, **last number**, and the **step size**. :

```
>>> np.arange(2, 9, 2)
array([2, 4, 6, 8])
```

You can also use `np.linspace()` to create an array with values that are spaced linearly in a specified interval:

```
>>> np.linspace(0, 10, num=5)
array([ 0. ,  2.5,  5. ,  7.5, 10. ])
```

Specifying your data type

While the default data type is floating point (`np.float64`), you can explicitly specify which data type you want using the `dtype` keyword. :

```
>>> x = np.ones(2, dtype=np.int64)
>>> x
array([1, 1])
```

Learn more about creating arrays here

Adding, removing, and sorting elements

This section covers `np.sort()`, `np.concatenate()`

Sorting an array is simple with `np.sort()`. You can specify the axis, kind, and order when you call the function.

If you start with this array:

```
>>> arr = np.array([2, 1, 5, 3, 7, 4, 6, 8])
```

You can quickly sort the numbers in ascending order with:

```
>>> np.sort(arr)
array([1, 2, 3, 4, 5, 6, 7, 8])
```

In addition to sort, which returns a sorted copy of an array, you can use:

- argsort, which is an indirect sort along a specified axis,
- lexsort, which is an indirect stable sort on multiple keys,
- searchsorted, which will find elements in a sorted array, and
- partition, which is a partial sort.

To read more about sorting an array, see: sort.

If you start with these arrays:

```
>>> a = np.array([1, 2, 3, 4])
>>> b = np.array([5, 6, 7, 8])
```

You can concatenate them with `np.concatenate()`. :

```
>>> np.concatenate((a, b))
array([1, 2, 3, 4, 5, 6, 7, 8])
```

Or, if you start with these arrays:

```
>>> x = np.array([[1, 2], [3, 4]])
>>> y = np.array([[5, 6]])
```

You can concatenate them with:

```
>>> np.concatenate((x, y), axis=0)
array([[1, 2],
       [3, 4],
       [5, 6]])
```

In order to remove elements from an array, it's simple to use indexing to select the elements that you want to keep.

To read more about concatenate, see: concatenate.

How do you know the shape and size of an array?

This section covers `ndarray.ndim`, `ndarray.size`, `ndarray.shape`

`ndarray.ndim` will tell you the number of axes, or dimensions, of the array.

`ndarray.size` will tell you the total number of elements of the array. This is the *product* of the elements of the array's shape.

`ndarray.shape` will display a tuple of integers that indicate the number of elements stored along each dimension of the array. If, for example, you have a 2-D array with 2 rows and 3 columns, the shape of your array is `(2, 3)`.

For example, if you create this array:

```
>>> array_example = np.array([[[0, 1, 2, 3],
...                            [4, 5, 6, 7]],
...
...                           [[0, 1, 2, 3],
...                            [4, 5, 6, 7]],
...
...                           [[0 ,1 ,2, 3],
...                            [4, 5, 6, 7]]])
```

To find the number of dimensions of the array, run:

```
>>> array_example.ndim
3
```

To find the total number of elements in the array, run:

```
>>> array_example.size
24
```

And to find the shape of your array, run:

```
>>> array_example.shape
(3, 2, 4)
```

Can you reshape an array?

This section covers `arr.reshape()`

Yes!

Using `arr.reshape()` will return a reshaped array without changing the data. Just remember that when you use the reshape method, the array you want to produce needs to have the same number of elements as the original array. If you start with an array with 12 elements, you'll need to make sure that your new array also has a total of 12 elements.

If you start with this array:

```
>>> a = np.arange(6)
>>> print(a)
[0 1 2 3 4 5]
```

You can use `reshape()` to reshape your array. For example, you can reshape this array to an array with three rows and two columns:

```
>>> b = a.reshape(3, 2)
>>> print(b)
[[0 1]
 [2 3]
 [4 5]]
```

With `np.reshape`, you can specify a few optional parameters:

```
>>> np.reshape(a, shape=(1, 6), order='C')
array([[0, 1, 2, 3, 4, 5]])
```

`a` is the array to be reshaped.

`shape` is the new shape you want. You can specify an integer or a tuple of integers. If you specify an integer, the result will be an array of that length. The shape should be compatible with the original shape.

`order: C` means to read/write the elements using C-like index order, `F` means to read/write the elements using Fortran-like index order, `A` means to read/write the elements in Fortran-like index order if a is Fortran contiguous in memory, C-like order otherwise. (This is an optional parameter and doesn't need to be specified.)

If you want to learn more about C and Fortran order, you can read more about the internal organization of NumPy arrays here. Essentially, C and Fortran orders have to do with how indices correspond to the order the array is stored in memory. In Fortran, when moving through the elements of a two-dimensional array as it is stored in memory, the **first** index is the most rapidly varying index. As the first index moves to the next row as it changes, the matrix is stored one column at a time. This is why Fortran is thought of as a **Column-major language**. In C on the other hand, the **last** index changes the most rapidly. The matrix is stored by rows, making it a **Row-major language**. What you do for C or Fortran depends on whether it's more important to preserve the indexing convention or not reorder the data.

Learn more about shape manipulation here.

How to convert a 1D array into a 2D array (how to add a new axis to an array)

This section covers `np.newaxis`, `np.expand_dims`

You can use `np.newaxis` and `np.expand_dims` to increase the dimensions of your existing array.

Using `np.newaxis` will increase the dimensions of your array by one dimension when used once. This means that a **1D** array will become a **2D** array, a **2D** array will become a **3D** array, and so on.

For example, if you start with this array:

```
>>> a = np.array([1, 2, 3, 4, 5, 6])
>>> a.shape
(6,)
```

You can use `np.newaxis` to add a new axis:

```
>>> a2 = a[np.newaxis, :]
>>> a2.shape
(1, 6)
```

You can explicitly convert a 1D array to either a row vector or a column vector using `np.newaxis`. For example, you can convert a 1D array to a row vector by inserting an axis along the first dimension:

```
>>> row_vector = a[np.newaxis, :]
>>> row_vector.shape
(1, 6)
```

Or, for a column vector, you can insert an axis along the second dimension:

```
>>> col_vector = a[:, np.newaxis]
>>> col_vector.shape
(6, 1)
```

You can also expand an array by inserting a new axis at a specified position with `np.expand_dims`.

For example, if you start with this array:

```
>>> a = np.array([1, 2, 3, 4, 5, 6])
>>> a.shape
(6,)
```

You can use `np.expand_dims` to add an axis at index position 1 with:

```
>>> b = np.expand_dims(a, axis=1)
>>> b.shape
(6, 1)
```

You can add an axis at index position 0 with:

```
>>> c = np.expand_dims(a, axis=0)
>>> c.shape
(1, 6)
```

Find more information about newaxis here and `expand_dims` at expand_dims.

Indexing and slicing

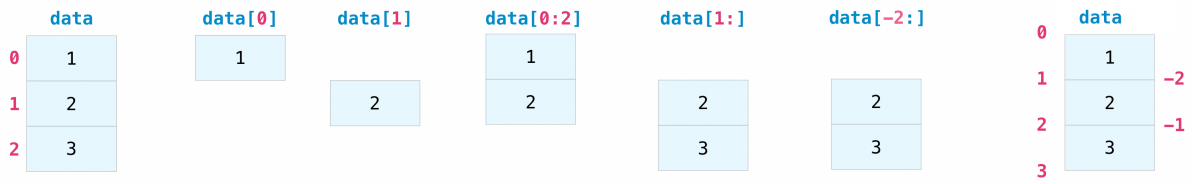
You can index and slice NumPy arrays in the same ways you can slice Python lists. :

```
>>> data = np.array([1, 2, 3])

>>> data[1]
2
>>> data[0:2]
array([1, 2])
```

```
>>> data[1:]
array([2, 3])
>>> data[-2:]
array([2, 3])
```

You can visualize it this way:



You may want to take a section of your array or specific array elements to use in further analysis or additional operations. To do that, you'll need to subset, slice, and/or index your arrays.

If you want to select values from your array that fulfill certain conditions, it's straightforward with NumPy.

For example, if you start with this array:

```
>>> a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

You can easily print all of the values in the array that are less than 5. :

```
>>> print(a[a < 5])
[1 2 3 4]
```

You can also select, for example, numbers that are equal to or greater than 5, and use that condition to index an array. :

```
>>> five_up = (a >= 5)
>>> print(a[five_up])
[ 5  6  7  8  9 10 11 12]
```

You can select elements that are divisible by 2:

```
>>> divisible_by_2 = a[a%2==0]
>>> print(divisible_by_2)
[ 2  4  6  8 10 12]
```

Or you can select elements that satisfy two conditions using the `&` and `|` operators:

```
>>> c = a[(a > 2) & (a < 11)]
>>> print(c)
[ 3  4  5  6  7  8  9 10]
```

You can also make use of the logical operators **&** and **|** in order to return boolean values that specify whether or not the values in an array fulfill a certain condition. This can be useful with arrays that contain names or other categorical values. :

```
>>> five_up = (a > 5) | (a == 5)
>>> print(five_up)
[[False False False False]
 [ True  True  True  True]
 [ True  True  True True]]
```

You can also use `np.nonzero()` to select elements or indices from an array.

Starting with this array:

```
>>> a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

You can use `np.nonzero()` to print the indices of elements that are, for example, less than 5:

```
>>> b = np.nonzero(a < 5)
>>> print(b)
(array([0, 0, 0, 0]), array([0, 1, 2, 3]))
```

In this example, a tuple of arrays was returned: one for each dimension. The first array represents the row indices where these values are found, and the second array represents the column indices where the values are found.

If you want to generate a list of coordinates where the elements exist, you can zip the arrays, iterate over the list of coordinates, and print them. For example:

```
>>> list_of_coordinates= list(zip(b[0], b[1]))

>>> for coord in list_of_coordinates:
...     print(coord)
(np.int64(0), np.int64(0))
(np.int64(0), np.int64(1))
(np.int64(0), np.int64(2))
(np.int64(0), np.int64(3))
```

You can also use `np.nonzero()` to print the elements in an array that are less than 5 with:

```
>>> print(a[b])
[1 2 3 4]
```

If the element you're looking for doesn't exist in the array, then the returned array of indices will be empty. For example:

```
>>> not_there = np.nonzero(a == 42)
>>> print(not_there)
(array([], dtype=int64), array([], dtype=int64))
```

Learn more about indexing and slicing here and here.

Read more about using the nonzero function at: nonzero.

How to create an array from existing data

This section covers `slicing and indexing`, `np.vstack()`, `np.hstack()`, `np.hsplit()`, `.view()`, `copy()`

You can easily create a new array from a section of an existing array.

Let's say you have this array:

```
>>> a = np.array([1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

You can create a new array from a section of your array any time by specifying where you want to slice your array. :

```
>>> arr1 = a[3:8]
>>> arr1
array([4, 5, 6, 7, 8])
```

Here, you grabbed a section of your array from index position 3 through index position 8 but not including position 8 itself.

Reminder: Array indexes begin at 0. This means the first element of the array is at index 0, the second element is at index 1, and so on.

You can also stack two existing arrays, both vertically and horizontally. Let's say you have two arrays, `a1` and `a2`:

```
>>> a1 = np.array([[1, 1],
...                [2, 2]])

>>> a2 = np.array([[3, 3],
...                [4, 4]])
```

You can stack them vertically with `vstack`:

```
>>> np.vstack((a1, a2))
array([[1, 1],
       [2, 2],
       [3, 3],
       [4, 4]])
```

Or stack them horizontally with `hstack`:

```
>>> np.hstack((a1, a2))
array([[1, 1, 3, 3],
       [2, 2, 4, 4]])
```

You can split an array into several smaller arrays using `hsplit`. You can specify either the number of equally shaped arrays to return or the columns *after* which the division should occur.

Let's say you have this array:

```
>>> x = np.arange(1, 25).reshape(2, 12)
>>> x
array([[ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12],
       [13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24]])
```

If you wanted to split this array into three equally shaped arrays, you would run:

```
>>> np.hsplit(x, 3)
  [array([[ 1,  2,  3,  4],
         [13, 14, 15, 16]]), array([[ 5,  6,  7,  8],
         [17, 18, 19, 20]]), array([[ 9, 10, 11, 12],
         [21, 22, 23, 24]])]
```

If you wanted to split your array after the third and fourth column, you'd run:

```
>>> np.hsplit(x, (3, 4))
  [array([[ 1,  2,  3],
         [13, 14, 15]]), array([[ 4],
         [16]]), array([[ 5,  6,  7,  8,  9, 10, 11, 12],
         [17, 18, 19, 20, 21, 22, 23, 24]])]
```

Learn more about stacking and splitting arrays here.

You can use the `view` method to create a new array object that looks at the same data as the original array (a *shallow copy*).

Views are an important NumPy concept! NumPy functions, as well as operations like indexing and slicing, will return views whenever possible. This saves memory and is faster (no copy of the data has to be made). However it's important to be aware of this - modifying data in a view also modifies the original array!

Let's say you create this array:

```
>>> a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

Now we create an array `b1` by slicing `a` and modify the first element of `b1`. This will modify the corresponding element in `a` as well! :

```
>>> b1 = a[0, :]
>>> b1
array([1, 2, 3, 4])
>>> b1[0] = 99
>>> b1
array([99,  2,  3,  4])
>>> a
array([[99,  2,  3,  4],
       [ 5,  6,  7,  8],
       [ 9, 10, 11, 12]])
```

Using the `copy` method will make a complete copy of the array and its data (a *deep copy*). To use this on your array, you could run:

```
>>> b2 = a.copy()
```

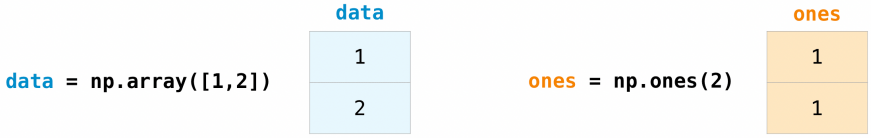
Learn more about copies and views here.

Basic array operations

This section covers addition, subtraction, multiplication, division, and more

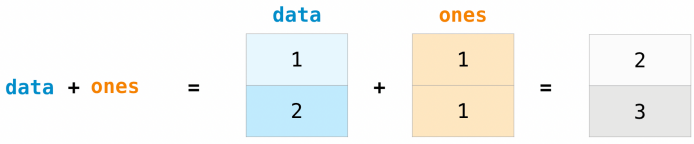
---

Once you've created your arrays, you can start to work with them. Let's say, for example, that you've created two arrays, one called ”data” and one called ”ones”



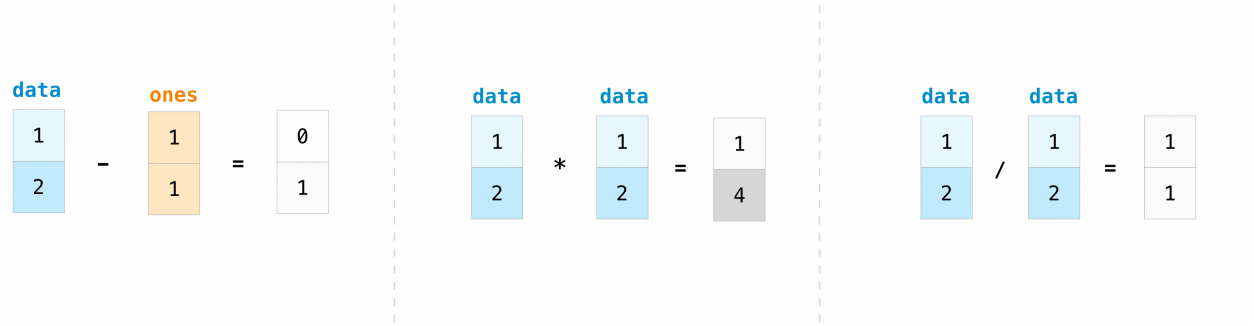
You can add the arrays together with the plus sign.

```
>>> data = np.array([1, 2])
>>> ones = np.ones(2, dtype=np.int_)
>>> data + ones
array([2, 3])
```



You can, of course, do more than just addition!

```
>>> data - ones
array([0, 1])
>>> data * data
array([1, 4])
>>> data / data
array([1., 1.])
```



Basic operations are simple with NumPy. If you want to find the sum of the elements in an array, you'd use `sum()`. This works for 1D arrays, 2D arrays, and arrays in higher dimensions. :

```
>>> a = np.array([1, 2, 3, 4])

>>> a.sum()
10
```

To add the rows or the columns in a 2D array, you would specify the axis.

If you start with this array:

```
>>> b = np.array([[1, 1], [2, 2]])
```

You can sum over the axis of rows with:

```
>>> b.sum(axis=0)
array([3, 3])
```

You can sum over the axis of columns with:

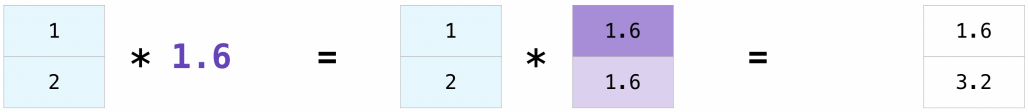
```
>>> b.sum(axis=1)
array([2, 4])
```

Learn more about basic operations here.

Broadcasting

There are times when you might want to carry out an operation between an array and a single number (also called *an operation between a vector and a scalar*) or between arrays of two different sizes. For example, your array (we'll call it "data") might contain information about distance in miles but you want to convert the information to kilometers. You can perform this operation with:

```
>>> data = np.array([1.0, 2.0])
>>> data * 1.6
array([1.6, 3.2])
```



NumPy understands that the multiplication should happen with each cell. That concept is called **broadcasting**. Broadcasting is a mechanism that allows NumPy to perform operations on arrays of different shapes. The dimensions of your array must be compatible, for example, when the dimensions of both arrays are equal or when one of them is 1. If the dimensions are not compatible, you will get a `ValueError`.

Learn more about broadcasting here.

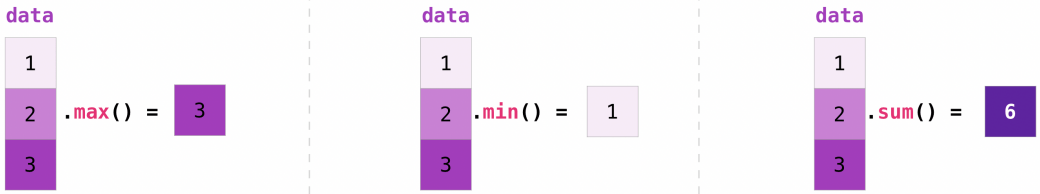
More useful array operations

This section covers maximum, minimum, sum, mean, product, standard deviation, and more

---

NumPy also performs aggregation functions. In addition to `min`, `max`, and `sum`, you can easily run `mean` to get the average, `prod` to get the result of multiplying the elements together, `std` to get the standard deviation, and more. :

```
>>> data = np.array([1, 2, 3])
>>> data.max()
3
>>> data.min()
1
>>> data.sum()
6
```



Let's start with this array, called "a" :

```
>>> a = np.array([[0.45053314, 0.17296777, 0.34376245, 0.5510652],
...               [0.54627315, 0.05093587, 0.40067661, 0.55645993],
...               [0.12697628, 0.82485143, 0.26590556, 0.56917101]])
```

It's very common to want to aggregate along a row or column. By default, every NumPy aggregation function will return the aggregate of the entire array. To find the sum or the minimum of the elements in your array, run:

```
>>> a.sum()
4.8595784
```

Or:

```
>>> a.min()
0.05093587
```

You can specify on which axis you want the aggregation function to be computed. For example, you can find the minimum value within each column by specifying `axis=0`. :

```
>>> a.min(axis=0)
array([0.12697628, 0.05093587, 0.26590556, 0.5510652 ])
```

The four values listed above correspond to the number of columns in your array. With a four-column array, you will get four values as your result.

Read more about array methods here.

Creating matrices

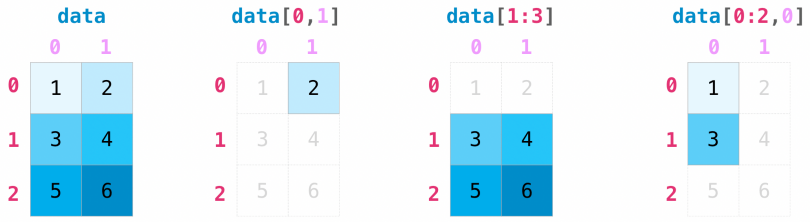
You can pass Python lists of lists to create a 2-D array (or ”matrix”) to represent them in NumPy. :

```
>>> data = np.array([[1, 2], [3, 4], [5, 6]])
>>> data
array([[1, 2],
       [3, 4],
       [5, 6]])
```



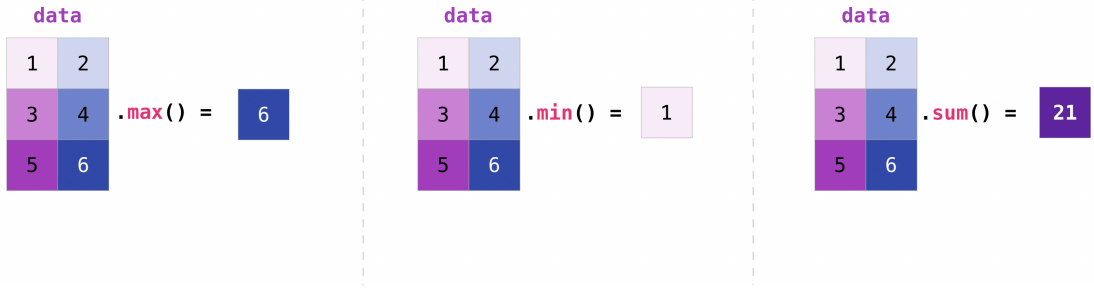
Indexing and slicing operations are useful when you're manipulating matrices:

```
>>> data[0, 1]
2
>>> data[1:3]
array([[3, 4],
       [5, 6]])
>>> data[0:2, 0]
array([1, 3])
```



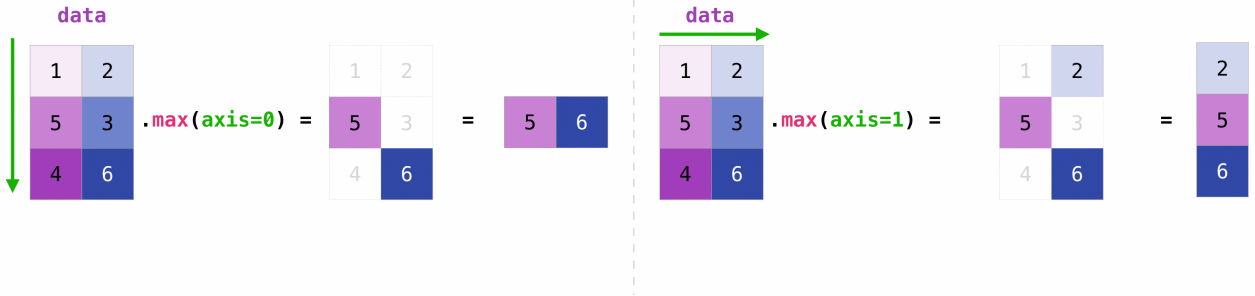
You can aggregate matrices the same way you aggregated vectors:

```
>>> data.max()
6
>>> data.min()
1
>>> data.sum()
21
```



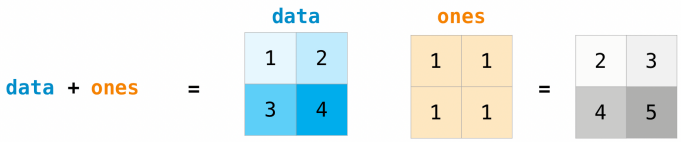
You can aggregate all the values in a matrix and you can aggregate them across columns or rows using the `axis` parameter. To illustrate this point, let's look at a slightly modified dataset:

```
>>> data = np.array([[1, 2], [5, 3], [4, 6]])
>>> data
array([[1, 2],
       [5, 3],
       [4, 6]])
>>> data.max(axis=0)
array([5, 6])
>>> data.max(axis=1)
array([2, 5, 6])
```



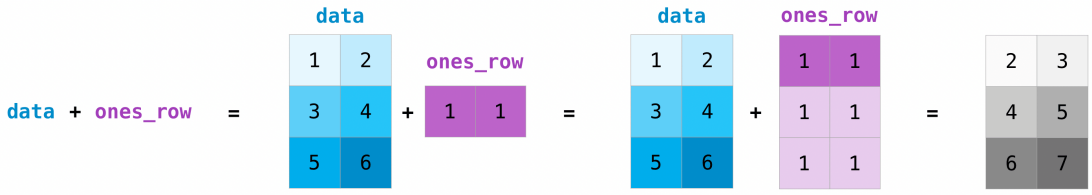
Once you've created your matrices, you can add and multiply them using arithmetic operators if you have two matrices that are the same size. :

```
>>> data = np.array([[1, 2], [3, 4]])
>>> ones = np.array([[1, 1], [1, 1]])
>>> data + ones
array([[2, 3],
       [4, 5]])
```



You can do these arithmetic operations on matrices of different sizes, but only if one matrix has only one column or one row. In this case, NumPy will use its broadcast rules for the operation. :

```
>>> data = np.array([[1, 2], [3, 4], [5, 6]])
>>> ones_row = np.array([[1, 1]])
>>> data + ones_row
array([[2, 3],
       [4, 5],
       [6, 7]])
```



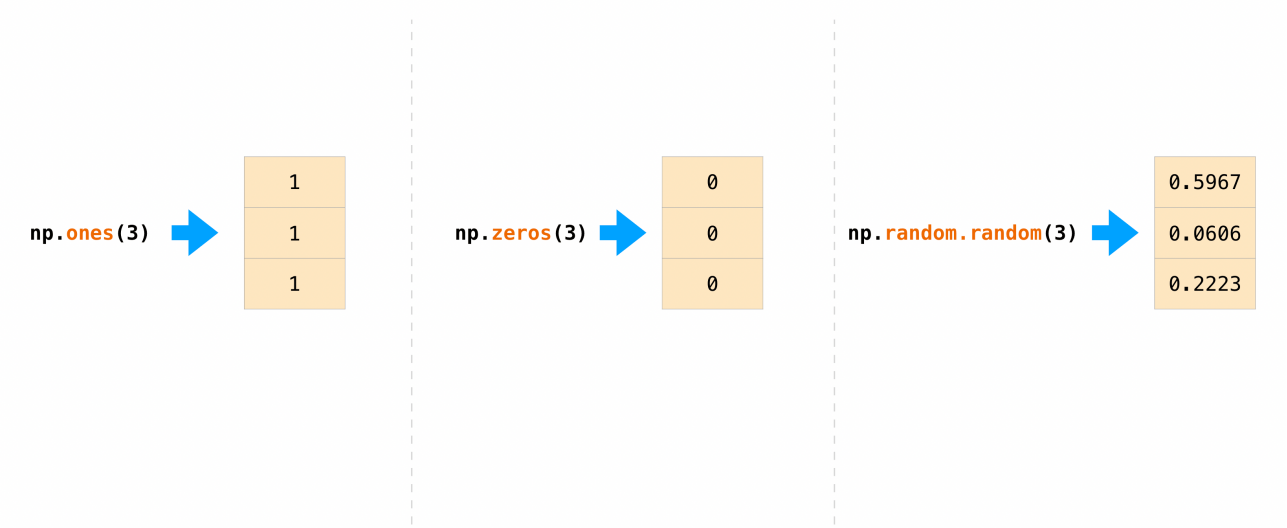
Be aware that when NumPy prints N-dimensional arrays, the last axis is looped over the fastest while the first axis is the slowest. For instance:

```
>>> np.ones((4, 3, 2))
array([[[1., 1.],
        [1., 1.],
        [1., 1.]],
<BLANKLINE>
       [[1., 1.],
        [1., 1.],
        [1., 1.]],
<BLANKLINE>
       [[1., 1.],
        [1., 1.],
        [1., 1.]],
<BLANKLINE>
       [[1., 1.],
        [1., 1.],
        [1., 1.]]])
```

There are often instances where we want NumPy to initialize the values of an array. NumPy offers functions like `ones()` and `zeros()`, and the `random.Generator` class for random number generation for that. All you need to do is pass in the number of elements you want it to generate:

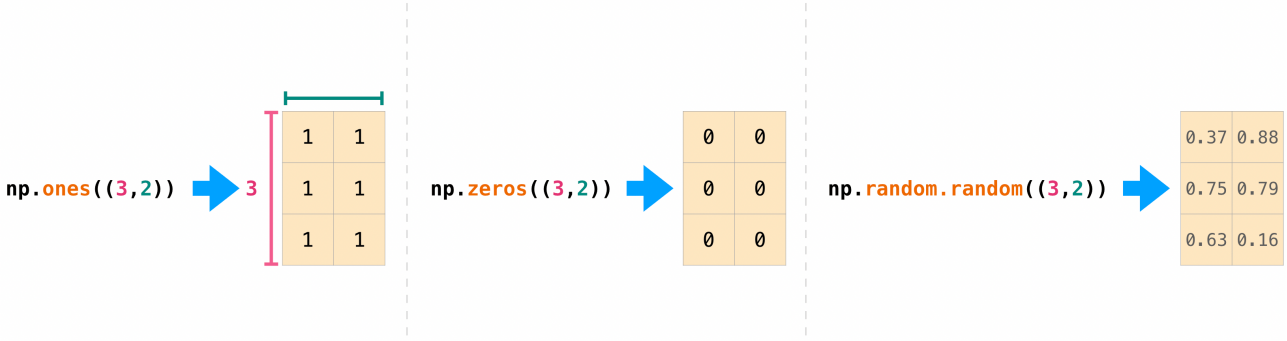
```
>>> np.ones(3)
array([1., 1., 1.])
>>> np.zeros(3)
```

```
array([0., 0., 0.])
>>> rng = np.random.default_rng()  # the simplest way to generate random numbers
>>> rng.random(3)
array([0.63696169, 0.26978671, 0.04097352])
```



You can also use `ones()`, `zeros()`, and `random()` to create a 2D array if you give them a tuple describing the dimensions of the matrix:

```
>>> np.ones((3, 2))
array([[1., 1.],
       [1., 1.],
       [1., 1.]])
>>> np.zeros((3, 2))
array([[0., 0.],
       [0., 0.],
       [0., 0.]])
>>> rng.random((3, 2))
array([[0.01652764, 0.81327024],
       [0.91275558, 0.60663578],
       [0.72949656, 0.54362499]])  # may vary
```



Read more about creating arrays, filled with `0`'s, `1`'s, other values or uninitialized, at array creation routines.

Generating random numbers

The use of random number generation is an important part of the configuration and evaluation of many numerical and machine learning algorithms. Whether you need to randomly initialize weights in an artificial neural network, split data into random sets, or randomly shuffle your dataset, being able to generate random numbers (actually, repeatable pseudo-random numbers) is essential.

With `Generator.integers`, you can generate random integers from low (remember that this is inclusive with NumPy) to high (exclusive). You can set `endpoint=True` to make the high number inclusive.

You can generate a 2 x 4 array of random integers between 0 and 4 with:

```
>>> rng.integers(5, size=(2, 4))
array([[2, 1, 1, 0],
       [0, 0, 0, 4]])  # may vary
```

Read more about random number generation here.

How to get unique items and counts

This section covers `np.unique()`

You can find the unique elements in an array easily with `np.unique`.

For example, if you start with this array:

```
>>> a = np.array([11, 11, 12, 13, 14, 15, 16, 17, 12, 13, 11, 14, 18, 19, 20])
```

you can use `np.unique` to print the unique values in your array:

```
>>> unique_values = np.unique(a)
>>> print(unique_values)
[11 12 13 14 15 16 17 18 19 20]
```

To get the indices of unique values in a NumPy array (an array of first index positions of unique values in the array), just pass the `return_index` argument in `np.unique()` as well as your array. :

```
>>> unique_values, indices_list = np.unique(a, return_index=True)
>>> print(indices_list)
[ 0  2  3  4  5  6  7 12 13 14]
```

You can pass the `return_counts` argument in `np.unique()` along with your array to get the frequency count of unique values in a NumPy array. :

```
>>> unique_values, occurrence_count = np.unique(a, return_counts=True)
>>> print(occurrence_count)
[3 2 2 2 1 1 1 1 1 1]
```

This also works with 2D arrays! If you start with this array:

```
>>> a_2d = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12], [1, 2, 3, 4]])
```

You can find unique values with:

```
>>> unique_values = np.unique(a_2d)
>>> print(unique_values)
[ 1  2  3  4  5  6  7  8  9 10 11 12]
```

If the axis argument isn't passed, your 2D array will be flattened.

If you want to get the unique rows or columns, make sure to pass the `axis` argument. To find the unique rows, specify `axis=0` and for columns, specify `axis=1`. :

```
>>> unique_rows = np.unique(a_2d, axis=0)
>>> print(unique_rows)
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

To get the unique rows, index position, and occurrence count, you can use:

```
>>> unique_rows, indices, occurrence_count = np.unique(
...      a_2d, axis=0, return_counts=True, return_index=True)
>>> print(unique_rows)
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
>>> print(indices)
[0 1 2]
>>> print(occurrence_count)
[2 1 1]
```

To learn more about finding the unique elements in an array, see unique.

Transposing and reshaping a matrix

This section covers `arr.reshape()`, `arr.transpose()`, `arr.T`

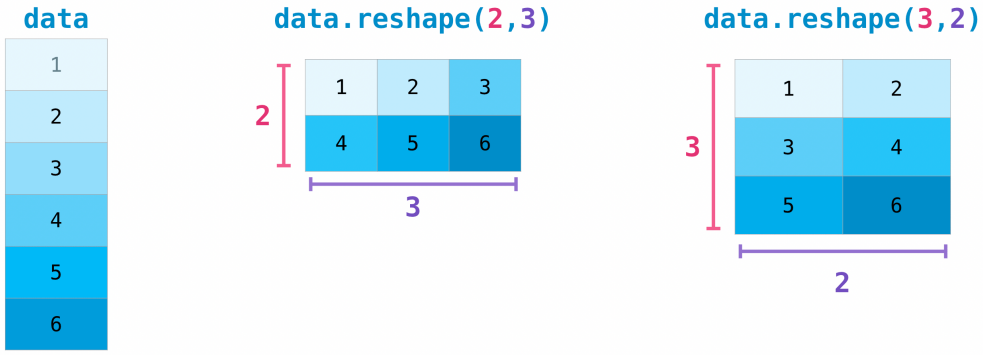
It's common to need to transpose your matrices. NumPy arrays have the property `T` that allows you to transpose a matrix.



You may also need to switch the dimensions of a matrix. This can happen when, for example, you have a model that expects a certain input shape that is different from your dataset. This is where the `reshape` method can be useful. You simply need to pass in the new dimensions that you want for the matrix. :

```
>>> data.reshape(2, 3)
array([[1, 2, 3],
       [4, 5, 6]])
>>> data.reshape(3, 2)
array([[1, 2],
       [3, 4],
```

```
       [5, 6]])
```



You can also use `.transpose()` to reverse or change the axes of an array according to the values you specify.

If you start with this array:

```
>>> arr = np.arange(6).reshape((2, 3))
>>> arr
array([[0, 1, 2],
       [3, 4, 5]])
```

You can transpose your array with `arr.transpose()`. :

```
>>> arr.transpose()
array([[0, 3],
       [1, 4],
       [2, 5]])
```

You can also use `arr.T`:

```
>>> arr.T
array([[0, 3],
       [1, 4],
       [2, 5]])
```

To learn more about transposing and reshaping arrays, see transpose and reshape.

How to reverse an array

This section covers `np.flip()`

NumPy's `np.flip()` function allows you to flip, or reverse, the contents of an array along an axis. When using `np.flip()`, specify the array you would like to reverse and the axis. If you don't specify the axis, NumPy will reverse the contents along all of the axes of your input array.

Reversing a 1D array

If you begin with a 1D array like this one:

```
>>> arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

You can reverse it with:

```
>>> reversed_arr = np.flip(arr)
```

If you want to print your reversed array, you can run:

```
>>> print('Reversed Array: ', reversed_arr)
Reversed Array:  [8 7 6 5 4 3 2 1]
```

Reversing a 2D array

A 2D array works much the same way.

If you start with this array:

```
>>> arr_2d = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

You can reverse the content in all of the rows and all of the columns with:

```
>>> reversed_arr = np.flip(arr_2d)
>>> print(reversed_arr)
[[12 11 10  9]
 [ 8  7  6  5]
 [ 4  3  2  1]]
```

You can easily reverse only the *rows* with:

```
>>> reversed_arr_rows = np.flip(arr_2d, axis=0)
>>> print(reversed_arr_rows)
[[ 9 10 11 12]
 [ 5  6  7  8]
 [ 1  2  3  4]]
```

Or reverse only the *columns* with:

```
>>> reversed_arr_columns = np.flip(arr_2d, axis=1)
>>> print(reversed_arr_columns)
[[ 4  3  2  1]
 [ 8  7  6  5]
 [12 11 10  9]]
```

You can also reverse the contents of only one column or row. For example, you can reverse the contents of the row at index position 1 (the second row):

```
>>> arr_2d[1] = np.flip(arr_2d[1])
>>> print(arr_2d)
[[ 1  2  3  4]
 [ 8  7  6  5]
 [ 9 10 11 12]]
```

You can also reverse the column at index position 1 (the second column):

```
>>> arr_2d[:,1] = np.flip(arr_2d[:,1])
>>> print(arr_2d)
[[ 1 10  3  4]
 [ 8  7  6  5]
 [ 9  2 11 12]]
```

Read more about reversing arrays at flip.

Reshaping and flattening multidimensional arrays

This section covers `.flatten()`, `ravel()`

There are two popular ways to flatten an array: `.flatten()` and `.ravel()`. The primary difference between the two is that the new array created using `ravel()` is actually a reference to the parent array (i.e., a ”view”). This means that any changes to the new array will affect the parent array as well. Since `ravel` does not create a copy, it's memory efficient.

If you start with this array:

```
>>> x = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])
```

You can use `flatten` to flatten your array into a 1D array. :

```
>>> x.flatten()
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12])
```

When you use `flatten`, changes to your new array won't change the parent array.

For example:

```
>>> a1 = x.flatten()
>>> a1[0] = 99
>>> print(x)  # Original array
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
>>> print(a1)  # New array
[99  2  3  4  5  6  7  8  9 10 11 12]
```

But when you use `ravel`, the changes you make to the new array will affect the parent array.

For example:

```
>>> a2 = x.ravel()
>>> a2[0] = 98
>>> print(x)  # Original array
[[98  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
>>> print(a2)  # New array
[98  2  3  4  5  6  7  8  9 10 11 12]
```

Read more about `flatten` at ndarray.flatten and `ravel` at ravel.

How to access the docstring for more information

This section covers `help()`, `?`, `??`

When it comes to the data science ecosystem, Python and NumPy are built with the user in mind. One of the best examples of this is the built-in access to documentation. Every object contains the reference to a string, which is known as the **docstring**. In most cases, this docstring contains a quick and concise summary of the object and how to use it. Python has a built-in `help()` function that can help you access this information. This means that nearly any time you need more information, you can use `help()` to quickly find the information that you need.

For example:

```
>>> help(max)
Help on built-in function max in module builtins:
<BLANKLINE>
max(...)
```

```
    max(iterable, *[, default=obj, key=func]) -> value
    max(arg1, arg2, *args, *[, key=func]) -> value
<BLANKLINE>
    With a single iterable argument, return its biggest item. The
    default keyword-only argument specifies an object to return if
    the provided iterable is empty.
    With two or more ...arguments, return the largest argument.
<BLANKLINE>
```

Because access to additional information is so useful, IPython uses the `?` character as a shorthand for accessing this documentation along with other relevant information. IPython is a command shell for interactive computing in multiple languages. You can find more information about IPython here.

For example:

```
In [0]: max?
max(iterable, *[, default=obj, key=func]) -> value
max(arg1, arg2, *args, *[, key=func]) -> value

With a single iterable argument, return its biggest item. The
default keyword-only argument specifies an object to return if
the provided iterable is empty.
With two or more arguments, return the largest argument.
Type:      builtin_function_or_method
```

You can even use this notation for object methods and objects themselves.

Let's say you create this array:

```
>>> a = np.array([1, 2, 3, 4, 5, 6])
```

Then you can obtain a lot of useful information (first details about `a` itself, followed by the docstring of `ndarray` of which `a` is an instance):

```
In [1]: a?
Type:            ndarray
String form:     [1 2 3 4 5 6]
Length:          6
File:            ~/anaconda3/lib/python3.9/site-packages/numpy/__init__.py
Docstring:       <no docstring>
Class docstring:
ndarray(shape, dtype=float, buffer=None, offset=0,
        strides=None, order=None)

An array object represents a multidimensional, homogeneous array
of fixed-size items.  An associated data-type object describes the
format of each element in the array (its byte-order, how many bytes it
occupies in memory, whether it is an integer, a floating point number,
or something else, etc.)

Arrays should be constructed using `array`, `zeros` or `empty` (refer
to the See Also section below).  The parameters given here refer to
a low-level method (`ndarray(...)`) for instantiating an array.

For more information, refer to the `numpy` module and examine the
methods and attributes of an array.

Parameters
----------
(for the __new__ method; see Notes below)
```

```
shape : tuple of ints
        Shape of created array.
...
```

This also works for functions and other objects that **you** create. Just remember to include a docstring with your function using a string literal (`""" """` or `''' '''` around your documentation).

For example, if you create this function:

```
>>> def double(a):
...    '''Return a * 2'''
...    return a * 2
```

You can obtain information about the function:

```
In [2]: double?
Signature: double(a)
Docstring: Return a * 2
File:      ~/Desktop/<ipython-input-23-b5adf20be596>
Type:      function
```

You can reach another level of information by reading the source code of the object you're interested in. Using a double question mark (`??`) allows you to access the source code.

For example:

```
In [3]: double??
Signature: double(a)
Source:
def double(a):
    '''Return a * 2'''
    return a * 2
File:      ~/Desktop/<ipython-input-23-b5adf20be596>
Type:      function
```

If the object in question is compiled in a language other than Python, using `??` will return the same information as `?`. You'll find this with a lot of built-in objects and types, for example:

```
In [4]: len?
Signature: len(obj, /)
Docstring: Return the number of items in a container.
Type:      builtin_function_or_method
```

and :

```
In [5]: len??
Signature: len(obj, /)
Docstring: Return the number of items in a container.
Type:      builtin_function_or_method
```

have the same output because they were compiled in a programming language other than Python.

Working with mathematical formulas

The ease of implementing mathematical formulas that work on arrays is one of the things that make NumPy so widely used in the scientific Python community.

For example, this is the mean square error formula (a central formula used in supervised machine learning models that deal with regression):

$$MeanSquareError = \frac{1}{n}\sum_{i=1}^{n}(Y_prediction_i - Y_i)^2$$

Implementing this formula is simple and straightforward in NumPy:

```
error = (1/n) * np.sum(np.square(predictions - labels))
```

What makes this work so well is that `predictions` and `labels` can contain one or a thousand values. They only need to be the same size.

You can visualize it this way:

```
                                  predictions  labels
error = (1/3) * np.sum(np.square(    1          1    ))
                                     1    -     2
                                     1          3
```

In this example, both the predictions and labels vectors contain three values, meaning `n` has a value of three. After we carry out subtractions the values in the vector are squared. Then NumPy sums the values, and your result is the error value for that prediction and a score for the quality of the model.

```
error = (1/3) * np.sum(np.square(    0    ))
                                    -1
                                    -2

error = (1/3) * np.sum(              0     )
                                     1
                                     4
```

```
    error = (1/3) * 5
```

How to save and load NumPy objects

This section covers `np.save`, `np.savez`, `np.savetxt`, `np.load`, `np.loadtxt`

You will, at some point, want to save your arrays to disk and load them back without having to re-run the code. Fortunately, there are several ways to save and load objects with NumPy. The ndarray objects can be saved to and loaded from the disk files with `loadtxt` and `savetxt` functions that handle normal text files, `load` and `save` functions that handle NumPy binary files with a **.npy** file extension, and a `savez` function that handles NumPy files with a **.npz** file extension.

The **.npy** and **.npz** files store data, shape, dtype, and other information required to reconstruct the ndarray in a way that allows the array to be correctly retrieved, even when the file is on another machine with different architecture.

If you want to store a single ndarray object, store it as a .npy file using `np.save`. If you want to store more than one ndarray object in a single file, save it as a .npz file using `np.savez`. You can also save several arrays into a single file in compressed npz format with savez_compressed.

It's easy to save and load an array with `np.save()`. Just make sure to specify the array you want to save and a file name. For example, if you create this array:

```
>>> a = np.array([1, 2, 3, 4, 5, 6])
```

You can save it as ”filename.npy” with:

```
>>> np.save('filename', a)
```

You can use `np.load()` to reconstruct your array. :

```
>>> b = np.load('filename.npy')
```

If you want to check your array, you can run:

```
>>> print(b)
[1 2 3 4 5 6]
```

You can save a NumPy array as a plain text file like a **.csv** or **.txt** file with `np.savetxt`.

For example, if you create this array:

```
>>> csv_arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

You can easily save it as a .csv file with the name ”new_file.csv” like this:

```
>>> np.savetxt('new_file.csv', csv_arr)
```

You can quickly and easily load your saved text file using `loadtxt()`:

```
>>> np.loadtxt('new_file.csv')
array([1., 2., 3., 4., 5., 6., 7., 8.])
```

The `savetxt()` and `loadtxt()` functions accept additional optional parameters such as header, footer, and delimiter. While text files can be easier for sharing, .npy and .npz files are smaller and faster to read. If you need more sophisticated handling of your text file (for example, if you need to work with lines that contain missing values), you will want to use the genfromtxt function.

With savetxt, you can specify headers, footers, comments, and more.

Learn more about input and output routines here.

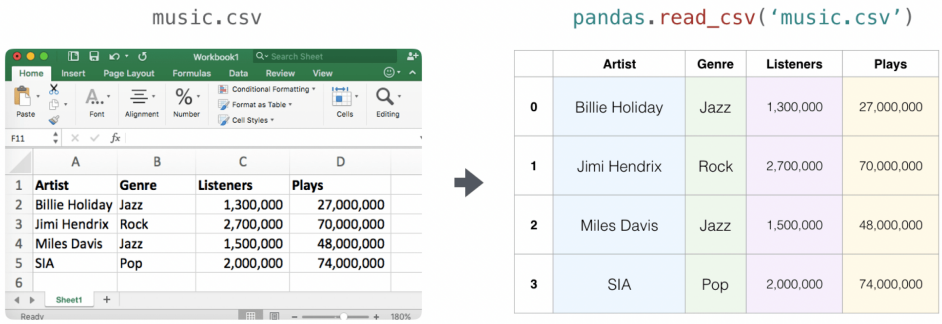
Importing and exporting a CSV

It's simple to read in a CSV that contains existing information. The best and easiest way to do this is to use Pandas. :

```
>>> import pandas as pd

>>> # If all of your columns are the same type:
>>> x = pd.read_csv('music.csv', header=0).values
>>> print(x)
[['Billie Holiday' 'Jazz' 1300000 27000000]
 ['Jimmie Hendrix' 'Rock' 2700000 70000000]
 ['Miles Davis' 'Jazz' 1500000 48000000]
 ['SIA' 'Pop' 2000000 74000000]]

>>> # You can also simply select the columns you need:
>>> x = pd.read_csv('music.csv', usecols=['Artist', 'Plays']).values
>>> print(x)
[['Billie Holiday' 27000000]
 ['Jimmie Hendrix' 70000000]
 ['Miles Davis' 48000000]
 ['SIA' 74000000]]
```



It's simple to use Pandas in order to export your array as well. If you are new to NumPy, you may want to create a Pandas dataframe from the values in your array and then write the data frame to a CSV file with Pandas.

If you created this array "a" :

```
>>> a = np.array([[-2.58289208,  0.43014843, -1.24082018, 1.59572603],
...               [ 0.99027828, 1.17150989,  0.94125714, -0.14692469],
...               [ 0.76989341,  0.81299683, -0.95068423, 0.11769564],
...               [ 0.20484034,  0.34784527,  1.96979195, 0.51992837]])
```

You could create a Pandas dataframe :

```
>>> df = pd.DataFrame(a)
>>> print(df)
          0         1         2         3
0 -2.582892  0.430148 -1.240820  1.595726
1  0.990278  1.171510  0.941257 -0.146925
```

```
2  0.769893  0.812997 -0.950684  0.117696
3  0.204840  0.347845  1.969792  0.519928
```

You can easily save your dataframe with:

```
>>> df.to_csv('pd.csv')
```

And read your CSV with:

```
>>> data = pd.read_csv('pd.csv')
```

	Unnamed: 0	0	1	2	3
0	0	-2.582892	0.430148	-1.240820	1.595726
1	1	0.990278	1.171510	0.941257	-0.146925
2	2	0.769893	0.812997	-0.950684	0.117696
3	3	0.204840	0.347845	1.969792	0.519928

You can also save your array with the NumPy `savetxt` method. :

```
>>> np.savetxt('np.csv', a, fmt='%.2f', delimiter=',', header='1,  2,  3,  4')
```

If you're using the command line, you can read your saved CSV any time with a command such as:

```
$ cat np.csv
#  1,  2,  3,  4
-2.58,0.43,-1.24,1.60
0.99,1.17,0.94,-0.15
0.77,0.81,-0.95,0.12
0.20,0.35,1.97,0.52
```

Or you can open the file any time with a text editor!

If you're interested in learning more about Pandas, take a look at the official Pandas documentation. Learn how to install Pandas with the official Pandas installation information.

Plotting arrays with Matplotlib

If you need to generate a plot for your values, it's very simple with Matplotlib.

For example, you may have an array like this one:

```
>>> a = np.array([2, 1, 5, 7, 4, 6, 8, 14, 10, 9, 18, 20, 22])
```

If you already have Matplotlib installed, you can import it with:

```
>>> import matplotlib.pyplot as plt

# If you're using Jupyter Notebook, you may also want to run the following
# line of code to display your code in the notebook:

%matplotlib inline
```

All you need to do to plot your values is run:

```
>>> plt.plot(a)

# If you are running from a command line, you may need to do this:
# >>> plt.show()
```

For example, you can plot a 1D array like this:

```
>>> x = np.linspace(0, 5, 20)
>>> y = np.linspace(0, 10, 20)
>>> plt.plot(x, y, 'purple') # line
>>> plt.plot(x, y, 'o')      # dots
```

With Matplotlib, you have access to an enormous number of visualization options. :

```
>>> fig = plt.figure()
>>> ax = fig.add_subplot(projection='3d')
>>> X = np.arange(-5, 5, 0.15)
>>> Y = np.arange(-5, 5, 0.15)
>>> X, Y = np.meshgrid(X, Y)
>>> R = np.sqrt(X**2 + Y**2)
>>> Z = np.sin(R)

>>> ax.plot_surface(X, Y, Z, rstride=1, cstride=1, cmap='viridis')
```

To read more about Matplotlib and what it can do, take a look at the official documentation. For directions regarding installing Matplotlib, see the official installation section.

Image credits: Jay Alammar https://jalammar.github.io/

Array creation

Source: https://numpy.org/doc/stable/user/basics.creation.html

Introduction

There are 6 general mechanisms for creating arrays:

1) Conversion from other Python structures (i.e. lists and tuples)
2) Intrinsic NumPy array creation functions (e.g. arange, ones, zeros, etc.)
3) Replicating, joining, or mutating existing arrays
4) Reading arrays from disk, either from standard or custom formats
5) Creating arrays from raw bytes through the use of strings or buffers
6) Use of special library functions (e.g., random)

You can use these methods to create ndarrays or structured_arrays. This document will cover general methods for ndarray creation.

1) Converting Python sequences to NumPy arrays

NumPy arrays can be defined using Python sequences such as lists and tuples. Lists and tuples are defined using `[...]` and `(...)`, respectively. Lists and tuples can define ndarray creation:

- a list of numbers will create a 1D array,
- a list of lists will create a 2D array,
- further nested lists will create higher-dimensional arrays. In general, any array object is called an **ndarray** in NumPy.

```
>>> import numpy as np
>>> a1D = np.array([1, 2, 3, 4])
>>> a2D = np.array([[1, 2], [3, 4]])
>>> a3D = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

When you use `numpy.array` to define a new array, you should consider the dtype of the elements in the array, which can be specified explicitly. This feature gives you more control over the underlying data structures and how the elements are handled in C/C++ functions. When values do not fit and you are using a `dtype`, NumPy may raise an error:

```
>>> import numpy as np
>>> np.array([127, 128, 129], dtype=np.int8)
Traceback (most recent call last):
...
OverflowError: Python integer 128 out of bounds for int8
```

An 8-bit signed integer represents integers from -128 to 127. Assigning the `int8` array to integers outside of this range results in overflow. This feature can often be misunderstood. If you perform calculations with mismatching `dtypes`, you can get unwanted results, for example:

```
>>> import numpy as np
>>> a = np.array([2, 3, 4], dtype=np.uint32)
>>> b = np.array([5, 6, 7], dtype=np.uint32)
>>> c_unsigned32 = a - b
```

```
>>> print('unsigned c:', c_unsigned32, c_unsigned32.dtype)
unsigned c: [4294967293 4294967293 4294967293] uint32
>>> c_signed32 = a - b.astype(np.int32)
>>> print('signed c:', c_signed32, c_signed32.dtype)
signed c: [-3 -3 -3] int64
```

Notice when you perform operations with two arrays of the same `dtype`: `uint32`, the resulting array is the same type. When you perform operations with different `dtype`, NumPy will assign a new type that satisfies all of the array elements involved in the computation, here `uint32` and `int32` can both be represented in as `int64`.

The default NumPy behavior is to create arrays in either 32 or 64-bit signed integers (platform dependent and matches C `long` size) or double precision floating point numbers. If you expect your integer arrays to be a specific type, then you need to specify the dtype while you create the array.

2) Intrinsic NumPy array creation functions

NumPy has over 40 built-in functions for creating arrays as laid out in the Array creation routines. These functions can be split into roughly three categories, based on the dimension of the array they create:

1) 1D arrays
2) 2D arrays
3) ndarrays

1 - 1D array creation functions

The 1D array creation functions e.g. `numpy.linspace` and `numpy.arange` generally need at least two inputs, `start` and `stop`.

`numpy.arange` creates arrays with regularly incrementing values. Check the documentation for complete information and examples. A few examples are shown:

```
>>> import numpy as np
>>> np.arange(10)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.arange(2, 10, dtype=np.float64)
array([2., 3., 4., 5., 6., 7., 8., 9.])
>>> np.arange(2, 3, 0.1)
array([2. , 2.1, 2.2, 2.3, 2.4, 2.5, 2.6, 2.7, 2.8, 2.9])
```

Note: best practice for `numpy.arange` is to use integer start, end, and step values. There are some subtleties regarding `dtype`. In the second example, the `dtype` is defined. In the third example, the array is `dtype=np.float64` to accommodate the step size of `0.1`. Due to roundoff error, the `stop` value is sometimes included.

`numpy.linspace` will create arrays with a specified number of elements, and spaced equally between the specified beginning and end values. For example: :

```
>>> import numpy as np
>>> np.linspace(1., 4., 6)
array([1. ,  1.6,  2.2,  2.8,  3.4,  4. ])
```

The advantage of this creation function is that you guarantee the number of elements and the starting and end point. The previous `arange(start, stop, step)` will not include the value `stop`.

2 - 2D array creation functions

The 2D array creation functions e.g. `numpy.eye`, `numpy.diag`, and `numpy.vander` define properties of special matrices represented as 2D arrays.

`np.eye(n, m)` defines a 2D identity matrix. The elements where i=j (row index and column index are equal) are 1 and the rest are 0, as such:

```
>>> import numpy as np
>>> np.eye(3)
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
>>> np.eye(3, 5)
array([[1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       [0., 0., 1., 0., 0.]])
```

`numpy.diag` can define either a square 2D array with given values along the diagonal *or* if given a 2D array returns a 1D array that is only the diagonal elements. The two array creation functions can be helpful while doing linear algebra, as such:

```
>>> import numpy as np
>>> np.diag([1, 2, 3])
array([[1, 0, 0],
       [0, 2, 0],
       [0, 0, 3]])
>>> np.diag([1, 2, 3], 1)
array([[0, 1, 0, 0],
       [0, 0, 2, 0],
       [0, 0, 0, 3],
       [0, 0, 0, 0]])
>>> a = np.array([[1, 2], [3, 4]])
>>> np.diag(a)
array([1, 4])
```

`vander(x, n)` defines a Vandermonde matrix as a 2D NumPy array. Each column of the Vandermonde matrix is a decreasing power of the input 1D array or list or tuple, `x` where the highest polynomial order is `n-1`. This array creation routine is helpful in generating linear least squares models, as such:

```
>>> import numpy as np
>>> np.vander(np.linspace(0, 2, 5), 2)
array([[0. , 1. ],
      [0.5, 1. ],
      [1. , 1. ],
      [1.5, 1. ],
      [2. , 1. ]])
>>> np.vander([1, 2, 3, 4], 2)
array([[1, 1],
       [2, 1],
       [3, 1],
       [4, 1]])
>>> np.vander((1, 2, 3, 4), 4)
array([[ 1,  1,  1,  1],
       [ 8,  4,  2,  1],
       [27,  9,  3,  1],
       [64, 16,  4,  1]])
```

3 - general ndarray creation functions

The ndarray creation functions e.g. `numpy.ones`, `numpy.zeros`, and `random` define arrays based upon the desired shape. The ndarray creation functions can create arrays with any dimension by specifying how many dimensions and length along that dimension in a tuple or list.

`numpy.zeros` will create an array filled with 0 values with the specified shape. The default dtype is `float64`:

```
>>> import numpy as np
>>> np.zeros((2, 3))
array([[0., 0., 0.],
       [0., 0., 0.]])
>>> np.zeros((2, 3, 2))
array([[[0., 0.],
        [0., 0.],
        [0., 0.]],
<BLANKLINE>
       [[0., 0.],
        [0., 0.],
        [0., 0.]]])
```

`numpy.ones` will create an array filled with 1 values. It is identical to `zeros` in all other respects as such:

```
>>> import numpy as np
>>> np.ones((2, 3))
array([[1., 1., 1.],
       [1., 1., 1.]])
>>> np.ones((2, 3, 2))
array([[[1., 1.],
        [1., 1.],
        [1., 1.]],
<BLANKLINE>
       [[1., 1.],
        [1., 1.],
        [1., 1.]]])
```

The `random` method of the result of `default_rng` will create an array filled with random values between 0 and 1. It is included with the `numpy.random` library. Below, two arrays are created with shapes (2,3) and (2,3,2), respectively. The seed is set to 42 so you can reproduce these pseudorandom numbers:

```
>>> import numpy as np
>>> from numpy.random import default_rng
>>> default_rng(42).random((2,3))
array([[0.77395605, 0.43887844, 0.85859792],
       [0.69736803, 0.09417735, 0.97562235]])
>>> default_rng(42).random((2,3,2))
array([[[0.77395605, 0.43887844],
        [0.85859792, 0.69736803],
        [0.09417735, 0.97562235]],
       [[0.7611397 , 0.78606431],
        [0.12811363, 0.45038594],
        [0.37079802, 0.92676499]]])
```

`numpy.indices` will create a set of arrays (stacked as a one-higher dimensioned array), one per dimension with each representing variation in that dimension:

```
>>> import numpy as np
>>> np.indices((3,3))
array([[[0, 0, 0],
        [1, 1, 1],
        [2, 2, 2]],
       [[0, 1, 2],
        [0, 1, 2],
        [0, 1, 2]]])
```

This is particularly useful for evaluating functions of multiple dimensions on a regular grid.

3) Replicating, joining, or mutating existing arrays

Once you have created arrays, you can replicate, join, or mutate those existing arrays to create new arrays. When you assign an array or its elements to a new variable, you have to explicitly `numpy.copy` the array, otherwise the variable is a view into the original array. Consider the following example:

```
>>> import numpy as np
>>> a = np.array([1, 2, 3, 4, 5, 6])
>>> b = a[:2]
>>> b += 1
>>> print('a =', a, '; b =', b)
a = [2 3 3 4 5 6] ; b = [2 3]
```

In this example, you did not create a new array. You created a variable, `b` that viewed the first 2 elements of `a`. When you added 1 to `b` you would get the same result by adding 1 to `a[:2]`. If you want to create a *new* array, use the `numpy.copy` array creation routine as such:

```
>>> import numpy as np
>>> a = np.array([1, 2, 3, 4])
>>> b = a[:2].copy()
>>> b += 1
>>> print('a = ', a, 'b = ', b)
a =  [1 2 3 4] b =  [2 3]
```

For more information and examples look at `Copies and Views <quickstart.copies-and-views>`.

There are a number of routines to join existing arrays e.g. `numpy.vstack`, `numpy.hstack`, and `numpy.block`. Here is an example of joining four 2-by-2 arrays into a 4-by-4 array using `block`:

```
>>> import numpy as np
>>> A = np.ones((2, 2))
>>> B = np.eye(2, 2)
>>> C = np.zeros((2, 2))
>>> D = np.diag((-3, -4))
>>> np.block([[A, B], [C, D]])
array([[ 1.,  1.,  1.,  0.],
       [ 1.,  1.,  0.,  1.],
       [ 0.,  0., -3.,  0.],
       [ 0.,  0.,  0., -4.]])
```

Other routines use similar syntax to join ndarrays. Check the routine's documentation for further examples and syntax.

4) Reading arrays from disk, either from standard or custom formats

This is the most common case of large array creation. The details depend greatly on the format of data on disk. This section gives general pointers on how to handle various formats. For more detailed examples of IO look at How to Read and Write files.

Standard binary formats

Various fields have standard formats for array data. The following lists the ones with known Python libraries to read them and return NumPy arrays (there may be others for which it is possible to read and convert to NumPy arrays so check the last section as well) :

```
HDF5: h5py
FITS: Astropy
```

Examples of formats that cannot be read directly but for which it is not hard to convert are those formats supported by libraries like PIL (able to read and write many image formats such as jpg, png, etc).

Common ASCII formats

Delimited files such as comma separated value (csv) and tab separated value (tsv) files are used for programs like Excel and LabView. Python functions can read and parse these files line-by-line. NumPy has two standard routines for importing a file with delimited data `numpy.loadtxt` and `numpy.genfromtxt`. These functions have more involved use cases in how-to-io. A simple example given a `simple.csv`:

```
$ cat simple.csv
x, y
0, 0
1, 1
2, 4
3, 9
```

Importing `simple.csv` is accomplished using `numpy.loadtxt`:

```
>>> import numpy as np
>>> np.loadtxt('simple.csv', delimiter = ',', skiprows = 1)
array([[0., 0.],
       [1., 1.],
       [2., 4.],
       [3., 9.]])
```

More generic ASCII files can be read using scipy.io and Pandas.

5) Creating arrays from raw bytes through the use of strings or buffers

There are a variety of approaches one can use. If the file has a relatively simple format then one can write a simple I/O library and use the NumPy `fromfile()` function and `.tofile()` method to read and write NumPy arrays directly (mind your byteorder though!) If a good C or C++ library exists that read the data, one can wrap that library with a variety of techniques though that certainly is much more work and requires significantly more advanced knowledge to interface with C or C++.

6) Use of special library functions (e.g., SciPy, pandas, and OpenCV)

NumPy is the fundamental library for array containers in the Python Scientific Computing stack. Many Python libraries, including SciPy, Pandas, and OpenCV, use NumPy ndarrays as the common format for data exchange, These libraries can create, operate on, and work with NumPy arrays.

Indexing on ndarrays

Source: https://numpy.org/doc/stable/user/basics.indexing.html

`ndarrays` can be indexed using the standard Python `x[obj]` syntax, where *x* is the array and *obj* the selection. There are different kinds of indexing available depending on *obj*: basic indexing, advanced indexing and field access.

Most of the following examples show the use of indexing when referencing data in an array. The examples work just as well when assigning to an array. See assigning-values-to-indexed-arrays for specific examples and explanations on how assignments work.

Note that in Python, `x[(exp1, exp2, ..., expN)]` is equivalent to `x[exp1, exp2, ..., expN]`; the latter is just syntactic sugar for the former.

Basic indexing

Single element indexing

Single element indexing works exactly like that for other standard Python sequences. It is 0-based, and accepts negative indices for indexing from the end of the array. :

```
>>> x = np.arange(10)
>>> x[2]
2
>>> x[-2]
8
```

It is not necessary to separate each dimension's index into its own set of square brackets. :

```
>>> x = x.reshape((2, 5))  # now x is 2-dimensional
>>> x[1, 3]
8
>>> x[1, -1]
9
```

Note that if one indexes a multidimensional array with fewer indices than dimensions, one gets a subdimensional array. For example: :

```
>>> x[0]
array([0, 1, 2, 3, 4])
```

That is, each index specified selects the array corresponding to the rest of the dimensions selected. In the above example, choosing 0 means that the remaining dimension of length 5 is being left unspecified, and that what is returned is an array of that dimensionality and size. It must be noted that the returned array is a view, i.e., it is not a copy of the original, but points to the same values in memory as does the original array. In this case, the 1-D array at the first position (0) is returned. So using a single index on the returned array, results in a single element being returned. That is: :

```
>>> x[0][2]
2
```

So note that `x[0, 2] == x[0][2]` though the second case is more inefficient as a new temporary array is created after the first index that is subsequently indexed by 2.

Note
NumPy uses C-order indexing. That means that the last index usually represents the most rapidly changing memory location, unlike Fortran or IDL, where the first index represents the most rapidly changing location in memory. This difference represents a great potential for confusion.

Slicing and striding

Basic slicing extends Python's basic concept of slicing to N dimensions. Basic slicing occurs when *obj* is a `slice` object (constructed by `start:stop:step` notation inside of brackets), an integer, or a tuple of slice objects and integers. `Ellipsis` and `newaxis` objects can be interspersed with these as well.

The simplest case of indexing with *N* integers returns an `array scalar <arrays.scalars>` representing the corresponding item. As in Python, all indices are zero-based: for the *i*-th index n_i, the valid range is $0 \leq n_i < d_i$ where d_i is the *i*-th element of the shape of the array. Negative indices are interpreted as counting from the end of the array (*i.e.*, if $n_i < 0$, it means $n_i + d_i$).

All arrays generated by basic slicing are always views of the original array.

Note
NumPy slicing creates a view instead of a copy as in the case of built-in Python sequences such as string, tuple and list. Care must be taken when extracting a small portion from a large array which becomes useless after the extraction, because the small portion extracted contains a reference to the large original array whose memory will not be released until all arrays derived from it are garbage-collected. In such cases an explicit `copy()` is recommended.

The standard rules of sequence slicing apply to basic slicing on a per-dimension basis (including using a step index). Some useful concepts to remember include:

- The basic slice syntax is `i:j:k` where *i* is the starting index, *j* is the stopping index, and *k* is the step ($k \neq 0$). This selects the *m* elements (in the corresponding dimension) with index values *i*, *i + k*, ..., *i + (m - 1) k* where $m = q + (r \neq 0)$ and *q* and *r* are the quotient and remainder obtained by dividing *j - i* by *k*: *j - i = q k + r*, so that *i + (m - 1) k < j*. For example:

  ```
  >>> x = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
  >>> x[1:7:2]
  array([1, 3, 5])
  ```

- Negative *i* and *j* are interpreted as *n + i* and *n + j* where *n* is the number of elements in the corresponding dimension. Negative *k* makes stepping go towards smaller indices. From the above example:

  ```
  >>> x[-2:10]
  array([8, 9])
  >>> x[-3:3:-1]
  array([7, 6, 5, 4])
  ```

- Assume *n* is the number of elements in the dimension being sliced. Then, if *i* is not given it defaults to 0 for *k > 0* and *n - 1* for *k < 0* . If *j* is not given it defaults to *n* for *k > 0* and *-n-1* for *k < 0* . If *k* is not given it defaults to 1. Note that `::` is the same as `:` and means select all indices along this axis. From the above example:

  ```
  >>> x[5:]
  array([5, 6, 7, 8, 9])
  ```

- If the number of objects in the selection tuple is less than *N*, then `:` is assumed for any subsequent dimensions. For example:

```
>>> x = np.array([[[1],[2],[3]], [[4],[5],[6]]])
>>> x.shape
(2, 3, 1)
>>> x[1:2]
array([[[4],
        [5],
        [6]]])
```

- An integer, *i*, returns the same values as `i:i+1` **except** the dimensionality of the returned object is reduced by

  1. In particular, a selection tuple with the *p*-th element an integer (and all other entries `:`) returns the corresponding sub-array with dimension *N - 1*. If *N = 1* then the returned object is an array scalar. These objects are explained in arrays.scalars.

- If the selection tuple has all entries `:` except the *p*-th entry which is a slice object `i:j:k`, then the returned array has dimension *N* formed by stacking, along the *p*-th axis, the sub-arrays returned by integer indexing of elements *i*, *i+k*, ..., *i + (m - 1) k < j*.

- Basic slicing with more than one non-`:` entry in the slicing tuple, acts like repeated application of slicing using a single non-`:` entry, where the non-`:` entries are successively taken (with all other non-`:` entries replaced by `:`). Thus, `x[ind1, ..., ind2,:]` acts like `x[ind1][..., ind2, :]` under basic slicing.

  > Warning
  > The above is **not** true for advanced indexing.

- You may use slicing to set values in the array, but (unlike lists) you can never grow the array. The size of the value to be set in `x[obj] = value` must be (broadcastable to) the same shape as `x[obj]`.

- A slicing tuple can always be constructed as *obj* and used in the `x[obj]` notation. Slice objects can be used in the construction in place of the `[start:stop:step]` notation. For example, `x[1:10:5, ::-1]` can also be implemented as `obj = (slice(1, 10, 5), slice(None, None, -1)); x[obj]` . This can be useful for constructing generic code that works on arrays of arbitrary dimensions. See dealing-with-variable-indices for more information.

Dimensional indexing tools

There are some tools to facilitate the easy matching of array shapes with expressions and in assignments.

`Ellipsis` expands to the number of `:` objects needed for the selection tuple to index all dimensions. In most cases, this means that the length of the expanded selection tuple is `x.ndim`. There may only be a single ellipsis present. From the above example:

```
>>> x[..., 0]
array([[1, 2, 3],
       [4, 5, 6]])
```

This is equivalent to:

```
>>> x[:, :, 0]
array([[1, 2, 3],
       [4, 5, 6]])
```

Each `newaxis` object in the selection tuple serves to expand the dimensions of the resulting selection by one unit-length dimension. The added dimension is the position of the `newaxis` object in the selection tuple. `newaxis` is an alias for `None`, and `None` can be used in place of this with the same result. From the above example:

```
>>> x[:, np.newaxis, :, :].shape
(2, 1, 3, 1)
>>> x[:, None, :, :].shape
(2, 1, 3, 1)
```

This can be handy to combine two arrays in a way that otherwise would require explicit reshaping operations. For example:

```
>>> x = np.arange(5)
>>> x[:, np.newaxis] + x[np.newaxis, :]
array([[0, 1, 2, 3, 4],
       [1, 2, 3, 4, 5],
       [2, 3, 4, 5, 6],
       [3, 4, 5, 6, 7],
       [4, 5, 6, 7, 8]])
```

Advanced indexing

Advanced indexing is triggered when the selection object, *obj*, is a non-tuple sequence object, an `ndarray` (of data type integer or bool), or a tuple with at least one sequence object or ndarray (of data type integer or bool). There are two types of advanced indexing: integer and Boolean.

Advanced indexing always returns a *copy* of the data (contrast with basic slicing that returns a view).

> Warning
> The definition of advanced indexing means that `x[(1, 2, 3),]` is fundamentally different than `x[(1, 2, 3)]`. The latter is equivalent to `x[1, 2, 3]` which will trigger basic selection while the former will trigger advanced indexing. Be sure to understand why this occurs.

Integer array indexing

Integer array indexing allows selection of arbitrary items in the array based on their *N*-dimensional index. Each integer array represents a number of indices into that dimension.

Negative values are permitted in the index arrays and work as they do with single indices or slices:

```
>>> x = np.arange(10, 1, -1)
>>> x
array([10,  9,  8,  7,  6,  5,  4,  3,  2])
>>> x[np.array([3, 3, 1, 8])]
array([7, 7, 9, 2])
>>> x[np.array([3, 3, -3, 8])]
array([7, 7, 4, 2])
```

If the index values are out of bounds then an `IndexError` is thrown:

```
>>> x = np.array([[1, 2], [3, 4], [5, 6]])
>>> x[np.array([1, -1])]
array([[3, 4],
       [5, 6]])
>>> x[np.array([3, 4])]
Traceback (most recent call last):
  ...
IndexError: index 3 is out of bounds for axis 0 with size 3
```

When the index consists of as many integer arrays as dimensions of the array being indexed, the indexing is straightforward, but different from slicing.

Advanced indices always are broadcast and iterated as *one*:

```
result[i_1, ..., i_M] == x[ind_1[i_1, ..., i_M], ind_2[i_1, ..., i_M],
                           ..., ind_N[i_1, ..., i_M]]
```

Note that the resulting shape is identical to the (broadcast) indexing array shapes `ind_1, ..., ind_N`. If the indices cannot be broadcast to the same shape, an exception `IndexError: shape mismatch: indexing arrays could not be broadcast together with shapes...` is raised.

Indexing with multidimensional index arrays tend to be more unusual uses, but they are permitted, and they are useful for some problems. We'll start with the simplest multidimensional case:

```
>>> y = np.arange(35).reshape(5, 7)
>>> y
array([[ 0,  1,  2,  3,  4,  5,  6],
       [ 7,  8,  9, 10, 11, 12, 13],
       [14, 15, 16, 17, 18, 19, 20],
       [21, 22, 23, 24, 25, 26, 27],
       [28, 29, 30, 31, 32, 33, 34]])
>>> y[np.array([0, 2, 4]), np.array([0, 1, 2])]
array([ 0, 15, 30])
```

In this case, if the index arrays have a matching shape, and there is an index array for each dimension of the array being indexed, the resultant array has the same shape as the index arrays, and the values correspond to the index set for each position in the index arrays. In this example, the first index value is 0 for both index arrays, and thus the first value of the resultant array is `y[0, 0]`. The next value is `y[2, 1]`, and the last is `y[4, 2]`.

If the index arrays do not have the same shape, there is an attempt to broadcast them to the same shape. If they cannot be broadcast to the same shape, an exception is raised:

```
>>> y[np.array([0, 2, 4]), np.array([0, 1])]
Traceback (most recent call last):
  ...
IndexError: shape mismatch: indexing arrays could not be broadcast
together with shapes (3,) (2,)
```

The broadcasting mechanism permits index arrays to be combined with scalars for other indices. The effect is that the scalar value is used for all the corresponding values of the index arrays:

```
>>> y[np.array([0, 2, 4]), 1]
array([ 1, 15, 29])
```

Jumping to the next level of complexity, it is possible to only partially index an array with index arrays. It takes a bit of thought to understand what happens in such cases. For example if we just use one index array with y:

```
>>> y[np.array([0, 2, 4])]
array([[ 0,  1,  2,  3,  4,  5,  6],
       [14, 15, 16, 17, 18, 19, 20],
       [28, 29, 30, 31, 32, 33, 34]])
```

It results in the construction of a new array where each value of the index array selects one row from the array being indexed and the resultant array has the resulting shape (number of index elements, size of row).

In general, the shape of the resultant array will be the concatenation of the shape of the index array (or the shape that all the index arrays were broadcast to) with the shape of any unused

dimensions (those not indexed) in the array being indexed.

Example

From each row, a specific element should be selected. The row index is just `[0, 1, 2]` and the column index specifies the element to choose for the corresponding row, here `[0, 1, 0]`. Using both together the task can be solved using advanced indexing:

```
>>> x = np.array([[1, 2], [3, 4], [5, 6]])
>>> x[[0, 1, 2], [0, 1, 0]]
array([1, 4, 5])
```

To achieve a behaviour similar to the basic slicing above, broadcasting can be used. The function `ix_` can help with this broadcasting. This is best understood with an example.

Example

From a 4x3 array the corner elements should be selected using advanced indexing. Thus all elements for which the column is one of `[0, 2]` and the row is one of `[0, 3]` need to be selected. To use advanced indexing one needs to select all elements *explicitly*. Using the method explained previously one could write:

```
>>> x = np.array([[ 0,  1,  2],
...               [ 3,  4,  5],
...               [ 6,  7,  8],
...               [ 9, 10, 11]])
>>> rows = np.array([[0, 0],
...                  [3, 3]], dtype=np.intp)
>>> columns = np.array([[0, 2],
...                     [0, 2]], dtype=np.intp)
>>> x[rows, columns]
array([[ 0,  2],
       [ 9, 11]])
```

However, since the indexing arrays above just repeat themselves, broadcasting can be used (compare operations such as `rows[:, np.newaxis] + columns`) to simplify this:

```
>>> rows = np.array([0, 3], dtype=np.intp)
>>> columns = np.array([0, 2], dtype=np.intp)
>>> rows[:, np.newaxis]
array([[0],
       [3]])
>>> x[rows[:, np.newaxis], columns]
array([[ 0,  2],
       [ 9, 11]])
```

This broadcasting can also be achieved using the function `ix_`:

```
>>> x[np.ix_(rows, columns)]
array([[ 0,  2],
       [ 9, 11]])
```

Note that without the `np.ix_` call, only the diagonal elements would be selected:

```
>>> x[rows, columns]
array([ 0, 11])
```

This difference is the most important thing to remember about indexing with multiple advanced indices.

Example

A real-life example of where advanced indexing may be useful is for a color lookup table where we want to map the values of an image into RGB triples for display. The lookup table could have a shape (nlookup, 3). Indexing such an array with an image with shape (ny, nx) with dtype=np.uint8 (or any integer type so long as values are with the bounds of the lookup table) will result in an array of shape (ny, nx, 3) where a triple of RGB values is associated with each pixel location.

Boolean array indexing

This advanced indexing occurs when *obj* is an array object of Boolean type, such as may be returned from comparison operators. A single boolean index array is practically identical to `x[obj.nonzero()]` where, as described above, `obj.nonzero()` returns a tuple (of length `obj.ndim`) of integer index arrays showing the `True` elements of *obj*. However, it is faster when `obj.shape == x.shape`.

If `obj.ndim == x.ndim`, `x[obj]` returns a 1-dimensional array filled with the elements of *x* corresponding to the `True` values of *obj*. The search order will be row-major, C-style. An index error will be raised if the shape of *obj* does not match the corresponding dimensions of *x*, regardless of whether those values are `True` or `False`.

A common use case for this is filtering for desired element values. For example, one may wish to select all entries from an array which are not `numpy.nan`:

```
>>> x = np.array([[1., 2.], [np.nan, 3.], [np.nan, np.nan]])
>>> x[~np.isnan(x)]
array([1., 2., 3.])
```

Or wish to add a constant to all negative elements:

```
>>> x = np.array([1., -1., -2., 3])
>>> x[x < 0] += 20
>>> x
array([ 1., 19., 18., 3.])
```

In general if an index includes a Boolean array, the result will be identical to inserting `obj.nonzero()` into the same position and using the integer array indexing mechanism described above. `x[ind_1, boolean_array, ind_2]` is equivalent to `x[(ind_1,) + boolean_array.nonzero() + (ind_2,)]`.

If there is only one Boolean array and no integer indexing array present, this is straightforward. Care must only be taken to make sure that the boolean index has *exactly* as many dimensions as it is supposed to work with.

In general, when the boolean array has fewer dimensions than the array being indexed, this is equivalent to `x[b, ...]`, which means x is indexed by b followed by as many `:` as are needed to fill out the rank of x. Thus the shape of the result is one dimension containing the number of True elements of the boolean array, followed by the remaining dimensions of the array being indexed:

```
>>> x = np.arange(35).reshape(5, 7)
>>> b = x > 20
>>> b[:, 5]
array([False, False, False,  True,  True])
>>> x[b[:, 5]]
array([[21, 22, 23, 24, 25, 26, 27],
       [28, 29, 30, 31, 32, 33, 34]])
```

Here the 4th and 5th rows are selected from the indexed array and combined to make a 2-D array.

Example

From an array, select all rows which sum up to less or equal two:

```
>>> x = np.array([[0, 1], [1, 1], [2, 2]])
>>> rowsum = x.sum(-1)
>>> x[rowsum <= 2, :]
array([[0, 1],
       [1, 1]])
```

Combining multiple Boolean indexing arrays or a Boolean with an integer indexing array can best be understood with the `obj.nonzero()` analogy. The function `ix_` also supports boolean arrays and will work without any surprises.

Example

Use boolean indexing to select all rows adding up to an even number. At the same time columns 0 and 2 should be selected with an advanced integer index. Using the `ix_` function this can be done with:

```
>>> x = np.array([[ 0,  1,  2],
...               [ 3,  4,  5],
...               [ 6,  7,  8],
...               [ 9, 10, 11]])
>>> rows = (x.sum(-1) % 2) == 0
>>> rows
array([False,  True, False,  True])
>>> columns = [0, 2]
>>> x[np.ix_(rows, columns)]
array([[ 3,  5],
       [ 9, 11]])
```

Without the `np.ix_` call, only the diagonal elements would be selected.

Or without `np.ix_` (compare the integer array examples):

```
>>> rows = rows.nonzero()[0]
>>> x[rows[:, np.newaxis], columns]
array([[ 3,  5],
       [ 9, 11]])
```

Example

Use a 2-D boolean array of shape (2, 3) with four True elements to select rows from a 3-D array of shape (2, 3, 5) results in a 2-D result of shape (4, 5):

```
>>> x = np.arange(30).reshape(2, 3, 5)
>>> x
array([[[ 0,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9],
        [10, 11, 12, 13, 14]],
      [[15, 16, 17, 18, 19],
        [20, 21, 22, 23, 24],
        [25, 26, 27, 28, 29]]])
>>> b = np.array([[True, True, False], [False, True, True]])
>>> x[b]
array([[ 0,  1,  2,  3,  4],
      [ 5,  6,  7,  8,  9],
      [20, 21, 22, 23, 24],
      [25, 26, 27, 28, 29]])
```

Combining advanced and basic indexing

When there is at least one slice (`:`), ellipsis (`...`) or `newaxis` in the index (or the array has more dimensions than there are advanced indices), then the behaviour can be more complicated. It is like concatenating the indexing result for each advanced index element.

In the simplest case, there is only a *single* advanced index combined with a slice. For example:

```
>>> y = np.arange(35).reshape(5,7)
>>> y[np.array([0, 2, 4]), 1:3]
array([[ 1,  2],
       [15, 16],
       [29, 30]])
```

In effect, the slice and index array operation are independent. The slice operation extracts columns with index 1 and 2, (i.e. the 2nd and 3rd columns), followed by the index array operation which extracts rows with index 0, 2 and 4 (i.e the first, third and fifth rows). This is equivalent to:

```
>>> y[:, 1:3][np.array([0, 2, 4]), :]
array([[ 1,  2],
       [15, 16],
       [29, 30]])
```

A single advanced index can, for example, replace a slice and the result array will be the same. However, it is a copy and may have a different memory layout. A slice is preferable when it is possible. For example:

```
>>> x = np.array([[ 0,  1,  2],
...               [ 3,  4,  5],
...               [ 6,  7,  8],
...               [ 9, 10, 11]])
>>> x[1:2, 1:3]
array([[4, 5]])
>>> x[1:2, [1, 2]]
array([[4, 5]])
```

The easiest way to understand a combination of *multiple* advanced indices may be to think in terms of the resulting shape. There are two parts to the indexing operation, the subspace defined by the basic indexing (excluding integers) and the subspace from the advanced indexing part. Two cases of index combination need to be distinguished:

- The advanced indices are separated by a slice, `Ellipsis` or `newaxis`. For example `x[arr1, :, arr2]`.
- The advanced indices are all next to each other. For example `x[..., arr1, arr2, :]` but *not* `x[arr1, :, 1]` since `1` is an advanced index in this regard.

In the first case, the dimensions resulting from the advanced indexing operation come first in the result array, and the subspace dimensions after that. In the second case, the dimensions from the advanced indexing operations are inserted into the result array at the same spot as they were in the initial array (the latter logic is what makes simple advanced indexing behave just like slicing).

Example

Suppose `x.shape` is (10, 20, 30) and `ind` is a (2, 5, 2)-shaped indexing `intp` array, then `result = x[..., ind, :]` has shape (10, 2, 5, 2, 30) because the (20,)-shaped subspace has been replaced with a (2, 5, 2)-shaped broadcasted indexing subspace. If we let *i, j, k* loop over the (2, 5, 2)-shaped subspace then `result[..., i, j, k, :] = x[..., ind[i, j, k], :]`. This example produces the same result as `x.take(ind, axis=-2)`.

Example

Let `x.shape` be (10, 20, 30, 40, 50) and suppose `ind_1` and `ind_2` can be broadcast to the shape (2, 3, 4). Then `x[:, ind_1, ind_2]` has shape (10, 2, 3, 4, 40, 50) because the (20, 30)-shaped subspace from X has been replaced with the (2, 3, 4) subspace from the indices. However, `x[:, ind_1, :, ind_2]` has shape (2, 3, 4, 10, 30, 50) because there is no unambiguous place to drop in the indexing subspace, thus it is tacked-on to the beginning. It is always possible to use `.transpose()` to move the subspace anywhere desired. Note that this example cannot be replicated using `take`.

Example

Slicing can be combined with broadcasted boolean indices:

```
>>> x = np.arange(35).reshape(5, 7)
>>> b = x > 20
>>> b
array([[False, False, False, False, False, False, False],
       [False, False, False, False, False, False, False],
       [False, False, False, False, False, False, False],
       [ True,  True,  True,  True,  True,  True,  True],
       [ True,  True,  True,  True,  True,  True,  True]])
>>> x[b[:, 5], 1:3]
array([[22, 23],
       [29, 30]])
```

Field access

If the `ndarray` object is a structured array the fields of the array can be accessed by indexing the array with strings, dictionary-like.

Indexing `x['field-name']` returns a new view to the array, which is of the same shape as *x* (except when the field is a sub-array) but of data type `x.dtype['field-name']` and contains only the part of the data in the specified field. Also, record array scalars can be ”indexed” this way.

Indexing into a structured array can also be done with a list of field names, e.g. `x[['field-name1', 'field-name2']]`. As of NumPy 1.16, this returns a view containing only those fields. In older versions of NumPy, it returned a copy. See the user guide section on structured_arrays for more information on multifield indexing.

If the accessed field is a sub-array, the dimensions of the sub-array are appended to the shape of the result. For example:

```
>>> x = np.zeros((2, 2), dtype=[('a', np.int32), ('b', np.float64, (3, 3))])
>>> x['a'].shape
(2, 2)
>>> x['a'].dtype
dtype('int32')
>>> x['b'].shape
(2, 2, 3, 3)
>>> x['b'].dtype
dtype('float64')
```

Flat iterator indexing

`x.flat` returns an iterator that will iterate over the entire array (in C-contiguous style with the last index varying the fastest). This iterator object can also be indexed using basic slicing or advanced indexing as long as the selection object is not a tuple. This should be clear from the

fact that `x.flat <ndarray.flat>` is a 1-dimensional view. It can be used for integer indexing with 1-dimensional C-style-flat indices. The shape of any returned array is therefore the shape of the integer indexing object.

Assigning values to indexed arrays

As mentioned, one can select a subset of an array to assign to using a single index, slices, and index and mask arrays. The value being assigned to the indexed array must be shape consistent (the same shape or broadcastable to the shape the index produces). For example, it is permitted to assign a constant to a slice: :

```
>>> x = np.arange(10)
>>> x[2:7] = 1
```

or an array of the right size: :

```
>>> x[2:7] = np.arange(5)
```

Note that assignments may result in changes if assigning higher types to lower types (like floats to ints) or even exceptions (assigning complex to floats or ints): :

```
>>> x[1] = 1.2
>>> x[1]
1
>>> x[1] = 1.2j
Traceback (most recent call last):
  ...
TypeError: can't convert complex to int
```

Unlike some of the references (such as array and mask indices) assignments are always made to the original data in the array (indeed, nothing else would make sense!). Note though, that some actions may not work as one may naively expect. This particular example is often surprising to people: :

```
>>> x = np.arange(0, 50, 10)
>>> x
array([ 0, 10, 20, 30, 40])
>>> x[np.array([1, 1, 3, 1])] += 1
>>> x
array([ 0, 11, 20, 31, 40])
```

Where people expect that the 1st location will be incremented by 3. In fact, it will only be incremented by 1. The reason is that a new array is extracted from the original (as a temporary) containing the values at 1, 1, 3, 1, then the value 1 is added to the temporary, and then the temporary is assigned back to the original array. Thus the value of the array at `x[1] + 1` is assigned to `x[1]` three times, rather than being incremented 3 times.

Dealing with variable numbers of indices within programs

The indexing syntax is very powerful but limiting when dealing with a variable number of indices. For example, if you want to write a function that can handle arguments with various numbers of dimensions without having to write special case code for each number of possible dimensions, how can that be done? If one supplies to the index a tuple, the tuple will be interpreted as a list of indices. For example:

```
>>> z = np.arange(81).reshape(3, 3, 3, 3)
>>> indices = (1, 1, 1, 1)
```

```
>>> z[indices]
40
```

So one can use code to construct tuples of any number of indices and then use these within an index.

Slices can be specified within programs by using the slice() function in Python. For example: :

```
>>> indices = (1, 1, 1, slice(0, 2))  # same as [1, 1, 1, 0:2]
>>> z[indices]
array([39, 40])
```

Likewise, ellipsis can be specified by code by using the Ellipsis object: :

```
>>> indices = (1, Ellipsis, 1)  # same as [1, ..., 1]
>>> z[indices]
array([[28, 31, 34],
       [37, 40, 43],
       [46, 49, 52]])
```

For this reason, it is possible to use the output from the `np.nonzero()` function directly as an index since it always returns a tuple of index arrays.

Because of the special treatment of tuples, they are not automatically converted to an array as a list would be. As an example: :

```
>>> z[[1, 1, 1, 1]]  # produces a large array
array([[[[27, 28, 29],
         [30, 31, 32], ...
>>> z[(1, 1, 1, 1)]  # returns a single value
40
```

Detailed notes

These are some detailed notes, which are not of importance for day to day indexing (in no particular order):

- The native NumPy indexing type is `intp` and may differ from the default integer array type. `intp` is the smallest data type sufficient to safely index any array; for advanced indexing it may be faster than other types.
- For advanced assignments, there is in general no guarantee for the iteration order. This means that if an element is set more than once, it is not possible to predict the final result.
- An empty (tuple) index is a full scalar index into a zero-dimensional array. `x[()]` returns a *scalar* if `x` is zero-dimensional and a view otherwise. On the other hand, `x[...]` always returns a view.
- If a zero-dimensional array is present in the index *and* it is a full integer index the result will be a *scalar* and not a zero-dimensional array. (Advanced indexing is not triggered.)
- When an ellipsis (`...`) is present but has no size (i.e. replaces zero `:`) the result will still always be an array. A view if no advanced index is present, otherwise a copy.
- The `nonzero` equivalence for Boolean arrays does not hold for zero dimensional boolean arrays.
- When the result of an advanced indexing operation has no elements but an individual index is out of bounds, whether or not an `IndexError` is raised is undefined (e.g. `x[[], [123]]` with `123` being out of bounds).
- When a *casting* error occurs during assignment (for example updating a numerical array using a sequence of strings), the array being assigned to may end up in an unpredictable partially updated state. However, if any other error (such as an out of bounds index) occurs, the array will remain unchanged.

- The memory layout of an advanced indexing result is optimized for each indexing operation and no particular memory order can be assumed.
- When using a subclass (especially one which manipulates its shape), the default `ndarray.__setitem__` behaviour will call `__getitem__` for *basic* indexing but not for *advanced* indexing. For such a subclass it may be preferable to call `ndarray.__setitem__` with a *base class* ndarray view on the data. This *must* be done if the subclasses `__getitem__` does not return views.

Data types

Source: https://numpy.org/doc/stable/user/basics.types.html

Array types and conversions between types

NumPy supports a much greater variety of numerical types than Python does. This section shows which are available, and how to modify an array's data-type.

NumPy numerical types are instances of numpy.dtype (data-type) objects, each having unique characteristics. Once you have imported NumPy using `import numpy as np` you can create arrays with a specified dtype using the scalar types in the numpy top-level API, e.g. numpy.bool, numpy.float32, etc.

These scalar types as arguments to the dtype keyword that many numpy functions or methods accept. For example:

```
>>> z = np.arange(3, dtype=np.uint8)
>>> z
array([0, 1, 2], dtype=uint8)
```

Array types can also be referred to by character codes, for example:

```
>>> np.array([1, 2, 3], dtype='f')
array([1.,  2.,  3.], dtype=float32)
>>> np.array([1, 2, 3], dtype='d')
array([1.,  2.,  3.], dtype=float64)
```

See arrays.dtypes.constructing for more information about specifying and constructing data type objects, including how to specify parameters like the byte order.

To determine the type of an array, look at the dtype attribute:

```
>>> z.dtype
dtype('uint8')
```

dtype objects also contain information about the type, such as its bit-width and its byte-order. The data type can also be used indirectly to query properties of the type, such as whether it is an integer:

```
>>> d = np.dtype(np.int64)
>>> d
dtype('int64')

>>> np.issubdtype(d, np.integer)
True

>>> np.issubdtype(d, np.floating)
False
```

To convert the type of an array, use the .astype() method. For example:

```
>>> z.astype(np.float64)
array([0.,  1.,  2.])
```

Note that, above, we could have used the *Python* float object as a dtype instead of numpy.float64. NumPy knows that `int` refers to numpy.int_, `bool` means numpy.bool, that `float` is numpy.float64 and `complex` is numpy.complex128. The other data-types do not have Python equivalents.

Sometimes the conversion can overflow, for instance when converting a numpy.int64 value 300 to numpy.int8. NumPy follows C casting rules, so that value would overflow and become 44 `(300 - 256)`. If you wish to avoid such overflows, you can specify that the overflow action fail by using `same_value` for the `casting` argument (see also overflow-errors):

```
>>> z.astype(np.float64, casting="same_value")
array([0.,  1.,  2.])
```

Numerical Data Types

There are 5 basic numerical types representing booleans (`bool`), integers (`int`), unsigned integers (`uint`) floating point (`float`) and `complex`. A basic numerical type name combined with a numeric bitsize defines a concrete type. The bitsize is the number of bits that are needed to represent a single value in memory. For example, numpy.float64 is a 64 bit floating point data type. Some types, such as numpy.int_ and numpy.intp, have differing bitsizes, dependent on the platforms (e.g. 32-bit vs. 64-bit CPU architectures). This should be taken into account when interfacing with low-level code (such as C or Fortran) where the raw memory is addressed.

Data Types for Strings and Bytes

In addition to numerical types, NumPy also supports storing unicode strings, via the numpy.str_ dtype (`U` character code), null-terminated byte sequences via numpy.bytes_ (`S` character code), and arbitrary byte sequences, via numpy.void (`V` character code).

All of the above are *fixed-width* data types. They are parameterized by a width, in either bytes or unicode points, that a single data element in the array must fit inside. This means that storing an array of byte sequences or strings using this dtype requires knowing or calculating the sizes of the longest text or byte sequence in advance.

As an example, we can create an array storing the words `"hello"` and `"world!"`:

```
>>> np.array(["hello", "world!"])
array(['hello', 'world!'], dtype='<U6')
```

Here the data type is detected as a unicode string that is a maximum of 6 code points long, enough to store both entries without truncation. If we specify a shorter or longer data type, the string is either truncated or zero-padded to fit in the specified width:

```
>>> np.array(["hello", "world!"], dtype="U5")
array(['hello', 'world'], dtype='<U5')
>>> np.array(["hello", "world!"], dtype="U7")
array(['hello', 'world!'], dtype='<U7')
```

We can see the zero-padding a little more clearly if we use the bytes data type and ask NumPy to print out the bytes in the array buffer:

```
>>> np.array(["hello", "world"], dtype="S7").tobytes()
b'hello\x00\x00world\x00\x00'
```

Each entry is padded with two extra null bytes. Note however that NumPy cannot tell the difference between intentionally stored trailing nulls and padding nulls:

```
>>> x = [b"hello\0\0", b"world"]
>>> a = np.array(x, dtype="S7")
>>> print(a[0])
b"hello"
```

```
>>> a[0] == x[0]
False
```

If you need to store and round-trip any trailing null bytes, you will need to use an unstructured void data type:

```
>>> a = np.array(x, dtype="V7")
>>> a
array([b'\x68\x65\x6C\x6C\x6F\x00\x00', b'\x77\x6F\x72\x6C\x64\x00\x00'],
      dtype='|V7')
>>> a[0] == np.void(x[0])
True
```

Advanced types, not listed above, are explored in section structured_arrays.

Relationship Between NumPy Data Types and C Data Types

NumPy provides both bit sized type names and names based on the names of C types. Since the definition of C types are platform dependent, this means the explicitly bit sized should be preferred to avoid platform-dependent behavior in programs using NumPy.

To ease integration with C code, where it is more natural to refer to platform-dependent C types, NumPy also provides type aliases that correspond to the C types for the platform. Some dtypes have trailing underscore to avoid confusion with builtin python type names, such as numpy.bool_.

Canonical Python API name	Python API "C-like" name	Actual C type	Description
numpy.bool or numpy.bool_	N/A	`bool` (defined in `stdbool.h`)	Boolean (T
numpy.int8	numpy.byte	`signed char`	Platform-d
numpy.uint8	numpy.ubyte	`unsigned char`	Platform-d
numpy.int16	numpy.short	`short`	Platform-d
numpy.uint16	numpy.ushort	`unsigned short`	Platform-d
numpy.int32	numpy.intc	`int`	Platform-d
numpy.uint32	numpy.uintc	`unsigned int`	Platform-d
numpy.intp	N/A	`ssize_t/Py_ssize_t`	Platform-d
numpy.uintp	N/A	`size_t`	Platform-d
N/A	`'p'`	`intptr_t`	Guaranteed
N/A	`'P'`	`uintptr_t`	Guaranteed
numpy.int32 or numpy.int64	numpy.long	`long`	Platform-d
numpy.uint32 or numpy.uint64	numpy.ulong	`unsigned long`	Platform-d
N/A	numpy.longlong	`long long`	Platform-d
N/A	numpy.ulonglong	`unsigned long long`	Platform-d
numpy.float16	numpy.half	N/A	Half precis
numpy.float32	numpy.single	`float`	Platform-d
numpy.float64	numpy.double	`double`	Platform-d
`numpy.float96` or numpy.float128	numpy.longdouble	`long double`	Platform-d
numpy.complex64	numpy.csingle	`float complex`	Complex nu
numpy.complex128	numpy.cdouble	`double complex`	Complex nu
`numpy.complex192` or numpy.complex256	numpy.clongdouble	`long double complex`	Complex nu

Since many of these have platform-dependent definitions, a set of fixed-size aliases are provided (See sized-aliases).

Array scalars

NumPy generally returns elements of arrays as array scalars (a scalar with an associated dtype). Array scalars differ from Python scalars, but for the most part they can be used interchangeably (the primary exception is for versions of Python older than v2.x, where integer array scalars cannot act as indices for lists and tuples). There are some exceptions, such as when code requires very specific attributes of a scalar or when it checks specifically whether a value is a Python scalar. Generally, problems are easily fixed by explicitly converting array scalars to Python scalars, using the corresponding Python type function (e.g., `int`, `float`, `complex`, `str`).

The primary advantage of using array scalars is that they preserve the array type (Python may not have a matching scalar type available, e.g. `int16`). Therefore, the use of array scalars ensures identical behaviour between arrays and scalars, irrespective of whether the value is inside an array or not. NumPy scalars also have many of the same methods arrays do.

Overflow errors

The fixed size of NumPy numeric types may cause overflow errors when a value requires more memory than available in the data type. For example, numpy.power evaluates `100 ** 9` correctly for 64-bit integers, but gives -1486618624 (incorrect) for a 32-bit integer.

```
>>> np.power(100, 9, dtype=np.int64)
1000000000000000000
>>> np.power(100, 9, dtype=np.int32)
np.int32(-1486618624)
```

The behaviour of NumPy and Python integer types differs significantly for integer overflows and may confuse users expecting NumPy integers to behave similar to Python's `int`. Unlike NumPy, the size of Python's `int` is flexible. This means Python integers may expand to accommodate any integer and will not overflow.

NumPy provides numpy.iinfo and numpy.finfo to verify the minimum or maximum values of NumPy integer and floating point values respectively :

```
>>> np.iinfo(int) # Bounds of the default integer on this system.
iinfo(min=-9223372036854775808, max=9223372036854775807, dtype=int64)
>>> np.iinfo(np.int32) # Bounds of a 32-bit integer
iinfo(min=-2147483648, max=2147483647, dtype=int32)
>>> np.iinfo(np.int64) # Bounds of a 64-bit integer
iinfo(min=-9223372036854775808, max=9223372036854775807, dtype=int64)
```

If 64-bit integers are still too small the result may be cast to a floating point number. Floating point numbers offer a larger, but inexact, range of possible values.

```
>>> np.power(100, 100, dtype=np.int64) # Incorrect even with 64-bit int
0
>>> np.power(100, 100, dtype=np.float64)
1e+200
```

Floating point precision

Many functions in NumPy, especially those in numpy.linalg, involve floating-point arithmetic, which can introduce small inaccuracies due to the way computers represent decimal numbers. For instance, when performing basic arithmetic operations involving floating-point numbers:

```
>>> 0.3 - 0.2 - 0.1  # This does not equal 0 due to floating-point precision
-2.77555756156289e-17
```

To handle such cases, it's advisable to use functions like np.isclose to compare values, rather than checking for exact equality:

```
>>> np.isclose(0.3 - 0.2 - 0.1, 0, rtol=1e-05)  # Check for closeness to 0
True
```

In this example, np.isclose accounts for the minor inaccuracies that occur in floating-point calculations by applying a relative tolerance, ensuring that results within a small threshold are considered close.

For information about precision in calculations, see Floating-Point Arithmetic.

Extended precision

Python's floating-point numbers are usually 64-bit floating-point numbers, nearly equivalent to numpy.float64. In some unusual situations it may be useful to use floating-point numbers with more precision. Whether this is possible in numpy depends on the hardware and on the development environment: specifically, x86 machines provide hardware floating-point with 80-bit precision, and while most C compilers provide this as their `long double` type, MSVC (standard for Windows builds) makes `long double` identical to `double` (64 bits). NumPy makes the compiler's `long double` available as numpy.longdouble (and `np.clongdouble` for the complex numbers). You can find out what your numpy provides with `np.finfo(np.longdouble)`.

NumPy does not provide a dtype with more precision than C's `long double`; in particular, the 128-bit IEEE quad precision data type (FORTRAN's `REAL*16`) is not available.

For efficient memory alignment, numpy.longdouble is usually stored padded with zero bits, either to 96 or 128 bits. Which is more efficient depends on hardware and development environment; typically on 32-bit systems they are padded to 96 bits, while on 64-bit systems they are typically padded to 128 bits. `np.longdouble` is padded to the system default; `np.float96` and `np.float128` are provided for users who want specific padding. In spite of the names, `np.float96` and `np.float128` provide only as much precision as `np.longdouble`, that is, 80 bits on most x86 machines and 64 bits in standard Windows builds.

Be warned that even if numpy.longdouble offers more precision than python `float`, it is easy to lose that extra precision, since python often forces values to pass through `float`. For example, the `%` formatting operator requires its arguments to be converted to standard python types, and it is therefore impossible to preserve extended precision even if many decimal places are requested. It can be useful to test your code with the value `1 + np.finfo(np.longdouble).eps`.

Broadcasting

Source: https://numpy.org/doc/stable/user/basics.broadcasting.html

The term broadcasting describes how NumPy treats arrays with different shapes during arithmetic operations. Subject to certain constraints, the smaller array is "broadcast" across the larger array so that they have compatible shapes. Broadcasting provides a means of vectorizing array operations so that looping occurs in C instead of Python. It does this without making needless copies of data and usually leads to efficient algorithm implementations. There are, however, cases where broadcasting is a bad idea because it leads to inefficient use of memory that slows computation.

NumPy operations are usually done on pairs of arrays on an element-by-element basis. In the simplest case, the two arrays must have exactly the same shape, as in the following example:

```
>>> import numpy as np
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = np.array([2.0, 2.0, 2.0])
>>> a * b
array([2.,  4.,  6.])
```

NumPy's broadcasting rule relaxes this constraint when the arrays' shapes meet certain constraints. The simplest broadcasting example occurs when an array and a scalar value are combined in an operation:

```
>>> import numpy as np
>>> a = np.array([1.0, 2.0, 3.0])
>>> b = 2.0
>>> a * b
array([2.,  4.,  6.])
```

The result is equivalent to the previous example where `b` was an array. We can think of the scalar `b` being *stretched* during the arithmetic operation into an array with the same shape as `a`. The new elements in `b`, as shown in broadcasting.figure-1, are simply copies of the original scalar. The stretching analogy is only conceptual. NumPy is smart enough to use the original scalar value without actually making copies so that broadcasting operations are as memory and computationally efficient as possible.

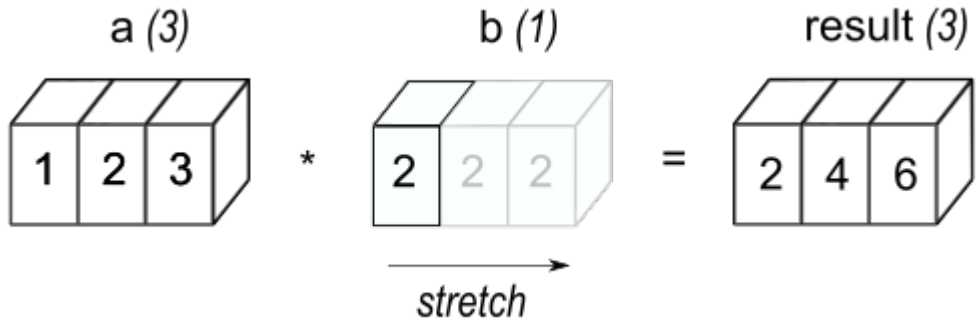


Figure 1 In the simplest example of broadcasting, the scalar `b` *is stretched to become an array of same shape as* `a` *so the shapes are compatible for element-by-element multiplication.*

The code in the second example is more efficient than that in the first because broadcasting

moves less memory around during the multiplication (`b` is a scalar rather than an array).

General broadcasting rules

When operating on two arrays, NumPy compares their shapes element-wise. It starts with the trailing (i.e. rightmost) dimension and works its way left. Two dimensions are compatible when

1) they are equal, or
2) one of them is 1.

If these conditions are not met, a `ValueError: operands could not be broadcast together` exception is thrown, indicating that the arrays have incompatible shapes.

Input arrays do not need to have the same *number* of dimensions. The resulting array will have the same number of dimensions as the input array with the greatest number of dimensions, where the *size* of each dimension is the largest size of the corresponding dimension among the input arrays. Note that missing dimensions are assumed to have size one.

For example, if you have a `256x256x3` array of RGB values, and you want to scale each color in the image by a different value, you can multiply the image by a one-dimensional array with 3 values. Lining up the sizes of the trailing axes of these arrays according to the broadcast rules, shows that they are compatible:

```
Image  (3d array): 256 x 256 x 3
Scale  (1d array):             3
Result (3d array): 256 x 256 x 3
```

When either of the dimensions compared is one, the other is used. In other words, dimensions with size 1 are stretched or ”copied” to match the other.

In the following example, both the `A` and `B` arrays have axes with length one that are expanded to a larger size during the broadcast operation:

```
A      (4d array):  8 x 1 x 6 x 1
B      (3d array):      7 x 1 x 5
Result (4d array):  8 x 7 x 6 x 5
```

Broadcastable arrays

A set of arrays is called ”broadcastable” to the same shape if the above rules produce a valid result.

For example, if `a.shape` is (5,1), `b.shape` is (1,6), `c.shape` is (6,) and `d.shape` is () so that *d* is a scalar, then *a*, *b*, *c*, and *d* are all broadcastable to dimension (5,6); and

- *a* acts like a (5,6) array where `a[:,0]` is broadcast to the other columns,
- *b* acts like a (5,6) array where `b[0,:]` is broadcast to the other rows,
- *c* acts like a (1,6) array and therefore like a (5,6) array where `c[:]` is broadcast to every row, and finally,
- *d* acts like a (5,6) array where the single value is repeated.

Here are some more examples:

```
A      (2d array):  5 x 4
B      (1d array):      1
Result (2d array):  5 x 4

A      (2d array):  5 x 4
B      (1d array):      4
```

```
Result (2d array):  5 x 4

A      (3d array):  15 x 3 x 5
B      (3d array):  15 x 1 x 5
Result (3d array):  15 x 3 x 5

A      (3d array):  15 x 3 x 5
B      (2d array):      3 x 5
Result (3d array):  15 x 3 x 5

A      (3d array):  15 x 3 x 5
B      (2d array):      3 x 1
Result (3d array):  15 x 3 x 5
```

Here are examples of shapes that do not broadcast:

```
A      (1d array):  3
B      (1d array):  4 # trailing dimensions do not match

A      (2d array):      2 x 1
B      (3d array):  8 x 4 x 3 # second from last dimensions mismatched
```

An example of broadcasting when a 1-d array is added to a 2-d array:

```
>>> import numpy as np
>>> a = np.array([[ 0.0,  0.0,  0.0],
...               [10.0, 10.0, 10.0],
...               [20.0, 20.0, 20.0],
...               [30.0, 30.0, 30.0]])
>>> b = np.array([1.0, 2.0, 3.0])
>>> a + b
array([[  1.,   2.,   3.],
       [11.,  12.,  13.],
       [21.,  22.,  23.],
       [31.,  32.,  33.]])
>>> b = np.array([1.0, 2.0, 3.0, 4.0])
>>> a + b
Traceback (most recent call last):
ValueError: operands could not be broadcast together with shapes (4,3) (4,)
```

As shown in broadcasting.figure-2, `b` is added to each row of `a`. In broadcasting.figure-3, an exception is raised because of the incompatible shapes.

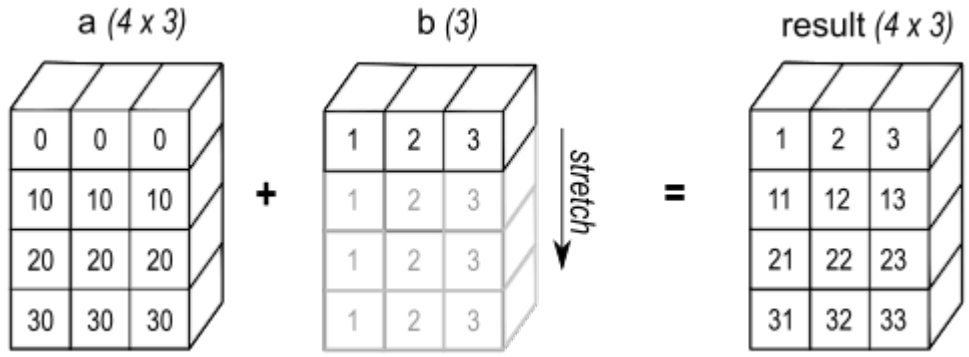


Figure 2 A one dimensional array added to a two dimensional array results in broadcasting if

number of 1-d array elements matches the number of 2-d array columns.

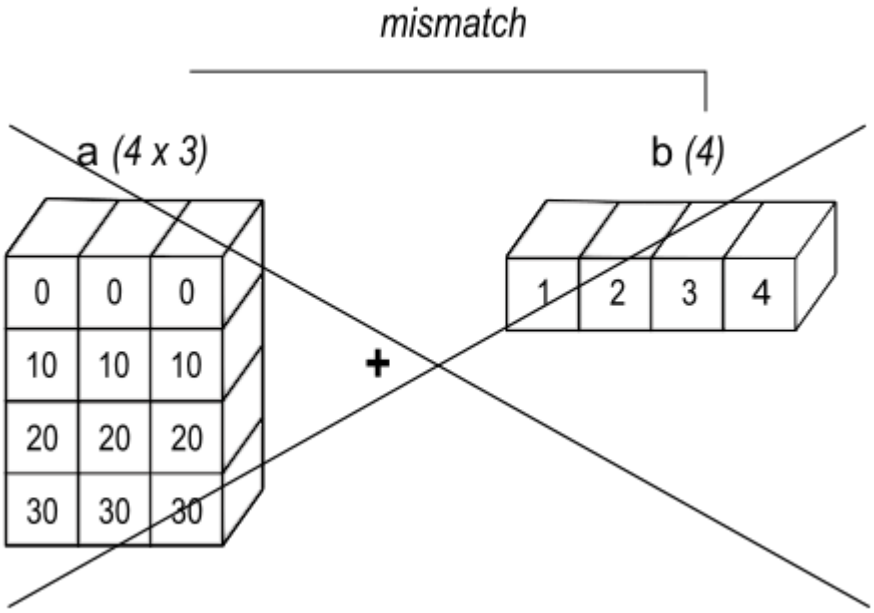


Figure 3 When the trailing dimensions of the arrays are unequal, broadcasting fails because it is impossible to align the values in the rows of the 1st array with the elements of the 2nd arrays for element-by-element addition.

Broadcasting provides a convenient way of taking the outer product (or any other outer operation) of two arrays. The following example shows an outer addition operation of two 1-d arrays:

```
>>> import numpy as np
>>> a = np.array([0.0, 10.0, 20.0, 30.0])
>>> b = np.array([1.0, 2.0, 3.0])
>>> a[:, np.newaxis] + b
array([[ 1.,   2.,   3.],
       [11.,  12.,  13.],
       [21.,  22.,  23.],
       [31.,  32.,  33.]])
```

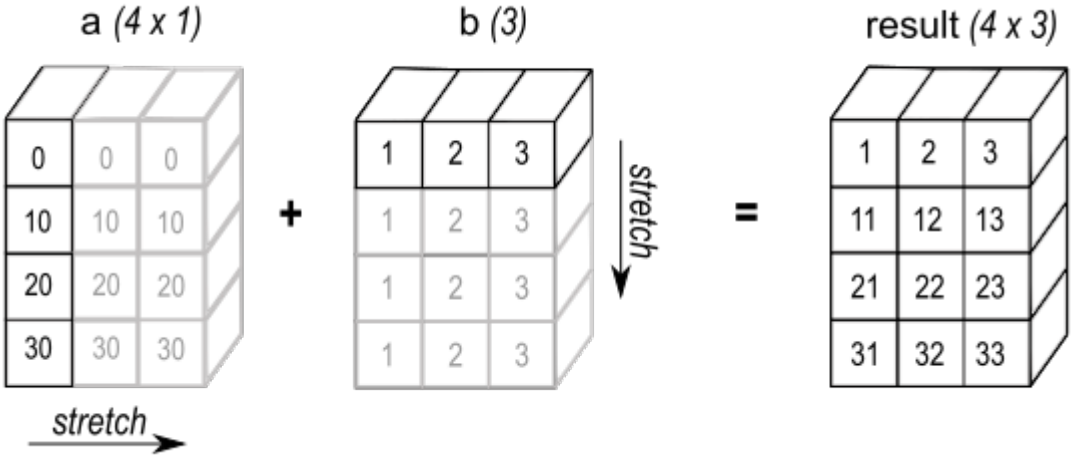


Figure 4 In some cases, broadcasting stretches both arrays to form an output array larger than either of the initial arrays.

Here the `newaxis` index operator inserts a new axis into `a`, making it a two-dimensional `4x1` array. Combining the `4x1` array with `b`, which has shape `(3,)`, yields a `4x3` array.

A practical example: vector quantization

Broadcasting comes up quite often in real world problems. A typical example occurs in the vector quantization (VQ) algorithm used in information theory, classification, and other related areas. The basic operation in VQ finds the closest point in a set of points, called `codes` in VQ jargon, to a given point, called the `observation`. In the very simple, two-dimensional case shown below, the values in `observation` describe the weight and height of an athlete to be classified. The `codes` represent different classes of athletes.[1] Finding the closest point requires calculating the distance between observation and each of the codes. The shortest distance provides the best match. In this example, `codes[0]` is the closest class indicating that the athlete is likely a basketball player.

```
>>> from numpy import array, argmin, sqrt, sum
>>> observation = array([111.0, 188.0])
>>> codes = array([[102.0, 203.0],
...                [132.0, 193.0],
...                [45.0, 155.0],
...                [57.0, 173.0]])
>>> diff = codes - observation    # the broadcast happens here
>>> dist = sqrt(sum(diff**2,axis=-1))
>>> argmin(dist)
0
```

In this example, the `observation` array is stretched to match the shape of the `codes` array:

```
Observation      (1d array):      2
Codes            (2d array):  4 x 2
Diff             (2d array):  4 x 2
```

[1] In this example, weight has more impact on the distance calculation than height because of the larger values. In practice, it is important to normalize the height and weight, often by their standard deviation across the data set, so that both have equal influence on the distance calculation.

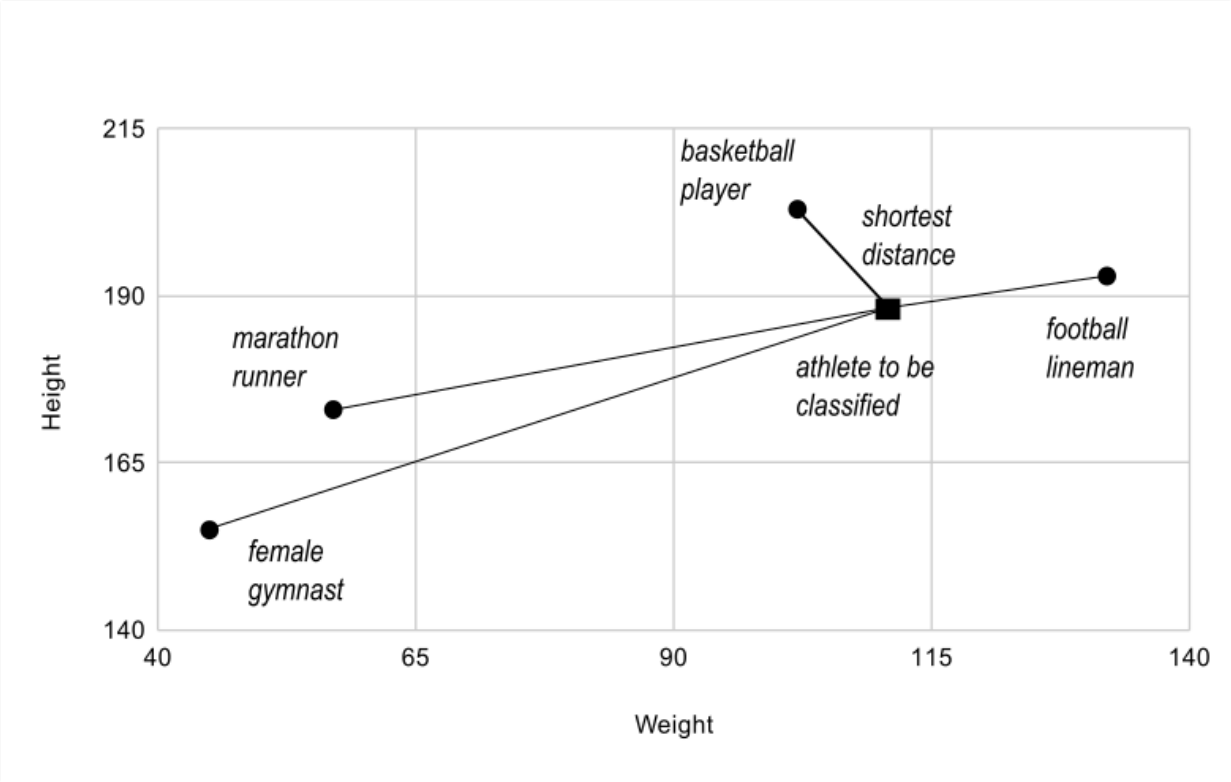


Figure 5 The basic operation of vector quantization calculates the distance between an object to be classified, the dark square, and multiple known codes, the gray circles. In this simple case, the codes represent individual classes. More complex cases use multiple codes per class.

Typically, a large number of `observations`, perhaps read from a database, are compared to a set of `codes`. Consider this scenario:

```
Observation     (2d array):      10 x 3
Codes           (3d array):   5 x 1 x 3
Diff            (3d array):  5 x 10 x 3
```

The three-dimensional array, `diff`, is a consequence of broadcasting, not a necessity for the calculation. Large data sets will generate a large intermediate array that is computationally inefficient. Instead, if each observation is calculated individually using a Python loop around the code in the two-dimensional example above, a much smaller array is used.

Broadcasting is a powerful tool for writing short and usually intuitive code that does its computations very efficiently in C. However, there are cases when broadcasting uses unnecessarily large amounts of memory for a particular algorithm. In these cases, it is better to write the algorithm's outer loop in Python. This may also produce more readable code, as algorithms that use broadcasting tend to become more difficult to interpret as the number of dimensions in the broadcast increases.

Footnotes

Copies and views

Source: https://numpy.org/doc/stable/user/basics.copies.html

When operating on NumPy arrays, it is possible to access the internal data buffer directly using a view without copying data around. This ensures good performance but can also cause unwanted problems if the user is not aware of how this works. Hence, it is important to know the difference between these two terms and to know which operations return copies and which return views.

The NumPy array is a data structure consisting of two parts: the contiguous data buffer with the actual data elements and the metadata that contains information about the data buffer. The metadata includes data type, strides, and other important information that helps manipulate the `.ndarray` easily. See the numpy-internals section for a detailed look.

View

It is possible to access the array differently by just changing certain metadata like stride and dtype without changing the data buffer. This creates a new way of looking at the data and these new arrays are called views. The data buffer remains the same, so any changes made to a view reflects in the original copy. A view can be forced through the `.ndarray.view` method.

Copy

When a new array is created by duplicating the data buffer as well as the metadata, it is called a copy. Changes made to the copy do not reflect on the original array. Making a copy is slower and memory-consuming but sometimes necessary. A copy can be forced by using `.ndarray.copy`.

Indexing operations

Views are created when elements can be addressed with offsets and strides in the original array. Hence, basic indexing always creates views. For example:

```
>>> import numpy as np
>>> x = np.arange(10)
>>> x
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> y = x[1:3]  # creates a view
>>> y
array([1, 2])
>>> x[1:3] = [10, 11]
>>> x
array([ 0, 10, 11,  3,  4,  5,  6,  7,  8,  9])
>>> y
array([10, 11])
```

Here, `y` gets changed when `x` is changed because it is a view.

advanced-indexing, on the other hand, always creates copies. For example:

```
>>> import numpy as np
>>> x = np.arange(9).reshape(3, 3)
```

```
>>> x
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
>>> y = x[[1, 2]]
>>> y
array([[3, 4, 5],
       [6, 7, 8]])
>>> y.base is None
True
```

Here, `y` is a copy, as signified by the `base` attribute. We can also confirm this by assigning new values to `x[[1, 2]]` which in turn will not affect `y` at all:

```
>>> x[[1, 2]] = [[10, 11, 12], [13, 14, 15]]
>>> x
array([[ 0,  1,  2],
       [10, 11, 12],
       [13, 14, 15]])
>>> y
array([[3, 4, 5],
       [6, 7, 8]])
```

It must be noted here that during the assignment of `x[[1, 2]]` no view or copy is created as the assignment happens in-place.

Other operations

The `numpy.reshape` function creates a view where possible or a copy otherwise. In most cases, the strides can be modified to reshape the array with a view. However, in some cases where the array becomes non-contiguous (perhaps after a `.ndarray.transpose` operation), the reshaping cannot be done by modifying strides and requires a copy.

Taking the example of another operation, `numpy.ravel` returns a contiguous flattened view of the array wherever possible. On the other hand, `.ndarray.flatten` always returns a flattened copy of the array. However, to guarantee a view in most cases, `x.reshape(-1)` may be preferable.

How to tell if the array is a view or a copy

The `base` attribute of the ndarray makes it easy to tell if an array is a view or a copy. The base attribute of a view returns the original array while it returns `None` for a copy.

```
>>> import numpy as np
>>> x = np.arange(9)
>>> x
array([0, 1, 2, 3, 4, 5, 6, 7, 8])
>>> y = x.reshape(3, 3)
>>> y
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
>>> y.base  # .reshape() creates a view
array([0, 1, 2, 3, 4, 5, 6, 7, 8])
>>> z = y[[2, 1]]
>>> z
array([[6, 7, 8],
```

```
       [3, 4, 5]])
>>> z.base is None  # advanced indexing creates a copy
True
```

Note that the `base` attribute should not be used to determine if an ndarray object is *new*; only if it is a view or a copy of another ndarray.

Universal functions (ufunc) basics

Source: https://numpy.org/doc/stable/user/basics.ufuncs.html

A universal function (or ufunc for short) is a function that operates on `ndarrays` in an element-by-element fashion, supporting array broadcasting, `type casting <ufuncs.casting>`, and several other standard features. That is, a ufunc is a ”vectorized” wrapper for a function that takes a fixed number of specific inputs and produces a fixed number of specific outputs.

There are also generalized ufuncs which are functions over vectors (or arrays) instead of single-element scalars. For example, `numpy.add` is a ufunc that operates element-by-element, while `numpy.matmul` is a gufunc that operates on vectors/matrices:

```
>>> a = np.arange(6).reshape(3, 2)
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
>>> np.add(a, a)  # element-wise addition
array([[ 0,  2],
       [ 4,  6],
       [ 8, 10]])
>>> np.matmul(a, a.T)  # matrix multiplication (3x2) @ (2x3) -> (3x3)
array([[ 1,  3,  5],
       [ 3, 13, 23],
       [ 5, 23, 41]])
```

In NumPy, universal functions are instances of the `numpy.ufunc` class. Many of the built-in functions are implemented in compiled C code. The basic ufuncs operate on scalars, but there is also a generalized kind for which the basic elements are sub-arrays (vectors, matrices, etc.), and broadcasting is done over other dimensions. The simplest example is the addition operator:

```
>>> np.array([0,2,3,4]) + np.array([1,1,-1,2])
array([1, 3, 2, 6])
```

One can also produce custom `numpy.ufunc` instances using the `numpy.frompyfunc` factory function.

Ufunc methods

All ufuncs have 5 methods. 4 reduce-like methods (`reduce`, `accumulate`, `reduceat`, `outer`) and one for inplace operations (`at`). See ufuncs.methods for more. However, these methods only make sense on ufuncs that take two input arguments and return one output argument (so-called ”scalar” ufuncs since the inner loop operates on a single scalar value). Attempting to call these methods on other ufuncs will cause a `ValueError`.

For example, `numpy.add` takes two inputs and returns one output, so its methods work:

```
>>> np.add.reduce([1, 2, 3])
6
```

But `numpy.divmod` returns two outputs (quotient and remainder), so calling its methods raises an error:

```
>>> np.divmod.reduce([1, 2, 3])
Traceback (most recent call last):
```

```
    ...
ValueError: reduce only supported for functions returning a single value
```

The reduce-like methods all take an *axis* keyword, a *dtype* keyword, and an *out* keyword, and the arrays must all have dimension >= 1. The *axis* keyword specifies the axis of the array over which the reduction will take place (with negative values counting backwards). Generally, it is an integer, though for `numpy.ufunc.reduce`, it can also be a tuple of `int` to reduce over several axes at once, or `None`, to reduce over all axes. For example:

```
>>> x = np.arange(9).reshape(3,3)
>>> x
array([[0, 1, 2],
       [3, 4, 5],
       [6, 7, 8]])
>>> np.add.reduce(x, 1)
array([ 3, 12, 21])
>>> np.add.reduce(x, (0, 1))
36
```

The *dtype* keyword allows you to manage a very common problem that arises when naively using `.ufunc.reduce`. Sometimes you may have an array of a certain data type and wish to add up all of its elements, but the result does not fit into the data type of the array. This commonly happens if you have an array of single-byte integers. The *dtype* keyword allows you to alter the data type over which the reduction takes place (and therefore the type of the output). Thus, you can ensure that the output is a data type with precision large enough to handle your output. The responsibility of altering the reduce type is mostly up to you. There is one exception: if no *dtype* is given for a reduction on the ”add” or ”multiply” operations, then if the input type is an integer (or Boolean) data-type and smaller than the size of the `numpy.int_` data type, it will be internally upcast to the `.int_` (or `numpy.uint`) data-type. In the previous example:

```
>>> x.dtype
dtype('int64')
>>> np.multiply.reduce(x, dtype=np.float64)
array([ 0., 28., 80.])
```

Finally, the *out* keyword allows you to provide an output array (or a tuple of output arrays for multi-output ufuncs). If *out* is given, the *dtype* argument is only used for the internal computations. Considering `x` from the previous example:

```
>>> y = np.zeros(3, dtype=np.int_)
>>> y
array([0, 0, 0])
>>> np.multiply.reduce(x, dtype=np.float64, out=y)
array([ 0, 28, 80])
```

Ufuncs also have a fifth method, `numpy.ufunc.at`, that allows in place operations to be performed using advanced indexing. No buffering is used on the dimensions where advanced indexing is used, so the advanced index can list an item more than once and the operation will be performed on the result of the previous operation for that item.

Output type determination

If the input arguments of the ufunc (or its methods) are `ndarrays`, then the output will be as well. The exception is when the result is zero-dimensional, in which case the output will be converted to an array scalar <arrays.scalars>. This can be avoided by passing in `out=...` or `out=Ellipsis`.

If some or all of the input arguments are not `ndarrays`, then the output may not be an `ndarray` either. Indeed, if any input defines an `__array_ufunc__` method, control will be passed completely

to that function, i.e., the ufunc is overridden.

If none of the inputs overrides the ufunc, then all output arrays will be passed to the `__array_wrap__` method of the input (besides `ndarrays`, and scalars) that defines it **and** has the highest `__array_priority__` of any other input to the universal function. The default `__array_priority__` of the ndarray is 0.0, and the default `__array_priority__` of a subtype is 0.0. Matrices have `__array_priority__` equal to 10.0.

All ufuncs can also take output arguments which must be arrays or subclasses. If necessary, the result will be cast to the data-type(s) of the provided output array(s). If the output has an `__array_wrap__` method it is called instead of the one found on the inputs.

Broadcasting

Each universal function takes array inputs and produces array outputs by performing the core function element-wise on the inputs (where an element is generally a scalar, but can be a vector or higher-order sub-array for generalized ufuncs). Standard broadcasting rules are applied so that inputs not sharing exactly the same shapes can still be usefully operated on.

By these rules, if an input has a dimension size of 1 in its shape, the first data entry in that dimension will be used for all calculations along that dimension. In other words, the stepping machinery of the ufunc will simply not step along that dimension (the stride will be 0 for that dimension).

Type casting rules

> Note
> In NumPy 1.6.0, a type promotion API was created to encapsulate the mechanism for determining output types. See the functions

At the core of every ufunc is a one-dimensional strided loop that implements the actual function for a specific type combination. When a ufunc is created, it is given a static list of inner loops and a corresponding list of type signatures over which the ufunc operates. The ufunc machinery uses this list to determine which inner loop to use for a particular case. You can inspect the `.types <.ufunc.types>` attribute for a particular ufunc to see which type combinations have a defined inner loop and which output type they produce (character codes are used in said output for brevity).

Casting must be done on one or more of the inputs whenever the ufunc does not have a core loop implementation for the input types provided. If an implementation for the input types cannot be found, then the algorithm searches for an implementation with a type signature to which all of the inputs can be cast "safely." The first one it finds in its internal list of loops is selected and performed, after all necessary type casting. Recall that internal copies during ufuncs (even for casting) are limited to the size of an internal buffer (which is user settable).

> Note
> Universal functions in NumPy are flexible enough to have mixed type signatures. Thus, for example, a universal function could be defined that works with floating-point and integer values. See

By the above description, the casting rules are essentially implemented by the question of when a data type can be cast "safely" to another data type. The answer to this question can be determined in Python with a function call: `can_cast(fromtype, totype) <numpy.can_cast>`. The example

below shows the results of this call for the 24 internally supported types on the author's 64-bit system. You can generate this table for your system with the code given in the example.

Example

Code segment showing the ”can cast safely” table for a 64-bit system. Generally the output depends on the system; your system might result in a different table.

```
>>> mark = {False: ' -', True: ' Y'}
>>> def print_table(ntypes):
...     print('X ' + ' '.join(ntypes))
...     for row in ntypes:
...         print(row, end='')
...         for col in ntypes:
...             print(mark[np.can_cast(row, col)], end='')
...         print()
...
>>> print_table(np.typecodes['All'])
X ? b h i l q n p B H I L Q N P e f d g F D G S U V O M m
? Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y - Y
b - Y Y Y Y Y Y - - - - - - - Y Y Y Y Y Y Y Y Y Y Y - Y
h - - Y Y Y Y Y - - - - - - - - Y Y Y Y Y Y Y Y Y Y - Y
i - - - Y Y Y Y - - - - - - - - Y Y - Y Y Y Y Y Y - Y
l - - - - Y Y Y Y - - - - - - - - Y Y - Y Y Y Y Y Y - Y
q - - - - Y Y Y Y - - - - - - - - Y Y - Y Y Y Y Y Y - Y
n - - - - Y Y Y Y - - - - - - - - Y Y - Y Y Y Y Y Y - Y
p - - - - Y Y Y Y - - - - - - - - Y Y - Y Y Y Y Y Y - Y
B - - Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y Y - Y
H - - - Y Y Y Y Y - Y Y Y Y Y Y - Y Y Y Y Y Y Y Y Y - Y
I - - - - Y Y Y Y - - Y Y Y Y Y - - Y Y - Y Y Y Y Y Y - Y
L - - - - - - - - - - - - Y Y Y Y - - Y Y - Y Y Y Y Y Y - -
Q - - - - - - - - - - - - Y Y Y Y - - Y Y - Y Y Y Y Y Y - -
N - - - - - - - - - - - - Y Y Y Y - - Y Y - Y Y Y Y Y Y - -
P - - - - - - - - - - - - Y Y Y Y - - Y Y - Y Y Y Y Y Y - -
e - - - - - - - - - - - - - - - - Y Y Y Y Y Y Y Y Y Y - -
f - - - - - - - - - - - - - - - - Y Y Y Y Y Y Y Y Y Y - -
d - - - - - - - - - - - - - - - - - Y Y - Y Y Y Y Y Y - -
g - - - - - - - - - - - - - - - - - - Y - - Y Y Y Y Y - -
F - - - - - - - - - - - - - - - - - - - Y Y Y Y Y Y Y - -
D - - - - - - - - - - - - - - - - - - - Y Y Y Y Y Y - -
G - - - - - - - - - - - - - - - - - - - - Y Y Y Y Y - -
S - - - - - - - - - - - - - - - - - - - - - Y Y Y Y - -
U - - - - - - - - - - - - - - - - - - - - - - Y Y Y - -
V - - - - - - - - - - - - - - - - - - - - - - - Y Y - -
O - - - - - - - - - - - - - - - - - - - - - - - - Y - -
M - - - - - - - - - - - - - - - - - - - - - - - Y Y Y -
m - - - - - - - - - - - - - - - - - - - - - - - Y Y - Y
```

You should note that, while included in the table for completeness, the 'S', 'U', and 'V' types cannot be operated on by ufuncs. Also, note that on a 32-bit system the integer types may have different sizes, resulting in a slightly altered table.

Mixed scalar-array operations use a different set of casting rules that ensure that a scalar cannot ”upcast” an array unless the scalar is of a fundamentally different kind of data (i.e., under a different hierarchy in the data-type hierarchy) than the array. This rule enables you to use scalar constants in your code (which, as Python types, are interpreted accordingly in ufuncs) without worrying about whether the precision of the scalar constant will cause upcasting on your large (small precision) array.

Use of internal buffers

Internally, buffers are used for misaligned data, swapped data, and data that has to be converted from one data type to another. The size of internal buffers is settable on a per-thread basis. There can be up to $2(n_{\text{inputs}} + n_{\text{outputs}})$ buffers of the specified size created to handle the data from all the inputs and outputs of a ufunc. The default size of a buffer is 10,000 elements. Whenever buffer-based calculation would be needed, but all input arrays are smaller than the buffer size, those misbehaved or incorrectly-typed arrays will be copied before the calculation proceeds. Adjusting the size of the buffer may therefore alter the speed at which ufunc calculations of various sorts are completed. A simple interface for setting this variable is accessible using the function `numpy.setbufsize`.

Error handling

Universal functions can trip special floating-point status registers in your hardware (such as divide-by-zero). If available on your platform, these registers will be regularly checked during calculation. Error handling is controlled on a per-thread basis, and can be configured using the functions `numpy.seterr` and `numpy.seterrcall`.

Overriding ufunc behavior

Classes (including ndarray subclasses) can override how ufuncs act on them by defining certain special methods. For details, see arrays.classes.

Writing performant NumPy code

Source: https://numpy.org/doc/stable/user/basics.performant_code.html

Introduction

NumPy is designed for high performance numerical computing in Python by leveraging vectorized operations. However, vectorization does not always fully utilize the capabilities of multi-core processors. To exploit parallelism, additional strategies are necessary.

In this section, we cover the following topics:

- General concepts for using multi-core processors in Python
- Using multi-core processors with Python standard libraries
- Third party libraries for multi-core processing

General concepts for multi-core processors in Python

Multiprocessing

Multiprocessing is a technique that allows the execution of multiple processes simultaneously, each with its own Python interpreter and memory space. As a high-level API, Python provides the concurrent.futures.ProcessPoolExecutor class to facilitate multiprocessing.

Firstly, we introduce brief Pros and Cons of multiprocessing:

Pros

- Bypasses the Global Interpreter Lock (GIL), allowing true parallelism
- Avoids accidental data sharing due to separate memory spaces

Cons

- Higher memory usage due to separate memory spaces for each process
- Difficulty in sharing data between processes, requiring serialization (pickling) of objects

General tips

The following are general tips for utilizing multiprocessing. Some of these tips are used in the Multiprocessing Example.

Reduce creation overhead

Process creation has a higher overhead compared to thread creation due to the need to initialize a new Python interpreter and memory space. To mitigate this overhead, consider the following strategies:

- Use process pools to reuse existing processes instead of creating new ones for each task. concurrent.futures.ProcessPoolExecutor provides this feature.

- Select appropriate startup methods. Avoid explicitly selecting `fork` unless you know that it is safe in your application. Forking a multithreaded process is problematic and can lead to deadlocks or crashes. Python 3.14 changed the default start method on POSIX platforms from `fork` to `forkserver` to avoid common multithreaded process incompatibilities. See the multiprocessing documentation for more details.

Reduce communication overhead

Inter-process communication (IPC) can introduce significant overhead due to data serialization and transfer between processes. In Python, only picklable objects are allowed to be passed between processes. Due to this limitation, multiprocessing is not suitable for programs which need to serialize data between processes frequently.

To reduce communication overhead, consider the following strategies:

- Minimize the amount of data transferred between processes.
- Use shared memory constructs such as multiprocessing.shared_memory, multiprocessing.Array or multiprocessing.Value for large data that needs to be accessed by multiple processes.
- Balance processing load to ensure that all processes are utilized efficiently and avoid idle time.

Pickling considerations

The worker function and its arguments must be picklable when using multiprocessing. This requirement can become a limitation when working with complex data structures or dynamically defined functions.

If you encounter pickling-related issues, consider the following strategies:

- Refactor your code to use simpler data structures or functions. For example, define worker functions at the top level of a module and avoid lambda or nested functions.
- Consider third-party libraries such as joblib. `joblib`'s default backend `loky` relies on cloudpickle for serialization and can handle a wider range of Python objects than the standard `pickle` module. See the `joblib` documentaion on Serialization of un-picklable objects for more details.

Multithreading

Multithreading allows multiple threads to run within the same process, sharing the same memory space.

Free-threaded Python was introduced experimentally in Python 3.13 and became a supported (non-experimental) feature in Python 3.14. When combined with libraries that are explicitly designed to be thread-safe, this can enable true parallel execution with threads. For details on free-threaded Python builds, see the Python Free-Threading Guide.

As a high-level API, Python provides the concurrent.futures.ThreadPoolExecutor class for thread-based parallelism. Python also provides the concurrent.futures.InterpreterPoolExecutor, which uses multiple interpreters running in separate threads and avoids sharing Python objects between them. However, it is not yet available in NumPy. (See gh-24755 for details.)

The main pros and cons of multithreading are as follows:

Pros

- Lower memory usage since threads share the same memory space
- Easier communication between threads

Cons

- Possibility of race conditions when mutating shared data simultaneously with reads in other threads
- Limited performance improvement if using Python libraries are not thread-safe or have limited support for free-threaded Python builds

General tips

The following are general tips for utilizing multithreading. For more details on thread safety guarantees for built-in types in Python's free-threaded build, see the Python documentation on Thread Safety Guarantees. Some of these tips are used in the Multithreading Example.

Avoid race conditions

Race conditions occur when multiple threads update shared data simultaneously, leading to unpredictable results. To avoid race conditions, consider the following strategies:

- Minimize the amount of shared data between threads by designing your program to use thread-local storage or by passing data explicitly to threads.
- Prefer immutable NumPy arrays or read-only access patterns when possible, since they reduce the need for explicit synchronization.
- Use thread-safe data structures or synchronization primitives like locks, semaphores, or condition variables to manage access to shared data. Note that improper use of these synchronization mechanisms can cause deadlocks, so they should be used with care.

Avoid CPU oversubscription

Some NumPy operations, such as matrix multiplication and linear algebra functions (See Linear Algebra), may use multiple threads provided by the underlying BLAS library (e.g. OpenBLAS, MKL).

If these operations are executed from another thread pool that already uses all available CPU cores, CPU oversubscription can occur. In this situation, both the outer thread pool and the BLAS threads compete for the same CPU resources, which can reduce performance.

To avoid CPU oversubscription, consider the following strategies:

- Limit the number of threads in BLAS to 1, for example using threadpoolctl

Common tips for both multiprocessing and multithreading

Balance processing load

If the processing load is not evenly distributed among workers, some workers may finish their tasks earlier and remain idle while others are still working. It leads to inefficient use of resources and longer overall execution time.

To achieve better load balancing, consider the following strategies:

- Use dynamic task allocation where tasks are assigned to workers as they become available, rather than pre-allocating tasks.
- Check `chunksize` parameter to ensure that tasks are neither too small (causing excessive overhead) nor too large (leading to load imbalance).

Determine the correct number of cpus

Pythons provides os.cpu_count and os.process_cpu_count functions to get the number of CPUs in the system and the current process, respectively. However, in some environments (e.g., Docker containers or HPC clusters), this may not reflect the actual number of CPUs available to the process.

To get a more accurate count of available CPUs, consider the following strategies:

- Use joblib.cpu_count(), which takes into account constraints such as CPU affinity settings and Linux CFS scheduler quotas. (See joblib section for more details about joblib.)

Using multi-core processors with Python standard libraries

In this section, we demonstrate how to use Python's standard libraries to leverage multi-core processors with NumPy.

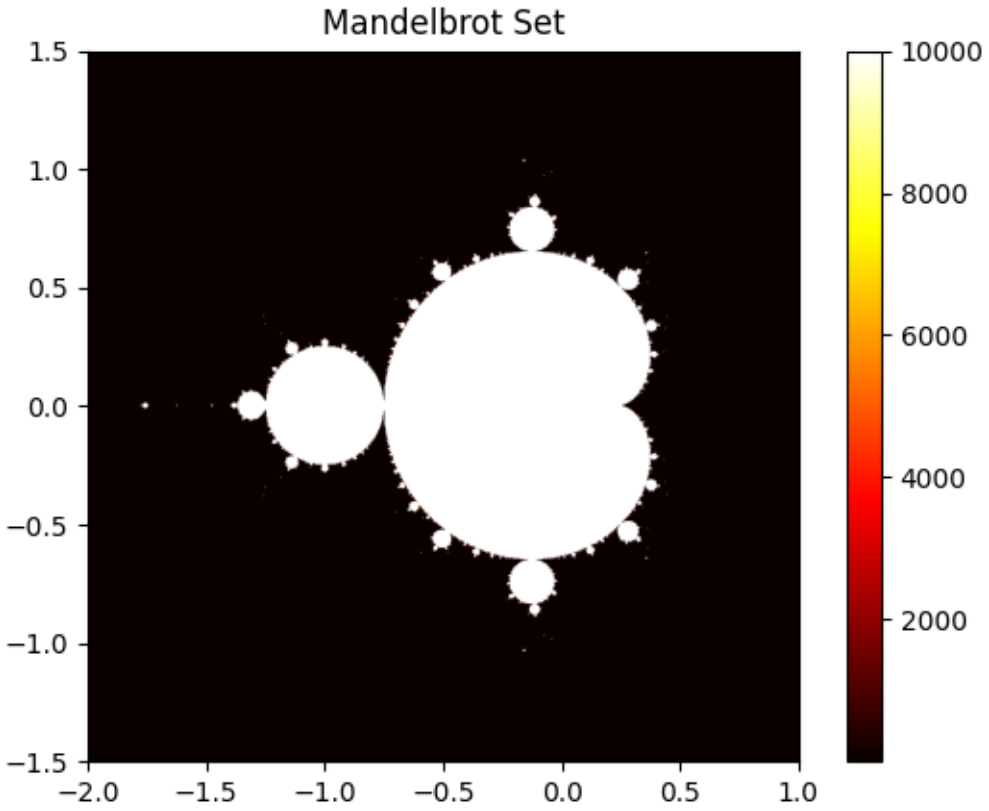
As an example, we use Mandelbrot set generation. Mandelbrot set is defined as the set of complex numbers `c` for which the sequence defined by the iterative function does not diverge to infinity:

$$z_{n+1} = z_n^2 + c, \quad z_0 = 0$$

If the absolute value of z_n remains bounded (i.e., does not exceed a certain threshold, typically `2`) after a fixed number of iterations, then `c` is considered to be in the Mandelbrot set.

Following to this definition, we can calculate each point in the complex plane independently, making it suited for parallel computation.

The hot colors in the image below represent the number of iterations it took for the sequence to diverge for each point in the complex plane.



Multiprocessing Example

The following code demonstrates how to use concurrent.futures.ProcessPoolExecutor to parallelize the Mandelbrot set generation across multiple processes.

This example prioritizes clarity over efficiency. In practice, transferring large NumPy arrays between processes can be expensive. Defining shared-memory arrays or creating arrays within each process may be more efficient implementation.

```
from concurrent.futures import ProcessPoolExecutor

import numpy as np
from numpy.typing import NDArray


def mandelbrot_block(
    c_block: NDArray[np.complex128], max_iter: int
) -> NDArray[np.int64]:
    z = np.zeros(c_block.shape, dtype=np.complex128)
    steps = np.zeros(c_block.shape, dtype=np.int64)

    for _ in range(max_iter):
        mask = np.abs(z) <= 2
        z[mask] = z[mask] * z[mask] + c_block[mask]
        steps[mask] += 1
    return steps
```

```
def mandelbrot_set(
    arr: NDArray[np.complex128],
    max_iter: int,
    n_workers: int,
) -> NDArray[np.int64]:
    n_workers = min(n_workers, arr.size)
    arrs = np.array_split(arr, n_workers)

    with ProcessPoolExecutor(max_workers=n_workers) as pool:
        futures = [
            pool.submit(mandelbrot_block, _arr, max_iter) for _arr in arrs
        ]
        results = [future.result() for future in futures]

    return np.concatenate(results)


if __name__ == '__main__':

    xmin, xmax, ymin, ymax = -2.0, 1.0, -1.5, 1.5
    nx, ny = 800, 800
    max_iter = 10000
    n_workers = 10

    real = np.linspace(xmin, xmax, nx, dtype=np.float64)
    imag = np.linspace(ymin, ymax, ny, dtype=np.float64)

    arr = (real[:, np.newaxis] + 1j * imag[np.newaxis, :]).ravel()
    mandelbrot_image = mandelbrot_set(arr, max_iter, n_workers)
    mandelbrot_image = mandelbrot_image.reshape((nx, ny))
```

Multithreading Example

As in the multiprocessing example, we demonstrate how to use concurrent.futures.ThreadPoolExecutor to parallelize the Mandelbrot set generation across multiple threads. For more detailed explanations and additional examples, see Examples Demonstrating Free-Threaded Python.

Setup

Install a free-threaded build Python

Before running the multithreading example, ensure you have a free-threaded build of Python 3.13 or later. About how to install a free-threaded build of Python, please refer to the Installing Free-Threaded Python.

According to the Python documentation, there are several ways to verify if your Python build is free-threaded.

- Run `python -VV` in your terminal and check `free-threading build` is shown
- Check the value of sys._is_gil_enabled() in a Python shell, which should return False.

Code Example

The following code demonstrates how to use concurrent.futures.ThreadPoolExecutor to parallelize the Mandelbrot set generation across multiple threads.

This implementation shares several arrays between threads. For example, `SHARED_readonly_arr` is a read-only array that holds the complex numbers to be evaluated, and `SHARED_updating_steps` is an array that holds the number of iterations for each point.

```
import sys
from concurrent.futures import ThreadPoolExecutor

import numpy as np


def mandelbrot_block(start: int, stop: int, max_iter: int) -> None:
    z_target = np.zeros(stop - start, dtype=np.complex128)

    indexes = slice(start, stop)
    arr_target = SHARED_readonly_arr[indexes]
    steps_target = SHARED_updating_steps[indexes]

    threshold = 2.0
    for _ in range(max_iter):
        mask = np.abs(z_target) <= threshold
        z_target[mask] = z_target[mask] * z_target[mask] + arr_target[mask]
        steps_target[mask] += 1
    SHARED_updating_steps[indexes] = steps_target
    return None


def mandelbrot_set(
    total_size: int,
    max_iter: int,
    n_workers: int,
) -> None:

    chunksize = total_size // n_workers
    with ThreadPoolExecutor(max_workers=n_workers) as pool:
        futures = [
            pool.submit(
                mandelbrot_block, start, min(start + chunksize, total_size), max_iter
            )
            for start in range(0, total_size, chunksize)
        ]
        _ = [future.result() for future in futures]


if __name__ == '__main__':

    print("Python version is free-threaded:", not sys._is_gil_enabled())
    assert not sys._is_gil_enabled()

    xmin, xmax, ymin, ymax = -2.0, 1.0, -1.5, 1.5
    nx, ny = 800, 800
    max_iter = 10000
    n_workers = 10

    real = np.linspace(xmin, xmax, nx, dtype=np.float64)
    imag = np.linspace(ymin, ymax, ny, dtype=np.float64)

    SHARED_readonly_arr = (real[:, np.newaxis] + 1j * imag[np.newaxis, :]).ravel()
    SHARED_readonly_arr.flags.writeable = False

    SHARED_updating_steps = np.zeros(SHARED_readonly_arr.shape, dtype=np.int64)
```

```
    mandelbrot_set(SHARED_readonly_arr.size, max_iter, n_workers)
    mandelbrot_image = SHARED_updating_steps.reshape((nx, ny))
```

Third Party Libraries for Multi-Core Processing

In many practical scenarios, third-party libraries can provide more convenient and efficient solutions than using Python's standard libraries.

Dask

Dask is an open-source library that provides parallel compuing features not only for a single machine but also for a cluster of machines. It also provides `DaskArray` which has a similar API to NumPy's `ndarray`. If you are familiar with NumPy, you can easily get started with `DaskArray`.

- Dask Documentaion: https://docs.dask.org/en/stable/
- Dask GitHub Repository: https://github.com/dask/dask

joblib

`joblib` is a library that provides helper functions which make it easy to parallelize tasks.

For example,

- `joblib`'s default backend `loky` relies on cloudpickle for serialization and can handle a wider range of Python objects than the standard `pickle` module. (e.g., lambda functions)
- joblib.cpu_count() returns the number of CPUs available to the current process, taking into account constraints such as CPU affinity settings and Linux CFS scheduler quotas. This may provide a more accurate value than os.cpu_count and os.process_cpu_count functions in Docker containers and other resource-constrained environments.

For more details on `joblib`, see the following resources:

- joblib Documentation: https://joblib.readthedocs.io/en/latest/
- joblib GitHub Repository: https://github.com/joblib/joblib

threadpoolctl

`threadpoolctl` is a library that provides utilities to control the behavior of thread pools in Python, including other thread pools used by libraries such as BLAS and OpenMP. It allows you to avoid CPU oversubscription when using multiple libraries that utilize threads.

For more details on `threadpoolctl`, see the following resources:

- threadpoolctl GitHub Repository: https://github.com/joblib/threadpoolctl

API quick reference

Generated from the docstrings of the installed NumPy 2.5.4 package, which are the text the official API reference pages are built from. Full reference: https://numpy.org/doc/stable/reference/index.html

Array creation

np.array

```
np.array(object, dtype=None, *, copy=True, order='K', subok=False, ndmin=0, ndmax=0, like=None)
```

Create an array.

Parameters

`object` : *array_like* An array, any object exposing the array interface, an object whose `__array__` method returns an array, or any (nested) sequence. If object is a scalar, a 0-dimensional array containing object is returned.

`dtype` : *data-type, optional* The desired data-type for the array. If not given, NumPy will try to use a default `dtype` that can represent the values (by applying promotion rules when necessary.)

`copy` : *bool, optional* If `True` (default), then the array data is copied. If `None`, a copy will only be made if `__array__` returns a copy, if obj is a nested sequence, or if a copy is needed to satisfy any of the other requirements (`dtype`, `order`, etc.). Note that any copy of the data is shallow, i.e., for arrays with object dtype, the new array will point to the same objects. See Examples for ndarray.copy. For `False` it raises a `ValueError` if a copy cannot be avoided. Default: `True`.

`order` : *{'K', 'A', 'C', 'F'}, optional* Specify the memory layout of the array. If object is not an array, the newly created array will be in C order (row major) unless 'F' is specified, in which case it will be in Fortran order (column major). If object is an array the following holds.

order	no copy	copy=True
'K'	unchanged	F & C order preserved, otherwise most similar order
'A'	unchanged	F order if input is F and not C, otherwise C order
'C'	C order	C order
'F'	F order	F order

When `copy=None` and a copy is made for other reasons, the result is the same as if `copy=True`, with some exceptions for 'A', see the Notes section. The default order is 'K'.

`subok` : *bool, optional* If True, then sub-classes will be passed-through, otherwise the returned array will be forced to be a base-class array (default).

`ndmin` : *int, optional* Specifies the minimum number of dimensions that the resulting array should have. Ones will be prepended to the shape as needed to meet this requirement.

`ndmax` : *int, optional* Specifies the maximum number of dimensions to create when inferring shape from nested sequences. By default (ndmax=0), NumPy recurses through all nesting

levels (up to the compile-time constant `NPY_MAXDIMS`). Setting `ndmax` stops recursion at the specified depth, preserving deeper nested structures as objects instead of promoting them to higher-dimensional arrays. In this case, `dtype=np.object_` is required.

New in version 2.4.0.

`like` : *array_like, optional* Reference object to allow the creation of arrays which are not NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

`out` : *ndarray* An array object satisfying the specified requirements.

See also: `empty_like`, `ones_like`, `zeros_like`, `full_like`, `empty`, `ones`, `zeros`, `full`, `copy`

Notes

When order is 'A' and `object` is an array in neither 'C' nor 'F' order, and a copy is forced by a change in dtype, then the order of the result is not necessarily 'C' as expected. This is likely a bug.

Examples

```
>>> np.array([1, 2, 3])
array([1, 2, 3])
```

Upcasting:

```
>>> np.array([1, 2, 3.0])
array([ 1.,  2.,  3.])
```

More than one dimension:

```
>>> np.array([[1, 2], [3, 4]])
array([[1, 2],
       [3, 4]])
```

Minimum dimensions 2:

```
>>> np.array([1, 2, 3], ndmin=2)
array([[1, 2, 3]])
...
```

np.asarray

```
np.asarray(a, dtype=None, order=None, *, device=None, copy=None, like=None)
```

Convert the input to an array.

Parameters

`a` : *array_like* Input data, in any form that can be converted to an array. This includes lists, lists of tuples, tuples, tuples of tuples, tuples of lists and ndarrays.

`dtype` : *data-type, optional* By default, the data-type is inferred from the input data.

`order` : *{'C', 'F', 'A', 'K'}, optional* The memory layout of the output. 'C' gives a row-major layout (C-style), 'F' gives a column-major layout (Fortran-style). 'C' and 'F' will copy if needed to ensure the output format. 'A' (any) is equivalent to 'F' if input a is non-contiguous or

Fortran-contiguous, otherwise, it is equivalent to 'C'. Unlike 'C' or 'F', 'A' does not ensure that the result is contiguous. 'K' (keep) is the default and preserves the input order for the output.

device : *str, optional* The device on which to place the created array. Default: `None`. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

copy : *bool, optional* If `True`, then the object is copied. If `None` then the object is copied only if needed, i.e. if `__array__` returns a copy, if obj is a nested sequence, or if a copy is needed to satisfy any of the other requirements (`dtype`, `order`, etc.). For `False` it raises a `ValueError` if a copy cannot be avoided. Default: `None`.

New in version 2.0.0.

like : *array_like, optional* Reference object to allow the creation of arrays which are not NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

out : *ndarray* Array interpretation of `a`. No copy is performed if the input is already an ndarray with matching dtype and order. If `a` is a subclass of ndarray, a base class ndarray is returned.

See also: `asanyarray`, `ascontiguousarray`, `asfortranarray`, `asarray_chkfinite`, `fromiter`, `fromfunction`

Examples

Convert a list into an array:

```
>>> a = [1, 2]
>>> np.asarray(a)
array([1, 2])
```

Existing arrays are not copied:

```
>>> a = np.array([1, 2])
>>> np.asarray(a) is a
True
```

If dtype is set, array is copied only if dtype does not match:

```
>>> a = np.array([1, 2], dtype=np.float32)
>>> np.shares_memory(np.asarray(a, dtype=np.float32), a)
True
>>> np.shares_memory(np.asarray(a, dtype=np.float64), a)
False
...
```

np.zeros

```
np.zeros(shape, dtype=None, order='C', *, device=None, like=None)
```

Return a new array of given shape and type, filled with zeros.

Parameters

shape : *int or tuple of ints* Shape of the new array, e.g., `(2, 3)` or `2`.

dtype : *data-type, optional* The desired data-type for the array, e.g., numpy.int8. Default is numpy.float64.

order : *{'C', 'F'}, optional, default: 'C'* Whether to store multi-dimensional data in row-major (C-style) or column-major (Fortran-style) order in memory.

device : *str, optional* The device on which to place the created array. Default: `None`. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

like : *array_like, optional* Reference object to allow the creation of arrays which are not NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

out : *ndarray* Array of zeros with the given shape, dtype, and order.

See also: `zeros_like`, `empty`, `ones`, `full`

Examples

```
>>> np.zeros(5)
array([ 0.,  0.,  0.,  0.,  0.])
```

```
>>> np.zeros((5,), dtype=np.int_)
array([0, 0, 0, 0, 0])
```

```
>>> np.zeros((2, 1))
array([[ 0.],
       [ 0.]])
```

```
>>> s = (2,2)
>>> np.zeros(s)
array([[ 0.,  0.],
       [ 0.,  0.]])
```

```
>>> np.zeros((2,), dtype=[('x', 'i4'), ('y', 'i4')]) # custom dtype
array([(0, 0), (0, 0)],
      dtype=[('x', '<i4'), ('y', '<i4')])
...
```

np.ones

```
np.ones(shape, dtype=None, order='C', *, device=None, like=None)
```

Return a new array of given shape and type, filled with ones.

Parameters

shape : *int or sequence of ints* Shape of the new array, e.g., `(2, 3)` or `2`.

dtype : *data-type, optional* The desired data-type for the array, e.g., numpy.int8. Default is numpy.float64.

order : *{'C', 'F'}, optional, default: C* Whether to store multi-dimensional data in row-major (C-style) or column-major (Fortran-style) order in memory.

device : *str, optional* The device on which to place the created array. Default: None. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

like : *array_like, optional* **Reference object to allow the creation of arrays which are not** NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

out : *ndarray* Array of ones with the given shape, dtype, and order.

See also: `ones_like`, `empty`, `zeros`, `full`

Examples

```
>>> np.ones(5)
array([1., 1., 1., 1., 1.])
```

```
>>> np.ones((5,), dtype=np.int_)
array([1, 1, 1, 1, 1])
```

```
>>> np.ones((2, 1))
array([[1.],
       [1.]])
```

```
>>> s = (2,2)
>>> np.ones(s)
array([[1.,  1.],
       [1.,  1.]])
```

np.full

```
np.full(shape, fill_value, dtype=None, order='C', *, device=None, like=None)
```

Return a new array of given shape and type, filled with fill_value.

Parameters

shape : *int or sequence of ints* Shape of the new array, e.g., `(2, 3)` or `2`.

fill_value : *scalar or array_like* Fill value.

dtype : *data-type, optional* **The desired data-type for the array The default, None, means** `np.array(fill_value).dtype`.

order : *{'C', 'F'}, optional* Whether to store multidimensional data in C- or Fortran-contiguous (row- or column-wise) order in memory.

device : *str, optional* The device on which to place the created array. Default: None. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

like : *array_like, optional* **Reference object to allow the creation of arrays which are not** NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

out : *ndarray* Array of fill_value with the given shape, dtype, and order.

See also: `full_like`, `empty`, `ones`, `zeros`

Examples

```
>>> np.full((2, 2), np.inf)
array([[inf, inf],
       [inf, inf]])
>>> np.full((2, 2), 10)
array([[10, 10],
       [10, 10]])
```

```
>>> np.full((2, 2), [1, 2])
array([[1, 2],
       [1, 2]])
```

np.empty

```
np.empty(shape, dtype=None, order='C', *, device=None, like=None)
```

Return a new array of given shape and type, without initializing entries.

Parameters

shape : *int or tuple of int* Shape of the empty array, e.g., `(2, 3)` or `2`.

dtype : *data-type, optional* Desired output data-type for the array, e.g, numpy.int8. Default is numpy.float64.

order : *{'C', 'F'}, optional, default: 'C'* Whether to store multi-dimensional data in row-major (C-style) or column-major (Fortran-style) order in memory.

device : *str, optional* The device on which to place the created array. Default: `None`. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

like : *array_like, optional* Reference object to allow the creation of arrays which are not NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

out : *ndarray* Array of uninitialized (arbitrary) data of the given shape, dtype, and order. Object arrays will be initialized to None.

See also: `empty_like`, `ones`, `zeros`, `full`

Notes

Unlike other array creation functions (e.g. zeros, ones, full), empty does not initialize the values of the array, and may therefore be marginally faster. However, the values stored in the newly allocated array are arbitrary. For reproducible behavior, be sure to set each element of the array before reading.

Examples

```
>>> np.empty([2, 2])
array([[ -9.74499359e+001,   6.69583040e-309],
       [  2.13182611e-314,   3.06959433e-309]])         #uninitialized
```

```
>>> np.empty([2, 2], dtype=np.int_)
array([[-1073741821, -1067949133],
       [  496041986,    19249760]])                     #uninitialized
```

np.arange

```
np.arange(start_or_stop, /, stop=None, step=1, *, dtype=None, device=None, like=None)
```

Return evenly spaced values within a given interval.

`arange` can be called with a varying number of positional arguments:

- `arange(stop)`: Values are generated within the half-open interval `[0, stop)` (in other words, the interval including start but excluding stop).
- `arange(start, stop)`: Values are generated within the half-open interval `[start, stop)`.
- `arange(start, stop, step)` Values are generated within the half-open interval `[start, stop)`, with spacing between values given by `step`.

For integer arguments the function is roughly equivalent to the Python built-in `range`, but returns an ndarray rather than a `range` instance.

When using a non-integer step, such as 0.1, it is often better to use numpy.linspace.

See the Warning sections below for more information.

Parameters

`start` : *integer or real, optional* Start of interval. The interval includes this value. The default start value is 0.

`stop` : *integer or real* End of interval. The interval does not include this value, except in some cases where step is not an integer and floating point round-off affects the length of out.

`step` : *integer or real, optional* Spacing between values. For any output out, this is the distance between two adjacent values, `out[i+1] - out[i]`. The default step size is 1. If step is specified as a position argument, start must also be given.

`dtype` : *dtype, optional* The type of the output array. If dtype is not given, infer the data type from the other input arguments.

`device` : *str, optional* The device on which to place the created array. Default: `None`. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

`like` : *array_like, optional* Reference object to allow the creation of arrays which are not NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

`arange` : *ndarray* Array of evenly spaced values.

For floating point arguments, the length of the result is `ceil((stop - start)/step)`. Because of floating point overflow, this rule may result in the last element of out being greater than stop.

See also: `numpy.linspace`, `numpy.ogrid`, `numpy.mgrid`, `how-to-partition`

Examples

```
>>> np.arange(3)
array([0, 1, 2])
>>> np.arange(3.0)
array([ 0.,  1.,  2.])
>>> np.arange(3,7)
array([3, 4, 5, 6])
>>> np.arange(3,7,2)
array([3, 5])
```

np.linspace

```
np.linspace(start, stop, num=50, endpoint=True, retstep=False, dtype=None, axis=0, *, device=None)
```

Return evenly spaced numbers over a specified interval.

Returns num evenly spaced samples, calculated over the interval [start, stop].

The endpoint of the interval can optionally be excluded.

Changed in version 1.20.0. Values are rounded towards `-inf` instead of `0` when an integer `dtype` is specified. The old behavior can still be obtained with `np.linspace(start, stop, num).astype(np.int_)`

Parameters

`start` : *array_like* The starting value of the sequence.

`stop` : *array_like* The end value of the sequence, unless endpoint is set to False. In that case, the sequence consists of all but the last of `num + 1` evenly spaced samples, so that stop is excluded. Note that the step size changes when endpoint is False.

`num` : *int, optional* Number of samples to generate. Default is 50. Must be non-negative.

`endpoint` : *bool, optional* If True, stop is the last sample. Otherwise, it is not included. Default is True.

`retstep` : *bool, optional* If True, return (samples, step), where step is the spacing between samples.

`dtype` : *dtype, optional* The type of the output array. If dtype is not given, the data type is inferred from start and stop. The inferred dtype will never be an integer; float is chosen even if the arguments would produce an array of integers.

`axis` : *int, optional* The axis in the result to store the samples. Relevant only if start or stop are array-like. By default (0), the samples will be along a new axis inserted at the beginning. Use -1 to get an axis at the end.

`device` : *str, optional* The device on which to place the created array. Default: None. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

Returns

`samples` : *ndarray* There are num equally spaced samples in the closed interval `[start, stop]` or the half-open interval `[start, stop)` (depending on whether endpoint is True or False).

`step` : *float, optional* Only returned if retstep is True

Size of spacing between samples.

See also: `arange`, `geomspace`, `logspace`, `how-to-partition`

Examples

```
>>> np.linspace(2.0, 3.0, num=5)
array([2.  , 2.25, 2.5 , 2.75, 3.  ])
>>> np.linspace(2.0, 3.0, num=5, endpoint=False)
array([2. ,  2.2,  2.4,  2.6,  2.8])
>>> np.linspace(2.0, 3.0, num=5, retstep=True)
(array([2.  ,  2.25,  2.5 ,  2.75,  3.  ]), 0.25)
```

Graphical illustration:

```
>>> import matplotlib.pyplot as plt
>>> N = 8
>>> y = np.zeros(N)
>>> x1 = np.linspace(0, 10, N, endpoint=True)
>>> x2 = np.linspace(0, 10, N, endpoint=False)
>>> plt.plot(x1, y, 'o')
[<matplotlib.lines.Line2D object at 0x...>]
>>> plt.plot(x2, y + 0.5, 'o')
[<matplotlib.lines.Line2D object at 0x...>]
>>> plt.ylim([-0.5, 1])
(-0.5, 1)
>>> plt.show()
...
```

np.eye

```
np.eye(N, M=None, k=0, dtype=<class 'float'>, order='C', *, device=None, like=None)
```

Return a 2-D array with ones on the diagonal and zeros elsewhere.

Parameters

`N` : *int* Number of rows in the output.

`M` : *int, optional* Number of columns in the output. If None, defaults to N.

`k` : *int, optional* Index of the diagonal: 0 (the default) refers to the main diagonal, a positive value refers to an upper diagonal, and a negative value to a lower diagonal.

`dtype` : *data-type, optional* Data-type of the returned array.

`order` : *{'C', 'F'}, optional* Whether the output should be stored in row-major (C-style) or column-major (Fortran-style) order in memory.

`device` : *str, optional* The device on which to place the created array. Default: None. For Array-API interoperability only, so must be `"cpu"` if passed.

New in version 2.0.0.

`like` : *array_like, optional* **Reference object to allow the creation of arrays which are not** NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

`I` : *ndarray of shape (N,M)* An array where all elements are equal to zero, except for the k-th diagonal, whose values are equal to one.

See also: `identity`, `diag`

Examples

```
>>> np.eye(2, dtype=np.int_)
array([[1, 0],
       [0, 1]])
>>> np.eye(3, k=1)
array([[0.,  1.,  0.],
       [0.,  0.,  1.],
       [0.,  0.,  0.]])
```

np.random.default_rng

```
np.random.default_rng(seed=None)
```

Construct a new Generator with the default BitGenerator (PCG64).

Parameters

seed : *{None, int, array_like[ints], SeedSequence, BitGenerator, Generator, RandomState}, optiona*
A seed to initialize the BitGenerator. If None, then fresh, unpredictable entropy will be pulled from the OS. If an `int` or `array_like[ints]` is passed, then all values must be non-negative and will be passed to SeedSequence to derive the initial BitGenerator state. One may also pass in a SeedSequence instance. Additionally, when passed a BitGenerator, it will be wrapped by Generator. If passed a Generator, it will be returned unaltered. When passed a legacy RandomState instance it will be coerced to a Generator.

Returns

Generator The initialized generator object.

Notes

If `seed` is not a BitGenerator or a Generator, a new BitGenerator is instantiated. This function does not manage a default global instance.

See seeding_and_entropy for more information about seeding.

Examples

default_rng is the recommended constructor for the random number class Generator. Here are several ways we can construct a random number generator using default_rng and the Generator class.

Here we use default_rng to generate a random float:

```
>>> rng = np.random.default_rng(12345)
>>> print(rng)
Generator(PCG64)
>>> rfloat = rng.random()
>>> rfloat
0.22733602246716966
>>> type(rfloat)
<class 'float'>
```

Here we use default_rng to generate 3 random integers between 0 (inclusive) and 10 (exclusive):
…

Shape manipulation

np.reshape

```
np.reshape(a, /, shape, order='C', *, copy=None)
```

Returns a reshaped ndarray without changing data.

Parameters

a : *array_like* Array to be reshaped.

shape : *int or tuple of ints* The new shape should be compatible with the original shape. If an integer, then the result will be a 1-D array of that length. One shape dimension can be -1. In this case, the value is inferred from the length of the array and remaining dimensions.

order : *{'C', 'F', 'A'}, optional* Read the elements of `a` using this index order, and place the elements into the reshaped array using this index order. 'C' means to read / write the elements using C-like index order, with the last axis index changing fastest, back to the first axis index changing slowest. 'F' means to read / write the elements using Fortran-like index order, with the first index changing fastest, and the last index changing slowest. Note that the 'C' and 'F' options take no account of the memory layout of the underlying array, and only refer to the order of indexing. 'A' means to read / write the elements in Fortran-like index order if `a` is Fortran *contiguous* in memory, C-like order otherwise.

copy : *bool, optional* If `True`, then the array data is copied. If `None`, a copy will only be made if it's required by `order`. For `False` it raises a `ValueError` if a copy cannot be avoided. Default: `None`.

Returns

reshaped_array : *ndarray* This will be a new view object if possible; otherwise, it will be a copy. Note there is no guarantee of the *memory layout* (C- or Fortran- contiguous) of the returned array.

See also: `ndarray.reshape`

Notes

It is not always possible to change the shape of an array without copying the data.

The `order` keyword gives the index ordering both for *fetching* the values from `a`, and then *placing* the values into the output array. For example, let's say you have an array:

```
>>> a = np.arange(6).reshape((3, 2))
>>> a
array([[0, 1],
       [2, 3],
       [4, 5]])
```

(Notes truncated; see the full reference page.)

Examples

```
>>> a = np.array([[1,2,3], [4,5,6]])
>>> np.reshape(a, 6)
array([1, 2, 3, 4, 5, 6])
>>> np.reshape(a, 6, order='F')
array([1, 4, 2, 5, 3, 6])
```

```
>>> np.reshape(a, (3,-1))       # the unspecified value is inferred to be 2
array([[1, 2],
```

```
       [3, 4],
       [5, 6]])
```

np.transpose

```
np.transpose(a, axes=None)
```

Returns an array with axes transposed.

For a 1-D array, this returns an unchanged view of the original array, as a transposed vector is simply the same vector. To convert a 1-D array into a 2-D column vector, an additional dimension must be added, e.g., `np.atleast_2d(a).T` achieves this, as does `a[:, np.newaxis]`. For a 2-D array, this is the standard matrix transpose. For an n-D array, if axes are given, their order indicates how the axes are permuted (see Examples). If axes are not provided, then `transpose(a).shape == a.shape[::-1]`.

Parameters

a : *array_like* Input array.

axes : *tuple or list of ints, optional* If specified, it must be a tuple or list which contains a permutation of [0, 1, ..., N-1] where N is the number of axes of a. Negative indices can also be used to specify axes. The i-th axis of the returned array will correspond to the axis numbered `axes[i]` of the input. If not specified, defaults to `range(a.ndim)[::-1]`, which reverses the order of the axes.

Returns

p : *ndarray* a with its axes permuted. A view is returned whenever possible.

See also: `ndarray.transpose`, `moveaxis`, `argsort`

Notes

Use `transpose(a, argsort(axes))` to invert the transposition of tensors when using the axes keyword argument.

Examples

```
>>> a = np.array([[1, 2], [3, 4]])
>>> a
array([[1, 2],
       [3, 4]])
>>> np.transpose(a)
array([[1, 3],
       [2, 4]])
```

```
>>> a = np.array([1, 2, 3, 4])
>>> a
array([1, 2, 3, 4])
>>> np.transpose(a)
array([1, 2, 3, 4])
```

```
>>> a = np.ones((1, 2, 3))
>>> np.transpose(a, (1, 0, 2)).shape
(2, 1, 3)
...
```

np.concatenate

```
np.concatenate(arrays, /, axis=0, out=None, *, dtype=None, casting='same_kind')
```

Join a sequence of arrays along an existing axis.

New in version 2.0. `numpy.concat` added as a shorthand for `numpy.concatenate`.

Parameters

`a1, a2, ...` : *sequence of array_like* The arrays must have the same shape, except in the dimension corresponding to axis (the first, by default).

`axis` : *int, optional* The axis along which the arrays will be joined. If axis is None, arrays are flattened before use. Default is 0.

`out` : *ndarray, optional* If provided, the destination to place the result. The shape must be correct, matching that of what concatenate would have returned if no out argument were specified.

`dtype` : *str or dtype* If provided, the destination array will have this dtype. Cannot be provided together with out.

New in version 1.20.0.

`casting` : *{'no', 'equiv', 'safe', 'same_kind', 'unsafe'}, optional* Controls what kind of data casting may occur. Defaults to 'same_kind'. For a description of the options, please see casting.

New in version 1.20.0.

Returns

`res` : *ndarray* The concatenated array.

See also: `ma.concatenate`, `array_split`, `split`, `hsplit`, `vsplit`, `dsplit`, `stack`, `block`, `hstack`, `vstack`, `dstack`, `column_stack`

Notes

When one or more of the arrays to be concatenated is a MaskedArray, this function will return a MaskedArray object instead of an ndarray, but the input masks are *not* preserved. In cases where a MaskedArray is expected as input, use the ma.concatenate function from the masked array module instead.

Examples

```
>>> a = np.array([[1, 2], [3, 4]])
>>> b = np.array([[5, 6]])
>>> np.concatenate((a, b), axis=0)
array([[1, 2],
       [3, 4],
       [5, 6]])
>>> np.concatenate((a, b.T), axis=1)
array([[1, 2, 5],
       [3, 4, 6]])
>>> np.concatenate((a, b), axis=None)
array([1, 2, 3, 4, 5, 6])
```

This function will not preserve masking of MaskedArray inputs.

```
>>> a = np.ma.arange(3)
>>> a[1] = np.ma.masked
>>> b = np.arange(2, 5)
>>> a
```

```
masked_array(data=[0, --, 2],
             mask=[False,  True, False],
       fill_value=999999)
>>> b
array([2, 3, 4])
>>> np.concatenate([a, b])
...
```

np.stack

```
np.stack(arrays, axis=0, out=None, *, dtype=None, casting='same_kind')
```

Join a sequence of arrays along a new axis.

The `axis` parameter specifies the index of the new axis in the dimensions of the result. For example, if `axis=0` it will be the first dimension and if `axis=-1` it will be the last dimension.

Parameters

`arrays` : ***sequence of ndarrays*** Each array must have the same shape. In the case of a single ndarray array_like input, it will be treated as a sequence of arrays; i.e., each element along the zeroth axis is treated as a separate array.

`axis` : ***int, optional*** The axis in the result array along which the input arrays are stacked.

`out` : ***ndarray, optional*** If provided, the destination to place the result. The shape must be correct, matching that of what stack would have returned if no out argument were specified.

`dtype` : ***str or dtype*** If provided, the destination array will have this dtype. Cannot be provided together with out.

New in version 1.24.

`casting` : ***{'no', 'equiv', 'safe', 'same_kind', 'unsafe'}, optional*** Controls what kind of data casting may occur. Defaults to 'same_kind'.

New in version 1.24.

Returns

`stacked` : ***ndarray*** The stacked array has one more dimension than the input arrays.

See also: `concatenate`, `block`, `split`, `unstack`

Examples

```
>>> rng = np.random.default_rng()
>>> arrays = [rng.normal(size=(3,4)) for _ in range(10)]
>>> np.stack(arrays, axis=0).shape
(10, 3, 4)
```

```
>>> np.stack(arrays, axis=1).shape
(3, 10, 4)
```

```
>>> np.stack(arrays, axis=2).shape
(3, 4, 10)
```

```
>>> a = np.array([1, 2, 3])
>>> b = np.array([4, 5, 6])
>>> np.stack((a, b))
array([[1, 2, 3],
```

```
       [4, 5, 6]])
...
```

np.vstack

```
np.vstack(tup, *, dtype=None, casting='same_kind')
```

Stack arrays in sequence vertically (row wise).

This is equivalent to concatenation along the first axis after 1-D arrays of shape (N,) have been reshaped to (1,N). Rebuilds arrays divided by vsplit.

This function makes most sense for arrays with up to 3 dimensions. For instance, for pixel-data with a height (first axis), width (second axis), and r/g/b channels (third axis). The functions concatenate, stack and block provide more general stacking and concatenation operations.

Parameters

tup : *sequence of ndarrays* The arrays must have the same shape along all but the first axis. 1-D arrays must have the same length. In the case of a single array_like input, it will be treated as a sequence of arrays; i.e., each element along the zeroth axis is treated as a separate array.

dtype : *str or dtype* If provided, the destination array will have this dtype. Cannot be provided together with out.

New in version 1.24.

casting : *{'no', 'equiv', 'safe', 'same_kind', 'unsafe'}, optional* Controls what kind of data casting may occur. Defaults to 'same_kind'.

New in version 1.24.

Returns

stacked : *ndarray* The array formed by stacking the given arrays, will be at least 2-D.

See also: `concatenate`, `stack`, `block`, `hstack`, `dstack`, `column_stack`, `vsplit`, `unstack`

Examples

```
>>> a = np.array([1, 2, 3])
>>> b = np.array([4, 5, 6])
>>> np.vstack((a,b))
array([[1, 2, 3],
       [4, 5, 6]])
```

```
>>> a = np.array([[1], [2], [3]])
>>> b = np.array([[4], [5], [6]])
>>> np.vstack((a,b))
array([[1],
       [2],
       [3],
       [4],
       [5],
       [6]])
```

np.hstack

```
np.hstack(tup, *, dtype=None, casting='same_kind')
```

Stack arrays in sequence horizontally (column wise).

This is equivalent to concatenation along the second axis, except for 1-D arrays where it concatenates along the first axis. Rebuilds arrays divided by hsplit.

This function makes most sense for arrays with up to 3 dimensions. For instance, for pixel-data with a height (first axis), width (second axis), and r/g/b channels (third axis). The functions concatenate, stack and block provide more general stacking and concatenation operations.

Parameters

tup : *sequence of ndarrays* The arrays must have the same shape along all but the second axis, except 1-D arrays which can be any length. In the case of a single array_like input, it will be treated as a sequence of arrays; i.e., each element along the zeroth axis is treated as a separate array.

dtype : *str or dtype* If provided, the destination array will have this dtype. Cannot be provided together with out.

New in version 1.24.

casting : *{'no', 'equiv', 'safe', 'same_kind', 'unsafe'}, optional* Controls what kind of data casting may occur. Defaults to 'same_kind'.

New in version 1.24.

Returns

stacked : *ndarray* The array formed by stacking the given arrays.

See also: `concatenate`, `stack`, `block`, `vstack`, `dstack`, `column_stack`, `hsplit`, `unstack`

Examples

```
>>> a = np.array((1,2,3))
>>> b = np.array((4,5,6))
>>> np.hstack((a,b))
array([1, 2, 3, 4, 5, 6])
>>> a = np.array([[1],[2],[3]])
>>> b = np.array([[4],[5],[6]])
>>> np.hstack((a,b))
array([[1, 4],
       [2, 5],
       [3, 6]])
```

np.split

```
np.split(ary, indices_or_sections, axis=0)
```

Split an array into multiple sub-arrays as views into ary.

Parameters

ary : *ndarray* Array to be divided into sub-arrays.

indices_or_sections : *int or 1-D array* If indices_or_sections is an integer, N, the array will be divided into N equal arrays along axis. If such a split is not possible, an error is raised.

If indices_or_sections is a 1-D array of sorted integers, the entries indicate where along axis the array is split. For example, `[2, 3]` would, for `axis=0`, result in

- ary[:2]
- ary[2:3]
- ary[3:]

If an index exceeds the dimension of the array along axis, an empty sub-array is returned correspondingly.

`axis` : *int, optional* The axis along which to split, default is 0.

Returns

`sub-arrays` : *list of ndarrays* A list of sub-arrays as views into ary.

Raises

ValueError If indices_or_sections is given as an integer, but a split does not result in equal division.

See also: `array_split`, `hsplit`, `vsplit`, `dsplit`, `concatenate`, `stack`, `hstack`, `vstack`, `dstack`

Examples

```
>>> x = np.arange(9.0)
>>> np.split(x, 3)
[array([0.,  1.,  2.]), array([3.,  4.,  5.]), array([6.,  7.,  8.])]
```

```
>>> x = np.arange(8.0)
>>> np.split(x, [3, 5, 6, 10])
[array([0.,  1.,  2.]),
 array([3.,  4.]),
 array([5.]),
 array([6.,  7.]),
 array([], dtype=float64)]
```

np.squeeze

```
np.squeeze(a, axis=None)
```

Remove axes of length one from a.

Parameters

`a` : *array_like* Input data.

`axis` : *None or int or tuple of ints, optional* Selects a subset of the entries of length one in the shape. If an axis is selected with shape entry greater than one, an error is raised.

Returns

`squeezed` : *ndarray* The input array, but with all or a subset of the dimensions of length 1 removed. This is always a itself or a view into a. Note that if all axes are squeezed, the result is a 0d array and not a scalar.

Raises

ValueError If axis is not None, and an axis being squeezed is not of length 1

See also: `expand_dims`, `reshape`

Examples

```
>>> x = np.array([[[0], [1], [2]]])
>>> x.shape
(1, 3, 1)
>>> np.squeeze(x).shape
(3,)
>>> np.squeeze(x, axis=0).shape
(3, 1)
>>> np.squeeze(x, axis=1).shape
Traceback (most recent call last):
...
ValueError: cannot select an axis to squeeze out which has size
not equal to one
>>> np.squeeze(x, axis=2).shape
(1, 3)
>>> x = np.array([[1234]])
>>> x.shape
(1, 1)
>>> np.squeeze(x)
array(1234)  # 0d array
>>> np.squeeze(x).shape
()
>>> np.squeeze(x)[()]
1234
...
```

np.expand_dims

```
np.expand_dims(a, axis)
```

Expand the shape of an array.

Insert a new axis that will appear at the axis position in the expanded array shape.

Parameters

a : *array_like* Input array.

axis : *int or tuple of ints* Position in the expanded axes where the new axis (or axes) is placed.

> *Deprecated since version 1.13.0.* Passing an axis where `axis > a.ndim` will be treated as `axis == a.ndim`, and passing `axis < -a.ndim - 1` will be treated as `axis == 0`. This behavior is deprecated.

Returns

result : *ndarray* View of a with the number of dimensions increased.

See also: `squeeze`, `reshape`, `atleast_1d`, `atleast_2d`, `atleast_3d`

Examples

```
>>> x = np.array([1, 2])
>>> x.shape
(2,)
```

The following is equivalent to `x[np.newaxis, :]` or `x[np.newaxis]`:

```
>>> y = np.expand_dims(x, axis=0)
>>> y
array([[1, 2]])
```

```
>>> y.shape
(1, 2)
```

The following is equivalent to `x[:, np.newaxis]`:

```
>>> y = np.expand_dims(x, axis=1)
>>> y
array([[1],
       [2]])
>>> y.shape
(2, 1)
...
```

np.ravel

```
np.ravel(a, order='C')
```

Return a contiguous flattened array.

A 1-D array, containing the elements of the input, is returned. A copy is made only if needed.

As of NumPy 1.10, the returned array will have the same type as the input array. (for example, a masked array will be returned for a masked array input)

Parameters

a : *array_like* Input array. The elements in a are read in the order specified by order, and packed as a 1-D array.

order : *{'C','F', 'A', 'K'}, optional* The elements of a are read using this index order. 'C' means to index the elements in row-major, C-style order, with the last axis index changing fastest, back to the first axis index changing slowest. 'F' means to index the elements in column-major, Fortran-style order, with the first index changing fastest, and the last index changing slowest. Note that the 'C' and 'F' options take no account of the memory layout of the underlying array, and only refer to the order of axis indexing. 'A' means to read the elements in Fortran-like index order if a is Fortran *contiguous* in memory, C-like order otherwise. 'K' means to read the elements in the order they occur in memory, except for reversing the data when strides are negative. By default, 'C' index order is used.

Returns

y : *array_like* y is a contiguous 1-D array of the same subtype as a, with shape `(a.size,)`. Note that matrices are special cased for backward compatibility, if a is a matrix, then y is a 1-D ndarray.

See also: `ndarray.flat`, `ndarray.flatten`, `ndarray.reshape`

Notes

In row-major, C-style order, in two dimensions, the row index varies the slowest, and the column index the quickest. This can be generalized to multiple dimensions, where row-major order implies that the index along the first axis varies slowest, and the index along the last quickest. The opposite holds for column-major, Fortran-style index ordering.

When a view is desired in as many cases as possible, `arr.reshape(-1)` may be preferable. However, `ravel` supports `K` in the optional `order` argument while `reshape` does not.

Examples

It is equivalent to `reshape(-1, order=order)`.

```
>>> x = np.array([[1, 2, 3], [4, 5, 6]])
>>> np.ravel(x)
array([1, 2, 3, 4, 5, 6])
```

```
>>> x.reshape(-1)
array([1, 2, 3, 4, 5, 6])
```

```
>>> np.ravel(x, order='F')
array([1, 4, 2, 5, 3, 6])
```

When `order` is 'A', it will preserve the array's 'C' or 'F' ordering:

```
>>> np.ravel(x.T)
array([1, 4, 2, 5, 3, 6])
>>> np.ravel(x.T, order='A')
array([1, 2, 3, 4, 5, 6])
...
```

Math and statistics

np.sum

```
np.sum(a, axis=None, dtype=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>)
```

Sum of array elements over a given axis.

Parameters

`a` : *array_like* Elements to sum.

`axis` : *None or int or tuple of ints, optional* Axis or axes along which a sum is performed. The default, axis=None, will sum all of the elements of the input array. If axis is negative it counts from the last to the first axis. If axis is a tuple of ints, a sum is performed on all of the axes specified in the tuple instead of a single axis or all the axes as before.

`dtype` : *dtype, optional* The type of the returned array and of the accumulator in which the elements are summed. The dtype of a is used by default unless a has an integer dtype of less precision than the default platform integer. In that case, if a is signed then the platform integer is used while if a is unsigned then an unsigned integer of the same precision as the platform integer is used.

`out` : *ndarray, optional* Alternative output array in which to place the result. It must have the same shape as the expected output, but the type of the output values will be cast if necessary.

`keepdims` : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

If the default value is passed, then keepdims will not be passed through to the sum method of sub-classes of ndarray, however any non-default value will be. If the sub-class' method does not implement keepdims any exceptions will be raised.

`initial` : *scalar, optional* Starting value for the sum. See ~numpy.ufunc.reduce for details.

`where` : *array_like of bool, optional* Elements to include in the sum. See ~numpy.ufunc.reduce for details.

Returns

`sum_along_axis` : *ndarray* An array with the same shape as a, with the specified axis removed. If a is a 0-d array, or if axis is None, a scalar is returned. If an output array is specified, a reference to out is returned.

See also: `ndarray.sum`, `add`, `cumsum`, `trapezoid`, `mean`, `average`

Notes

Arithmetic is modular when using integer types, and no error is raised on overflow.

The sum of an empty array is the neutral element 0:

```
>>> np.sum([])
0.0
```

(Notes truncated; see the full reference page.)

Examples

```
>>> np.sum([0.5, 1.5])
2.0
>>> np.sum([0.5, 0.7, 0.2, 1.5], dtype=np.int32)
np.int32(1)
>>> np.sum([[0, 1], [0, 5]])
6
>>> np.sum([[0, 1], [0, 5]], axis=0)
array([0, 6])
>>> np.sum([[0, 1], [0, 5]], axis=1)
array([1, 5])
>>> np.sum([[0, 1], [np.nan, 5]], where=[False, True], axis=1)
array([1., 5.])
```

If the accumulator is too small, overflow occurs:

```
>>> np.ones(128, dtype=np.int8).sum(dtype=np.int8)
np.int8(-128)
...
```

np.mean

```
np.mean(a, axis=None, dtype=None, out=None, keepdims=<no value>, *, where=<no value>)
```

Compute the arithmetic mean along the specified axis.

Returns the average of the array elements. The average is taken over the flattened array by default, otherwise over the specified axis. float64 intermediate and return values are used for integer inputs.

Parameters

`a` : *array_like* Array containing numbers whose mean is desired. If a is not an array, a conversion is attempted.

`axis` : *None or int or tuple of ints, optional* Axis or axes along which the means are computed. The default is to compute the mean of the flattened array.

If this is a tuple of ints, a mean is performed over multiple axes, instead of a single axis or all the axes as before.

`dtype` : *data-type, optional* Type to use in computing the mean. For integer inputs, the default is float64; for floating point inputs, it is the same as the input dtype.

out : *ndarray, optional* Alternate output array in which to place the result. The default is None; if provided, it must have the same shape as the expected output, but the type will be cast if necessary. See ufuncs-output-type for more details. See ufuncs-output-type for more details.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

If the default value is passed, then keepdims will not be passed through to the mean method of sub-classes of ndarray, however any non-default value will be. If the sub-class' method does not implement keepdims any exceptions will be raised.

where : *array_like of bool, optional* Elements to include in the mean. See ~numpy.ufunc.reduce for details.

New in version 1.20.0.

Returns

m : *ndarray, see dtype parameter above* If out=None, returns a new array containing the mean values, otherwise a reference to the output array is returned.

See also: `average`, `std`, `var`, `nanmean`, `nanstd`, `nanvar`

Notes

The arithmetic mean is the sum of the elements along the axis divided by the number of elements.

Note that for floating-point input, the mean is computed using the same precision the input has. Depending on the input data, this can cause the results to be inaccurate, especially for float32 (see example below). Specifying a higher-precision accumulator using the dtype keyword can alleviate this issue.

By default, float16 results are computed using float32 intermediates for extra precision.

Examples

```
>>> a = np.array([[1, 2], [3, 4]])
>>> np.mean(a)
2.5
>>> np.mean(a, axis=0)
array([2., 3.])
>>> np.mean(a, axis=1)
array([1.5, 3.5])
```

In single precision, mean can be inaccurate:

```
>>> a = np.zeros((2, 512*512), dtype=np.float32)
>>> a[0, :] = 1.0
>>> a[1, :] = 0.1
>>> np.mean(a)
np.float32(0.54999924)
```

Computing the mean in float64 is more accurate: ...

np.median

```
np.median(a, axis=None, out=None, overwrite_input=False, keepdims=False)
```

Compute the median along the specified axis.

Returns the median of the array elements.

Parameters

a : *array_like* Input array or object that can be converted to an array.

axis : *{int, sequence of int, None}, optional* Axis or axes along which the medians are computed. The default, axis=None, will compute the median along a flattened version of the array. If a sequence of axes, the array is first flattened along the given axes, then the median is computed along the resulting flattened axis.

out : *ndarray, optional* Alternative output array in which to place the result. It must have the same shape and buffer length as the expected output, but the type (of the output) will be cast if necessary.

overwrite_input : *bool, optional* If True, then allow use of memory of input array a for calculations. The input array will be modified by the call to median. This will save memory when you do not need to preserve the contents of the input array. Treat the input as undefined, but it will probably be fully or partially sorted. Default is False. If overwrite_input is `True` and a is not already an ndarray, an error will be raised.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the original arr.

Returns

median : *ndarray* A new array holding the result. If the input contains integers or floats smaller than `float64`, then the output data-type is `np.float64`. Otherwise, the data-type of the output is the same as that of the input. If out is specified, that array is returned instead.

See also: `mean`, `percentile`

Notes

Given a vector `V` of length `N`, the median of `V` is the middle value of a sorted copy of `V`, `V_sorted` - i e., `V_sorted[(N-1)/2]`, when `N` is odd, and the average of the two middle values of `V_sorted` when `N` is even.

Examples

```
>>> a = np.array([[10, 7, 4], [3, 2, 1]])
>>> a
array([[10,  7,  4],
       [ 3,  2,  1]])
>>> np.median(a)
np.float64(3.5)
>>> np.median(a, axis=0)
array([6.5, 4.5, 2.5])
>>> np.median(a, axis=1)
array([7.,  2.])
>>> np.median(a, axis=(0, 1))
np.float64(3.5)
>>> m = np.median(a, axis=0)
>>> out = np.zeros_like(m)
>>> np.median(a, axis=0, out=m)
array([6.5,  4.5,  2.5])
>>> m
array([6.5,  4.5,  2.5])
>>> b = a.copy()
>>> np.median(b, axis=1, overwrite_input=True)
array([7.,  2.])
>>> assert not np.all(a==b)
>>> b = a.copy()
```

```
>>> np.median(b, axis=None, overwrite_input=True)
...
```

np.std

```
np.std(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>, *, where=<no value>, mean=<no
↪  value>, correction=<no value>)
```

Compute the standard deviation along the specified axis.

Returns the standard deviation, a measure of the spread of a distribution, of the array elements. The standard deviation is computed for the flattened array by default, otherwise over the specified axis.

Parameters

a : *array_like* Calculate the standard deviation of these values.

axis : *None or int or tuple of ints, optional* Axis or axes along which the standard deviation is computed. The default is to compute the standard deviation of the flattened array. If this is a tuple of ints, a standard deviation is performed over multiple axes, instead of a single axis or all the axes as before.

dtype : *dtype, optional* Type to use in computing the standard deviation. For arrays of integer type the default is float64, for arrays of float types it is the same as the array type.

out : *ndarray, optional* Alternative output array in which to place the result. It must have the same shape as the expected output but the type (of the calculated values) will be cast if necessary. See ufuncs-output-type for more details.

ddof : *{int, float}, optional* Means Delta Degrees of Freedom. The divisor used in calculations is `N - ddof`, where `N` represents the number of elements. By default ddof is zero. See Notes for details about use of ddof.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

If the default value is passed, then keepdims will not be passed through to the std method of sub-classes of ndarray, however any non-default value will be. If the sub-class' method does not implement keepdims any exceptions will be raised.

where : *array_like of bool, optional* Elements to include in the standard deviation. See ~numpy.ufunc.reduce for details.

New in version 1.20.0.

mean : *array_like, optional* Provide the mean to prevent its recalculation. The mean should have a shape as if it was calculated with `keepdims=True`. The axis for the calculation of the mean should be the same as used in the call to this std function.

New in version 2.0.0.

correction : *{int, float}, optional* Array API compatible name for the `ddof` parameter. Only one of them can be provided at the same time.

New in version 2.0.0.

Returns

standard_deviation : *ndarray, see dtype parameter above.* If out is None, return a new array containing the standard deviation, otherwise return a reference to the output array.

See also: `var`, `mean`, `nanmean`, `nanstd`, `nanvar`, `ufuncs-output-type`

Notes

There are several common variants of the array standard deviation calculation. Assuming the input a is a one-dimensional NumPy array and `mean` is either provided as an argument or computed as `a.mean()`, NumPy computes the standard deviation of an array as:

```
N = len(a)
d2 = abs(a - mean)**2  # abs is for complex `a`
var = d2.sum() / (N - ddof)  # note use of `ddof`
std = var**0.5
```

Different values of the argument ddof are useful in different contexts. NumPy's default `ddof=0` corresponds with the expression:

(Notes truncated; see the full reference page.)

Examples

```
>>> a = np.array([[1, 2], [3, 4]])
>>> np.std(a)
1.1180339887498949 # may vary
>>> np.std(a, axis=0)
array([1.,  1.])
>>> np.std(a, axis=1)
array([0.5,  0.5])
```

In single precision, std() can be inaccurate:

```
>>> a = np.zeros((2, 512*512), dtype=np.float32)
>>> a[0, :] = 1.0
>>> a[1, :] = 0.1
>>> np.std(a)
np.float32(0.45000005)
```

Computing the standard deviation in float64 is more accurate: ...

np.var

```
np.var(a, axis=None, dtype=None, out=None, ddof=0, keepdims=<no value>, *, where=<no value>, mean=<no
↪  value>, correction=<no value>)
```

Compute the variance along the specified axis.

Returns the variance of the array elements, a measure of the spread of a distribution. The variance is computed for the flattened array by default, otherwise over the specified axis.

Parameters

`a` : ***array_like*** Array containing numbers whose variance is desired. If a is not an array, a conversion is attempted.

`axis` : ***None or int or tuple of ints, optional*** Axis or axes along which the variance is computed. The default is to compute the variance of the flattened array. If this is a tuple of ints, a variance is performed over multiple axes, instead of a single axis or all the axes as before.

`dtype` : ***data-type, optional*** Type to use in computing the variance. For arrays of integer type the default is float64; for arrays of float types it is the same as the array type.

`out` : ***ndarray, optional*** Alternate output array in which to place the result. It must have the same shape as the expected output, but the type is cast if necessary.

ddof : *{int, float}, optional* "Delta Degrees of Freedom": the divisor used in the calculation is `N - ddof`, where `N` represents the number of elements. By default ddof is zero. See notes for details about use of ddof.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

If the default value is passed, then keepdims will not be passed through to the var method of sub-classes of ndarray, however any non-default value will be. If the sub-class' method does not implement keepdims any exceptions will be raised.

where : *array_like of bool, optional* Elements to include in the variance. See ~numpy.ufunc.reduce for details.

New in version 1.20.0.

mean : *array like, optional* Provide the mean to prevent its recalculation. The mean should have a shape as if it was calculated with `keepdims=True`. The axis for the calculation of the mean should be the same as used in the call to this var function.

New in version 2.0.0.

correction : *{int, float}, optional* Array API compatible name for the `ddof` parameter. Only one of them can be provided at the same time.

New in version 2.0.0.

Returns

variance : *ndarray, see dtype parameter above* If `out=None`, returns a new array containing the variance; otherwise, a reference to the output array is returned.

See also: `std`, `mean`, `nanmean`, `nanstd`, `nanvar`, `ufuncs-output-type`

Notes

There are several common variants of the array variance calculation. Assuming the input a is a one-dimensional NumPy array and `mean` is either provided as an argument or computed as `a.mean()`, NumPy computes the variance of an array as:

```
N = len(a)
d2 = abs(a - mean)**2  # abs is for complex `a`
var = d2.sum() / (N - ddof)  # note use of `ddof`
```

Different values of the argument ddof are useful in different contexts. NumPy's default `ddof=0` corresponds with the expression:

(Notes truncated; see the full reference page.)

Examples

```
>>> a = np.array([[1, 2], [3, 4]])
>>> np.var(a)
1.25
>>> np.var(a, axis=0)
array([1.,  1.])
>>> np.var(a, axis=1)
array([0.25,  0.25])
```

In single precision, var() can be inaccurate:

```
>>> a = np.zeros((2, 512*512), dtype=np.float32)
>>> a[0, :] = 1.0
>>> a[1, :] = 0.1
```

```
>>> np.var(a)
np.float32(0.20250003)
```

Computing the variance in float64 is more accurate: ...

np.min

```
np.min(a, axis=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>)
```

Return the minimum of an array or minimum along an axis.

Parameters

a : *array_like* Input data.

axis : *None or int or tuple of ints, optional* Axis or axes along which to operate. By default, flattened input is used.

If this is a tuple of ints, the minimum is selected over multiple axes, instead of a single axis or all the axes as before.

out : *ndarray, optional* Alternative output array in which to place the result. Must be of the same shape and buffer length as the expected output. See ufuncs-output-type for more details.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

If the default value is passed, then keepdims will not be passed through to the `min` method of sub-classes of ndarray, however any non-default value will be. If the sub-class' method does not implement keepdims any exceptions will be raised.

initial : *scalar, optional* The maximum value of an output element. Must be present to allow computation on empty slice. See ~numpy.ufunc.reduce for details.

where : *array_like of bool, optional* Elements to compare for the minimum. See ~numpy.ufunc.reduce for details.

Returns

min : *ndarray or scalar* Minimum of a. If axis is None, the result is a scalar value. If axis is an int, the result is an array of dimension `a.ndim - 1`. If axis is a tuple, the result is an array of dimension `a.ndim - len(axis)`.

See also: `max`, `nanmin`, `minimum`, `fmin`, `argmin`, `nanmax`, `maximum`, `fmax`

Notes

NaN values are propagated, that is if at least one item is NaN, the corresponding min value will be NaN as well. To ignore NaN values (MATLAB behavior), please use nanmin.

Don't use ~numpy.min for element-wise comparison of 2 arrays; when `a.shape[0]` is 2, `minimum(a[0], a[1])` is faster than `min(a, axis=0)`.

Examples

```
>>> a = np.arange(4).reshape((2,2))
>>> a
array([[0, 1],
       [2, 3]])
>>> np.min(a)           # Minimum of the flattened array
0
```

```
>>> np.min(a, axis=0)   # Minima along the first axis
array([0, 1])
>>> np.min(a, axis=1)   # Minima along the second axis
array([0, 2])
>>> np.min(a, where=[False, True], initial=10, axis=0)
array([10,  1])
```

```
>>> b = np.arange(5, dtype=np.float64)
>>> b[2] = np.nan
>>> np.min(b)
np.float64(nan)
>>> np.min(b, where=~np.isnan(b), initial=10)
0.0
>>> np.nanmin(b)
0.0
...
```

np.max

```
np.max(a, axis=None, out=None, keepdims=<no value>, initial=<no value>, where=<no value>)
```

Return the maximum of an array or maximum along an axis.

Parameters

a : ***array_like*** Input data.

axis : ***None or int or tuple of ints, optional*** Axis or axes along which to operate. By default, flattened input is used. If this is a tuple of ints, the maximum is selected over multiple axes, instead of a single axis or all the axes as before.

out : ***ndarray, optional*** Alternative output array in which to place the result. Must be of the same shape and buffer length as the expected output. See ufuncs-output-type for more details.

keepdims : ***bool, optional*** If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the input array.

If the default value is passed, then keepdims will not be passed through to the `max` method of sub-classes of ndarray, however any non-default value will be. If the sub-class' method does not implement keepdims any exceptions will be raised.

initial : ***scalar, optional*** The minimum value of an output element. Must be present to allow computation on empty slice. See ~numpy.ufunc.reduce for details.

where : ***array_like of bool, optional*** Elements to compare for the maximum. See ~numpy.ufunc.reduce for details.

Returns

max : ***ndarray or scalar*** Maximum of a. If axis is None, the result is a scalar value. If axis is an int, the result is an array of dimension `a.ndim - 1`. If axis is a tuple, the result is an array of dimension `a.ndim - len(axis)`.

See also: `min`, `nanmax`, `maximum`, `fmax`, `argmax`, `nanmin`, `minimum`, `fmin`

Notes

NaN values are propagated, that is if at least one item is NaN, the corresponding max value will be NaN as well. To ignore NaN values (MATLAB behavior), please use nanmax.

Don't use ~numpy.max for element-wise comparison of 2 arrays; when `a.shape[0]` is 2, `maximum(a[0], a[1])` is faster than `max(a, axis=0)`.

Examples

```
>>> a = np.arange(4).reshape((2,2))
>>> a
array([[0, 1],
       [2, 3]])
>>> np.max(a)           # Maximum of the flattened array
3
>>> np.max(a, axis=0)   # Maxima along the first axis
array([2, 3])
>>> np.max(a, axis=1)   # Maxima along the second axis
array([1, 3])
>>> np.max(a, where=[False, True], initial=-1, axis=0)
array([-1,  3])
>>> b = np.arange(5, dtype=np.float64)
>>> b[2] = np.nan
>>> np.max(b)
np.float64(nan)
>>> np.max(b, where=~np.isnan(b), initial=-1)
4.0
>>> np.nanmax(b)
4.0
...
```

np.argmax

```
np.argmax(a, axis=None, out=None, *, keepdims=<no value>)
```

Returns the indices of the maximum values along an axis.

Parameters

a : *array_like* Input array.

axis : *int, optional* By default, the index is into the flattened array, otherwise along the specified axis.

out : *array, optional* If provided, the result will be inserted into this array. It should be of the appropriate shape and dtype.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the array.

New in version 1.22.0.

Returns

index_array : *ndarray of ints* Array of indices into the array. It has the same shape as `a.shape` with the dimension along axis removed. If keepdims is set to True, then the size of axis will be 1 with the resulting array having same shape as `a.shape`.

See also: `ndarray.argmax`, `argmin`, `amax`, `unravel_index`, `take_along_axis`

Notes

In case of multiple occurrences of the maximum values, the indices corresponding to the first occurrence are returned.

Examples

```
>>> a = np.arange(6).reshape(2,3) + 10
>>> a
array([[10, 11, 12],
       [13, 14, 15]])
>>> np.argmax(a)
5
>>> np.argmax(a, axis=0)
array([1, 1, 1])
>>> np.argmax(a, axis=1)
array([2, 2])
```

Indexes of the maximal elements of an N-dimensional array:

```
>>> a.flat[np.argmax(a)]
15
>>> ind = np.unravel_index(np.argmax(a, axis=None), a.shape)
>>> ind
(1, 2)
>>> a[ind]
15
...
```

np.cumsum

```
np.cumsum(a, axis=None, dtype=None, out=None)
```

Return the cumulative sum of the elements along a given axis.

Parameters

a : *array_like* Input array.

axis : *int, optional* Axis along which the cumulative sum is computed. The default (None) is to compute the cumsum over the flattened array.

dtype : *dtype, optional* Type of the returned array and of the accumulator in which the elements are summed. If dtype is not specified, it defaults to the dtype of a, unless a has an integer dtype with a precision less than that of the default platform integer. In that case, the default platform integer is used.

out : *ndarray, optional* Alternative output array in which to place the result. It must have the same shape and buffer length as the expected output but the type will be cast if necessary. See ufuncs-output-type for more details.

Returns

cumsum_along_axis : *ndarray.* A new array holding the result is returned unless out is specified, in which case a reference to out is returned. The result has the same size as a, and the same shape as a if axis is not None or a is a 1-d array.

See also: `cumulative_sum`, `sum`, `trapezoid`, `diff`

Notes

Arithmetic is modular when using integer types, and no error is raised on overflow.

`cumsum(a)[-1]` may not be equal to `sum(a)` for floating-point values since `sum` may use a pairwise summation routine, reducing the roundoff-error. See sum for more information.

Examples

```
>>> a = np.array([[1,2,3], [4,5,6]])
>>> a
array([[1, 2, 3],
       [4, 5, 6]])
>>> np.cumsum(a)
array([ 1,  3,  6, 10, 15, 21])
>>> np.cumsum(a, dtype=np.float64)  # specifies type of output value(s)
array([  1.,   3.,   6.,  10.,  15.,  21.])
```

```
>>> np.cumsum(a,axis=0)      # sum over rows for each of the 3 columns
array([[1, 2, 3],
       [5, 7, 9]])
>>> np.cumsum(a,axis=1)      # sum over columns for each of the 2 rows
array([[ 1,  3,  6],
       [ 4,  9, 15]])
```

`cumsum(b)[-1]` may not be equal to `sum(b)` ...

np.percentile

```
np.percentile(a, q, axis=None, out=None, overwrite_input=False, method='linear', keepdims=False, *,
↪  weights=None)
```

Compute the q-th percentile of the data along the specified axis.

Returns the q-th percentile(s) of the array elements.

Parameters

`a` : *array_like of real numbers* Input array or object that can be converted to an array.

`q` : *array_like of float* Percentage or sequence of percentages for the percentiles to compute. Values must be between 0 and 100 inclusive.

`axis` : *{int, tuple of int, None}, optional* Axis or axes along which the percentiles are computed. The default is to compute the percentile(s) along a flattened version of the array.

`out` : *ndarray, optional* Alternative output array in which to place the result. It must have the same shape and buffer length as the expected output, but the type (of the output) will be cast if necessary.

`overwrite_input` : *bool, optional* If True, then allow the input array a to be modified by intermediate calculations, to save memory. In this case, the contents of the input a after this function completes is undefined.

`method` : *str, optional* This parameter specifies the method to use for estimating the percentile. There are many different methods, some unique to NumPy. See the notes for explanation. The options sorted by their R type as summarized in the H&F paper are:

1. 'inverted_cdf'
2. 'averaged_inverted_cdf'
3. 'closest_observation'
4. 'interpolated_inverted_cdf'
5. 'hazen'
6. 'weibull'
7. 'linear' (default)
8. 'median_unbiased'
9. 'normal_unbiased'

The first three methods are discontinuous. NumPy further defines the following discontinuous variations of the default 'linear' (7.) option:

- 'lower'
- 'higher',
- 'midpoint'
- 'nearest'

Changed in version 1.22.0. This argument was previously called "interpolation" and only offered the "linear" default and last four options.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the original array a.

weights : *array_like, optional* An array of weights associated with the values in a. Each value in a contributes to the percentile according to its associated weight. The weights array can either be 1-D (in which case its length must be the size of a along the given axis) or of the same shape as a. If weights=None, then all data in a are assumed to have a weight equal to one. Only method="inverted_cdf" supports weights. See the notes for more details.

New in version 2.0.0.

Returns

percentile : *scalar or ndarray* If q is a single percentile and axis=None, then the result is a scalar. If multiple percentiles are given, first axis of the result corresponds to the percentiles. The other axes are the axes that remain after the reduction of a. If the input contains integers or floats smaller than `float64`, the output data-type is `float64`. Otherwise, the output data-type is the same as that of the input. If out is specified, that array is returned instead.

See also: `mean`, `median`, `nanpercentile`, `quantile`

Notes

The behavior of numpy.percentile with percentage q is that of numpy.quantile with argument `q/100`. For more information, please see numpy.quantile.

Examples

```
>>> a = np.array([[10, 7, 4], [3, 2, 1]])
>>> a
array([[10,  7,  4],
       [ 3,  2,  1]])
>>> np.percentile(a, 50)
3.5
>>> np.percentile(a, 50, axis=0)
array([6.5, 4.5, 2.5])
>>> np.percentile(a, 50, axis=1)
array([7.,  2.])
>>> np.percentile(a, 50, axis=1, keepdims=True)
array([[7.],
       [2.]])
```

```
>>> m = np.percentile(a, 50, axis=0)
>>> out = np.zeros_like(m)
>>> np.percentile(a, 50, axis=0, out=out)
array([6.5, 4.5, 2.5])
>>> m
array([6.5, 4.5, 2.5])
...
```

np.quantile

```
np.quantile(a, q, axis=None, out=None, overwrite_input=False, method='linear', keepdims=False, *,
↪  weights=None)
```

Compute the q-th quantile of the data along the specified axis.

Parameters

a : *array_like of real numbers* Input array or object that can be converted to an array.

q : *array_like of float* Probability or sequence of probabilities of the quantiles to compute. Values must be between 0 and 1 inclusive.

axis : *{int, tuple of int, None}, optional* Axis or axes along which the quantiles are computed. The default is to compute the quantile(s) along a flattened version of the array.

out : *ndarray, optional* Alternative output array in which to place the result. It must have the same shape and buffer length as the expected output, but the type (of the output) will be cast if necessary.

overwrite_input : *bool, optional* If True, then allow the input array a to be modified by intermediate calculations, to save memory. In this case, the contents of the input a after this function completes is undefined.

method : *str, optional* This parameter specifies the method to use for estimating the quantile. There are many different methods, some unique to NumPy. The recommended options, numbered as they appear in, are:

1. 'inverted_cdf'
2. 'averaged_inverted_cdf'
3. 'closest_observation'
4. 'interpolated_inverted_cdf'
5. 'hazen'
6. 'weibull'
7. 'linear' (default)
8. 'median_unbiased'
9. 'normal_unbiased'

The first three methods are discontinuous. For backward compatibility with previous versions of NumPy, the following discontinuous variations of the default 'linear' (7.) option are available:

- 'lower'
- 'higher',
- 'midpoint'
- 'nearest'

See Notes for details.

Changed in version 1.22.0. This argument was previously called ”interpolation” and only offered the ”linear” default and last four options.

keepdims : *bool, optional* If this is set to True, the axes which are reduced are left in the result as dimensions with size one. With this option, the result will broadcast correctly against the original array a.

weights : *array_like, optional* An array of weights associated with the values in a. Each value in a contributes to the quantile according to its associated weight. The weights array can either be 1-D (in which case its length must be the size of a along the given axis) or of the same shape as a. If weights=None, then all data in a are assumed to have a weight equal to one. Only method=”inverted_cdf” supports weights. See the notes for more details.

New in version 2.0.0.

Returns

`quantile` : *scalar or ndarray* If q is a single probability and axis=None, then the result is a scalar. If multiple probability levels are given, first axis of the result corresponds to the quantiles. The other axes are the axes that remain after the reduction of a. If the input contains integers or floats smaller than `float64`, the output data-type is `float64`. Otherwise, the output data-type is the same as that of the input. If out is specified, that array is returned instead.

See also: `mean`, `percentile`, `median`, `nanquantile`

Notes

Given a sample a from an underlying distribution, quantile provides a nonparametric estimate of the inverse cumulative distribution function.

By default, this is done by interpolating between adjacent elements in `y`, a sorted copy of ‘a‘:

`(1-g)*y[j] + g*y[j+1]`

where the index `j` and coefficient `g` are the integral and fractional components of `q * (n-1)`, and `n` is the number of elements in the sample.

This is a special case of Equation 1 of H&F. More generally,

(Notes truncated; see the full reference page.)

Examples

```
>>> a = np.array([[10, 7, 4], [3, 2, 1]])
>>> a
array([[10,  7,  4],
       [ 3,  2,  1]])
>>> np.quantile(a, 0.5)
3.5
>>> np.quantile(a, 0.5, axis=0)
array([6.5, 4.5, 2.5])
>>> np.quantile(a, 0.5, axis=1)
array([7.,  2.])
>>> np.quantile(a, 0.5, axis=1, keepdims=True)
array([[7.],
       [2.]])
>>> m = np.quantile(a, 0.5, axis=0)
>>> out = np.zeros_like(m)
>>> np.quantile(a, 0.5, axis=0, out=out)
array([6.5, 4.5, 2.5])
>>> m
array([6.5, 4.5, 2.5])
>>> b = a.copy()
>>> np.quantile(b, 0.5, axis=1, overwrite_input=True)
array([7.,  2.])
>>> assert not np.all(a == b)
...
```

np.corrcoef

```
np.corrcoef(x, y=None, rowvar=True, *, dtype=None)
```

Return Pearson product-moment correlation coefficients.

Please refer to the documentation for cov for more detail. The relationship between the correlation coefficient matrix, R, and the covariance matrix, C, is

$$R_{ij} = \frac{C_{ij}}{\sqrt{C_{ii}C_{jj}}}$$

The values of R are between -1 and 1, inclusive.

Parameters

x : *array_like* A 1-D or 2-D array containing multiple variables and observations. Each row of x represents a variable, and each column a single observation of all those variables. Also see rowvar below.

y : *array_like, optional* An additional set of variables and observations. y has the same shape as x.

rowvar : *bool, optional* If rowvar is True (default), then each row represents a variable, with observations in the columns. Otherwise, the relationship is transposed: each column represents a variable, while the rows contain observations.

dtype : *data-type, optional* Data-type of the result. By default, the return data-type will have at least numpy.float64 precision.

New in version 1.20.

Returns

R : *ndarray* The correlation coefficient matrix of the variables.

See also: `cov`

Notes

Due to floating point rounding the resulting array may not be Hermitian, the diagonal elements may not be 1, and the elements may not satisfy the inequality abs(a) <= 1. The real and imaginary parts are clipped to the interval [-1, 1] in an attempt to improve on that situation but is not much help in the complex case.

Examples

In this example we generate two random arrays, `xarr` and `yarr`, and compute the row-wise and column-wise Pearson correlation coefficients, `R`. Since `rowvar` is true by default, we first find the row-wise Pearson correlation coefficients between the variables of `xarr`.

```
>>> rng = np.random.default_rng(seed=42)
>>> xarr = rng.random((3, 3))
>>> xarr
array([[0.77395605, 0.43887844, 0.85859792],
       [0.69736803, 0.09417735, 0.97562235],
       [0.7611397 , 0.78606431, 0.12811363]])
>>> R1 = np.corrcoef(xarr)
>>> R1
array([[ 1.        ,  0.99256089, -0.68080986],
       [ 0.99256089,  1.        , -0.76492172],
       [-0.68080986, -0.76492172,  1.        ]])
...
```

np.cov

```
np.cov(m, y=None, rowvar=True, bias=False, ddof=None, fweights=None, aweights=None, *, dtype=None)
```

Estimate a covariance matrix, given data and weights.

Covariance indicates the level to which two variables vary together. If we examine N-dimensional samples, $X = [x_1, x_2, ..., x_N]^T$, then the covariance matrix element C_{ij} is the covariance of x_i and x_j. The element C_{ii} is the variance of x_i.

See the notes for an outline of the algorithm.

Parameters

m : ***array_like*** A 1-D or 2-D array containing multiple variables and observations. Each row of m represents a variable, and each column a single observation of all those variables. Also see rowvar below.

y : ***array_like, optional*** An additional set of variables and observations. y has the same form as that of m.

rowvar : ***bool, optional*** If rowvar is True (default), then each row represents a variable, with observations in the columns. Otherwise, the relationship is transposed: each column represents a variable, while the rows contain observations.

bias : ***bool, optional*** Default normalization (False) is by `(N - 1)`, where `N` is the number of observations given (unbiased estimate). If bias is True, then normalization is by `N`. These values can be overridden by using the keyword `ddof` in numpy versions >= 1.5.

ddof : ***int, optional*** If not `None` the default value implied by bias is overridden. Note that `ddof=1` will return the unbiased estimate, even if both fweights and aweights are specified, and `ddof=0` will return the simple average. See the notes for the details. The default value is `None`.

fweights : ***array_like, int, optional*** 1-D array of integer frequency weights; the number of times each observation vector should be repeated.

aweights : ***array_like, optional*** 1-D array of observation vector weights. These relative weights are typically large for observations considered ”important” and smaller for observations considered less ”important”. If `ddof=0` the array of weights can be used to assign probabilities to observation vectors.

dtype : ***data-type, optional*** Data-type of the result. By default, the return data-type will have at least numpy.float64 precision.

New in version 1.20.

Returns

out : ***ndarray*** The covariance matrix of the variables.

See also: `corrcoef`

Notes

Assume that the observations are in the columns of the observation array m and let `f = fweights` and `a = aweights` for brevity. The steps to compute the weighted covariance are as follows:

```
>>> m = np.arange(10, dtype=np.float64)
>>> f = np.arange(10) * 2
>>> a = np.arange(10) ** 2.
>>> ddof = 1
>>> w = f * a
>>> v1 = np.sum(w)
```

```
>>> v2 = np.sum(w * a)
>>> m -= np.sum(m * w, axis=None, keepdims=True) / v1
>>> cov = np.dot(m * w, m.T) * v1 / (v1**2 - ddof * v2)
```

(Notes truncated; see the full reference page.)

Examples

Consider two variables, x_0 and x_1, which correlate perfectly, but in opposite directions:

```
>>> x = np.array([[0, 2], [1, 1], [2, 0]]).T
>>> x
array([[0, 1, 2],
       [2, 1, 0]])
```

Note how x_0 increases while x_1 decreases. The covariance matrix shows this clearly:

```
>>> np.cov(x)
array([[ 1., -1.],
       [-1.,  1.]])
```

Note that element $C_{0,1}$, which shows the correlation between x_0 and x_1, is negative. ...

np.histogram

```
np.histogram(a, bins=10, range=None, density=None, weights=None)
```

Compute the histogram of a dataset.

Parameters

a : *array_like* Input data. The histogram is computed over the flattened array.

bins : *int or sequence of scalars or str, optional* If bins is an int, it defines the number of equal-width bins in the given range (10, by default). If bins is a sequence, it defines a monotonically increasing array of bin edges, including the rightmost edge, allowing for non-uniform bin widths.

If bins is a string, it defines the method used to calculate the optimal bin width, as defined by histogram_bin_edges.

range : *(float, float), optional* The lower and upper range of the bins. If not provided, range is simply `(a.min(), a.max())`. Values outside the range are ignored. The first element of the range must be less than or equal to the second. range affects the automatic bin computation as well. While bin width is computed to be optimal based on the actual data within range, the bin count will fill the entire range including portions containing no data.

weights : *array_like, optional* An array of weights, of the same shape as a. Each value in a only contributes its associated weight towards the bin count (instead of 1). If density is True, the weights are normalized, so that the integral of the density over the range remains 1. Please note that the `dtype` of weights will also become the `dtype` of the returned accumulator (hist), so it must be large enough to hold accumulated values as well.

density : *bool, optional* If `False`, the result will contain the number of samples in each bin. If `True`, the result is the value of the probability *density* function at the bin, normalized such that the *integral* over the range is 1. Note that the sum of the histogram values will not be equal to 1 unless bins of unity width are chosen; it is not a probability *mass* function.

Returns

hist : *array* The values of the histogram. See density and weights for a description of the possible semantics. If weights are given, `hist.dtype` will be taken from weights.

bin_edges : *array of dtype float* Return the bin edges `(length(hist)+1)`.

See also: `histogramdd`, `bincount`, `searchsorted`, `digitize`, `histogram_bin_edges`

Notes

All but the last (righthand-most) bin is half-open. In other words, if bins is:

`[1, 2, 3, 4]`

then the first bin is `[1, 2)` (including 1, but excluding 2) and the second `[2, 3)`. The last bin, however, is `[3, 4]`, which *includes* 4.

Examples

```
>>> np.histogram([1, 2, 1], bins=[0, 1, 2, 3])
(array([0, 2, 1]), array([0, 1, 2, 3]))
>>> np.histogram(np.arange(4), bins=np.arange(5), density=True)
(array([0.25, 0.25, 0.25, 0.25]), array([0, 1, 2, 3, 4]))
>>> np.histogram([[1, 2, 1], [1, 0, 1]], bins=[0,1,2,3])
(array([1, 4, 1]), array([0, 1, 2, 3]))
```

```
>>> a = np.arange(5)
>>> hist, bin_edges = np.histogram(a, density=True)
>>> hist
array([0.5, 0. , 0.5, 0. , 0. , 0.5, 0. , 0.5, 0. , 0.5])
>>> hist.sum()
2.4999999999999996
>>> np.sum(hist * np.diff(bin_edges))
1.0
```

Automated Bin Selection Methods example, using 2 peak random data with 2000 points. ...

np.clip

```
np.clip(a, a_min=<no value>, a_max=<no value>, out=None, *, min=<no value>, max=<no value>, **kwargs)
```

Clip (limit) the values in an array.

Given an interval, values outside the interval are clipped to the interval edges. For example, if an interval of `[0, 1]` is specified, values smaller than 0 become 0, and values larger than 1 become 1.

Equivalent to but faster than `np.minimum(a_max, np.maximum(a, a_min))`.

No check is performed to ensure `a_min < a_max`.

Parameters

a : *array_like* Array containing elements to clip.

a_min, a_max : *array_like or None* Minimum and maximum value. If `None`, clipping is not performed on the corresponding edge. If both `a_min` and `a_max` are `None`, the elements of the returned array stay the same. Both are broadcasted against `a`.

out : *ndarray, optional* The results will be placed in this array. It may be the input array for in-place clipping. out must be of the right shape to hold the output. Its type is preserved.

min, max : *array_like or None* Array API compatible alternatives for `a_min` and `a_max` arguments. Either `a_min` and `a_max` or `min` and `max` can be passed at the same time. Default: `None`.

New in version 2.1.0.

****kwargs** For other keyword-only arguments, see the ufunc docs.

Returns

clipped_array : *ndarray* An array with the elements of a, but where values < a_min are replaced with a_min, and those > a_max with a_max.

See also: `ufuncs-output-type`

Notes

When a_min is greater than a_max, clip returns an array in which all values are equal to a_max, as shown in the second example.

Examples

```
>>> a = np.arange(10)
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.clip(a, 1, 8)
array([1, 1, 2, 3, 4, 5, 6, 7, 8, 8])
>>> np.clip(a, 8, 1)
array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
>>> np.clip(a, 3, 6, out=a)
array([3, 3, 3, 3, 4, 5, 6, 6, 6, 6])
>>> a
array([3, 3, 3, 3, 4, 5, 6, 6, 6, 6])
>>> a = np.arange(10)
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.clip(a, [3, 4, 1, 1, 1, 4, 4, 4, 4, 4], 8)
array([3, 4, 2, 3, 4, 5, 6, 7, 8, 8])
```

Logic, sorting and searching

np.where

```
np.where(condition, x=None, y=None, /)
```

Return elements chosen from x or y depending on condition.

> Note
> When only condition is provided, this function is a shorthand for `np.asarray(condition).nonzero()`. Using nonzero directly should be preferred, as it behaves correctly for subclasses. The rest of this documentation covers only the case where all three arguments are provided.

Parameters

condition : *array_like, bool* Where True, yield x, otherwise yield y.

x, y : *array_like* Values from which to choose. x, y and condition need to be broadcastable to some shape.

Returns

out : *ndarray* An array with elements from x where condition is True, and elements from y elsewhere.

See also: `choose`, `nonzero`

Notes

If all the arrays are 1-D, where is equivalent to:

```
[xv if c else yv
 for c, xv, yv in zip(condition, x, y)]
```

Examples

```
>>> a = np.arange(10)
>>> a
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
>>> np.where(a < 5, a, 10*a)
array([ 0,  1,  2,  3,  4, 50, 60, 70, 80, 90])
```

This can be used on multidimensional arrays too:

```
>>> np.where([[True, False], [True, True]],
...          [[1, 2], [3, 4]],
...          [[9, 8], [7, 6]])
array([[1, 8],
       [3, 4]])
```

The shapes of x, y, and the condition are broadcast together:

```
>>> x, y = np.ogrid[:3, :4]
>>> np.where(x < y, x, 10 + y)  # both x and 10+y are broadcast
array([[10,  0,  0,  0],
       [10, 11,  1,  1],
       [10, 11, 12,  2]])
...
```

np.unique

```
np.unique(ar, return_index=False, return_inverse=False, return_counts=False, axis=None, *,
↪  equal_nan=True, sorted=True)
```

Find the unique elements of an array.

Returns the sorted unique elements of an array. There are three optional outputs in addition to the unique elements:

- the indices of the input array that give the unique values
- the indices of the unique array that reconstruct the input array
- the number of times each unique value comes up in the input array

Parameters

`ar` : *array_like* Input array. Unless axis is specified, this will be flattened if it is not already 1-D.

`return_index` : *bool, optional* If True, also return the indices of ar (along the specified axis, if provided, or in the flattened array) that result in the unique array.

`return_inverse` : *bool, optional* If True, also return the indices of the unique array (for the specified axis, if provided) that can be used to reconstruct ar.

`return_counts` : *bool, optional* If True, also return the number of times each unique item appears in ar.

`axis` : *int or None, optional* The axis to operate on. If None, ar will be flattened. If an integer, the subarrays indexed by the given axis will be flattened and treated as the elements of a 1-D array with the dimension of the given axis, see the notes for more details. Object arrays or structured arrays that contain objects are not supported if the axis kwarg is used. The default is None.

equal_nan : *bool, optional* If True, collapses multiple NaN values in the return array into one.

New in version 1.24.

sorted : *bool, optional* If True, the unique elements are sorted. Elements may be sorted in practice even if `sorted=False`, but this could change without notice.

New in version 2.3.

Returns

unique : *ndarray* The sorted unique values.

unique_indices : *ndarray, optional* The indices of the first occurrences of the unique values in the original array. Only provided if return_index is True.

unique_inverse : *ndarray, optional* The indices to reconstruct the original array from the unique array. Only provided if return_inverse is True.

unique_counts : *ndarray, optional* The number of times each of the unique values comes up in the original array. Only provided if return_counts is True.

See also: `repeat`, `sort`

Notes

When an axis is specified the subarrays indexed by the axis are sorted. This is done by making the specified axis the first dimension of the array (move the axis to the first dimension to keep the order of the other axes) and then flattening the subarrays in C order. The flattened subarrays are then viewed as a structured type with each element given a label, with the effect that we end up with a 1-D array of structured types that can be treated in the same way as any other 1-D array. The result is that the flattened subarrays are sorted in lexicographic order starting with the first element.

(Notes truncated; see the full reference page.)

Examples

```
>>> np.unique([1, 1, 2, 2, 3, 3])
array([1, 2, 3])
>>> a = np.array([[1, 1], [2, 3]])
>>> np.unique(a)
array([1, 2, 3])
```

Return the unique rows of a 2D array

```
>>> a = np.array([[1, 0, 0], [1, 0, 0], [2, 3, 4]])
>>> np.unique(a, axis=0)
array([[1, 0, 0], [2, 3, 4]])
```

Return the indices of the original array that give the unique values:

```
>>> a = np.array(['a', 'b', 'b', 'c', 'a'])
>>> u, indices = np.unique(a, return_index=True)
>>> u
array(['a', 'b', 'c'], dtype='<U1')
>>> indices
array([0, 1, 3])
>>> a[indices]
array(['a', 'b', 'c'], dtype='<U1')
...
```

np.sort

```
np.sort(a, axis=-1, kind=None, order=None, *, stable=None, descending=<no value>)
```

Return a sorted copy of an array.

Parameters

a : *array_like* Array to be sorted.

axis : *int or None, optional* Axis along which to sort. If None, the array is flattened before sorting. The default is -1, which sorts along the last axis.

kind : *{'quicksort', 'mergesort', 'heapsort', 'stable'}, optional* Sorting algorithm. The default is 'quicksort'. Note that both 'stable' and 'mergesort' use timsort or radix sort under the covers and, in general, the actual implementation will vary with data type. The 'mergesort' option is retained for backwards compatibility.

order : *str or list of str, optional* When a is an array with fields defined, this argument specifies which fields to compare first, second, etc. A single field can be specified as a string, and not all fields need be specified, but unspecified fields will still be used, in the order in which they come up in the dtype, to break ties.

stable : *bool, optional* Sort stability. If `True`, the returned array will maintain the relative order of `a` values which compare as equal. If `False` or `None`, this is not guaranteed. Internally, this option selects `kind='stable'`. Default: `None`.

New in version 2.0.0.

descending : *bool, optional* Sort order. If `True`, the returned array will be sorted in descending order. If `False` or `None`, the returned array will be sorted in ascending order. Values that are NaN are sorted to the end for both orders. Default: `None`.

New in version 2.5.0.

Returns

sorted_array : *ndarray* Array of the same type and shape as a.

See also: `ndarray.sort`, `argsort`, `lexsort`, `searchsorted`, `partition`

Notes

The various sorting algorithms are characterized by their average speed, worst case performance, work space size, and whether they are stable. A stable sort keeps items with the same key in the same relative order. The four algorithms implemented in NumPy have the following properties:

(Notes truncated; see the full reference page.)

Examples

```
>>> a = np.array([[1,4],[3,1]])
>>> np.sort(a)                # sort along the last axis
array([[1, 4],
       [1, 3]])
>>> np.sort(a, axis=None)     # sort the flattened array
array([1, 1, 3, 4])
>>> np.sort(a, axis=0)        # sort along the first axis
array([[1, 1],
       [3, 4]])
```

Use the order keyword to specify a field to use when sorting a structured array:

```
>>> dtype = [('name', 'S10'), ('height', float), ('age', int)]
>>> values = [('Arthur', 1.8, 41), ('Lancelot', 1.9, 38),
...           ('Galahad', 1.7, 38)]
>>> a = np.array(values, dtype=dtype)       # create a structured array
>>> np.sort(a, order='height')
array([('Galahad', 1.7, 38), ('Arthur', 1.8, 41),
       ('Lancelot', 1.8999999999999999, 38)],
      dtype=[('name', '|S10'), ('height', '<f8'), ('age', '<i4')])
...
```

np.argsort

```
np.argsort(a, axis=-1, kind=None, order=None, *, stable=None, descending=<no value>)
```

Returns the indices that would sort an array.

Perform an indirect sort along the given axis using the algorithm specified by the kind keyword. It returns an array of indices of the same shape as a that index data along the given axis in sorted order.

Parameters

a : *array_like* Array to sort.

axis : *int or None, optional* Axis along which to sort. The default is -1 (the last axis). If None, the flattened array is used.

kind : *{'quicksort', 'mergesort', 'heapsort', 'stable'}, optional* Sorting algorithm. The default is 'quicksort'. Note that both 'stable' and 'mergesort' use timsort under the covers and, in general, the actual implementation will vary with data type. The 'mergesort' option is retained for backwards compatibility.

order : *str or list of str, optional* When a is an array with fields defined, this argument specifies which fields to compare first, second, etc. A single field can be specified as a string, and not all fields need be specified, but unspecified fields will still be used, in the order in which they come up in the dtype, to break ties.

stable : *bool, optional* Sort stability. If `True`, the returned array will maintain the relative order of `a` values which compare as equal. If `False` or `None`, this is not guaranteed. Internally, this option selects `kind='stable'`. Default: `None`.

New in version 2.0.0.

descending : *bool, optional* Sort order. If `True`, the returned array will be sorted in descending order. If `False` or `None`, the returned array will be sorted in ascending order. Values that are NaN are sorted to the end for both orders. Default: `None`.

New in version 2.5.0.

Returns

index_array : *ndarray, int* Array of indices that sort a along the specified axis. If a is one-dimensional, `a[index_array]` yields a sorted a. More generally, `np.take_along_axis(a, index_array, axis=axis)` always yields the sorted a, irrespective of dimensionality.

See also: `sort`, `lexsort`, `ndarray.sort`, `argpartition`, `take_along_axis`

Notes

See sort for notes on the different sorting algorithms.

As of NumPy 1.4.0 argsort works with real/complex arrays containing nan values. The enhanced sort order is documented in sort.

Examples

One dimensional array:

```
>>> x = np.array([3, 1, 2])
>>> np.argsort(x)
array([1, 2, 0])
```

Two-dimensional array:

```
>>> x = np.array([[0, 3], [2, 2]])
>>> x
array([[0, 3],
       [2, 2]])
```

```
>>> ind = np.argsort(x, axis=0)  # sorts along first axis (down)
>>> ind
array([[0, 1],
       [1, 0]])
>>> np.take_along_axis(x, ind, axis=0)  # same as np.sort(x, axis=0)
array([[0, 2],
       [2, 3]])
...
```

np.isnan

```
np.isnan(x, /, out=None, *, where=True, casting='same_kind', order='K', dtype=None, subok=True,
 ↪  signature=None)
```

Test element-wise for NaN and return result as a boolean array.

Parameters

x : *array_like* Input array.

out : *ndarray, None, or tuple of ndarray and None, optional* A location into which the result is stored. If provided, it must have a shape that the inputs broadcast to. If not provided or None, a freshly-allocated array is returned. A tuple (possible only as a keyword argument) must have length equal to the number of outputs.

where : *array_like, optional* This condition is broadcast over the input. At locations where the condition is True, the out array will be set to the ufunc result. Elsewhere, the out array will retain its original value. Note that if an uninitialized out array is created via the default `out=None`, locations within it where the condition is False will remain uninitialized.

****kwargs** For other keyword-only arguments, see the ufunc docs.

Returns

y : *ndarray or bool* True where x is NaN, false otherwise. This is a scalar if x is a scalar.

See also: `isinf`, `isneginf`, `isposinf`, `isfinite`, `isnat`

Notes

NumPy uses the IEEE Standard for Binary Floating-Point for Arithmetic (IEEE 754). This means that Not a Number is not equivalent to infinity.

Examples

```
>>> np.isnan(np.nan)
True
>>> np.isnan(np.inf)
False
>>> np.isnan([np.log(-1.),1.,np.log(0)])
array([ True, False, False])
```

np.allclose

```
np.allclose(a, b, rtol=1e-05, atol=1e-08, equal_nan=False)
```

Returns True if two arrays are element-wise equal within a tolerance.

The tolerance values are positive, typically very small numbers. The relative difference (rtol * abs(b)) and the absolute difference atol are added together to compare against the absolute difference between a and b.

> Warning
> The default atol is not appropriate for comparing numbers
> with magnitudes much smaller than one (see Notes).

NaNs are treated as equal if they are in the same place and if `equal_nan=True`. Infs are treated as equal if they are in the same place and of the same sign in both arrays.

Parameters

`a, b` : *array_like* Input arrays to compare.

`rtol` : *array_like* The relative tolerance parameter (see Notes).

`atol` : *array_like* The absolute tolerance parameter (see Notes).

`equal_nan` : *bool* Whether to compare NaN's as equal. If True, NaN's in a will be considered equal to NaN's in b in the output array.

Returns

`allclose` : *bool* Returns True if the two arrays are equal within the given tolerance; False otherwise.

See also: `isclose`, `all`, `any`, `equal`

Notes

If the following equation is element-wise True, then allclose returns True.:

```
absolute(a - b) <= (atol + rtol * absolute(b))
```

The above equation is not symmetric in a and b, so that `allclose(a, b)` might be different from `allclose(b, a)` in some rare cases.

(Notes truncated; see the full reference page.)

Examples

```
>>> np.allclose([1e10,1e-7], [1.00001e10,1e-8])
False
```

```
>>> np.allclose([1e10,1e-8], [1.00001e10,1e-9])
True
```

```
>>> np.allclose([1e10,1e-8], [1.0001e10,1e-9])
False
```

```
>>> np.allclose([1.0, np.nan], [1.0, np.nan])
False
```

```
>>> np.allclose([1.0, np.nan], [1.0, np.nan], equal_nan=True)
True
```

np.nonzero

```
np.nonzero(a)
```

Return the indices of the elements that are non-zero.

Returns a tuple of arrays, one for each dimension of a, containing the indices of the non-zero elements in that dimension. The values in a are always tested and returned in row-major, C-style order.

To group the indices by element, rather than dimension, use argwhere, which returns a row for each non-zero element.

Parameters

a : *array_like* Input array.

Returns

tuple_of_arrays : *tuple* Indices of elements that are non-zero.

See also: `flatnonzero`, `ndarray.nonzero`, `count_nonzero`

Notes

While the nonzero values can be obtained with `a[nonzero(a)]`, it is recommended to use `x[x.astype(np.bool)]` or `x[x != 0]` instead, which will correctly handle 0-d arrays.

Examples

```
>>> x = np.array([[3, 0, 0], [0, 4, 0], [5, 6, 0]])
>>> x
array([[3, 0, 0],
       [0, 4, 0],
       [5, 6, 0]])
>>> np.nonzero(x)
(array([0, 1, 2, 2]), array([0, 1, 0, 1]))
```

```
>>> x[np.nonzero(x)]
array([3, 4, 5, 6])
>>> np.transpose(np.nonzero(x))
array([[0, 0],
       [1, 1],
       [2, 0],
       [2, 1]])
```

A common use for `nonzero` is to find the indices of an array, where a condition is True. Given an array a, the condition a > 3 is a boolean array and since False is interpreted as 0, np.nonzero(a > 3) yields the indices of the a where the condition is true. ...

np.searchsorted

```
np.searchsorted(a, v, side='left', sorter=None)
```

Find indices where elements should be inserted to maintain order.

Find the indices into a sorted array a such that, if the corresponding elements in v were inserted before the indices, the order of a would be preserved.

Assuming that a is sorted:

side	returned index i satisfies
left	`a[i-1] < v <= a[i]`
right	`a[i-1] <= v < a[i]`

Parameters

a : *1-D array_like* Input array. If sorter is None, then it must be sorted in ascending order, otherwise sorter must be an array of indices that sort it.

v : *array_like* Values to insert into a.

side : *{'left', 'right'}, optional* If 'left', the index of the first suitable location found is given. If 'right', return the last such index. If there is no suitable index, return either 0 or N (where N is the length of a).

sorter : *1-D array_like, optional* Optional array of integer indices that sort array a into ascending order. They are typically the result of argsort.

Returns

indices : *int or array of ints* Array of insertion points with the same shape as v, or an integer if v is a scalar.

See also: `sort`, `histogram`

Notes

Binary search is used to find the required insertion points.

As of NumPy 1.4.0 searchsorted works with real/complex arrays containing nan values. The enhanced sort order is documented in sort.

This function uses the same algorithm as the builtin python bisect.bisect_left (`side='left'`) and bisect.bisect_right (`side='right'`) functions, which is also vectorized in the v argument.

Examples

```
>>> np.searchsorted([11,12,13,14,15], 13)
2
>>> np.searchsorted([11,12,13,14,15], 13, side='right')
3
>>> np.searchsorted([11,12,13,14,15], [-10, 20, 12, 13])
array([0, 5, 1, 2])
```

When sorter is used, the returned indices refer to the sorted array of a and not a itself:

```
>>> a = np.array([40, 10, 20, 30])
>>> sorter = np.argsort(a)
>>> sorter
array([1, 2, 3, 0])  # Indices that would sort the array 'a'
>>> result = np.searchsorted(a, 25, sorter=sorter)
```

```
>>> result
2
>>> a[sorter[result]]
30  # The element at index 2 of the sorted array is 30.
...
```

Linear algebra

np.matmul

```
np.matmul(x1, x2, /, out=None, *, axes=<no value>, axis=<no value>, keepdims=False, casting='same_kind',
 ↪  order='K', dtype=None, subok=True, signature=None)
```

Matrix product of two arrays.

Parameters

x1, x2 : *array_like* Input arrays, scalars not allowed.

out : *ndarray, optional* A location into which the result is stored. If provided, it must have a shape that matches the signature (n,k),(k,m)->(n,m). If not provided or None, a freshly-allocated array is returned.

****kwargs** For other keyword-only arguments, see the ufunc docs.

Returns

y : *ndarray* The matrix product of the inputs. This is a scalar only when both x1, x2 are 1-d vectors.

Raises

ValueError If the last dimension of x1 is not the same size as the second-to-last dimension of x2.

If a scalar value is passed in.

See also: `vecdot`, `matvec`, `vecmat`, `tensordot`, `einsum`, `dot`

Notes

The behavior depends on the arguments in the following way.

- If both arguments are 2-D they are multiplied like conventional matrices.
- If either argument is N-D, N > 2, it is treated as a stack of matrices residing in the last two indexes and broadcast accordingly.
- If the first argument is 1-D, it is promoted to a matrix by prepending a 1 to its dimensions. After matrix multiplication the prepended 1 is removed. (For stacks of vectors, use `vecmat`.)
- If the second argument is 1-D, it is promoted to a matrix by appending a 1 to its dimensions. After matrix multiplication the appended 1 is removed. (For stacks of vectors, use `matvec`.)

(Notes truncated; see the full reference page.)

Examples

For 2-D arrays it is the matrix product:

```
>>> a = np.array([[1, 0],
...               [0, 1]])
>>> b = np.array([[4, 1],
...               [2, 2]])
>>> np.matmul(a, b)
```

```
array([[4, 1],
       [2, 2]])
```

For 2-D mixed with 1-D, the result is the usual.

```
>>> a = np.array([[1, 0],
...               [0, 1]])
>>> b = np.array([1, 2])
>>> np.matmul(a, b)
array([1, 2])
>>> np.matmul(b, a)
array([1, 2])
...
```

np.linalg.inv

```
np.linalg.inv(a)
```

Compute the inverse of a matrix.

Given a square matrix a, return the matrix ainv satisfying `a @ ainv = ainv @ a = eye(a.shape[0])`.

Parameters

a : *(..., M, M) array_like* Matrix to be inverted.

Returns

ainv : *(..., M, M) ndarray or matrix* Inverse of the matrix a.

Raises

LinAlgError If a is not square or inversion fails.

See also: `scipy.linalg.inv`, `numpy.linalg.cond`, `numpy.linalg.svd`

Notes

Broadcasting rules apply, see the numpy.linalg documentation for details.

If a is detected to be singular, a LinAlgError is raised. If a is ill-conditioned, a LinAlgError may or may not be raised, and results may be inaccurate due to floating-point errors.

Examples

```
>>> from numpy.linalg import inv
>>> a = np.array([[1., 2.], [3., 4.]])
>>> ainv = inv(a)
>>> np.allclose(a @ ainv, np.eye(2))
True
>>> np.allclose(ainv @ a, np.eye(2))
True
```

If a is a matrix object, then the return value is a matrix as well:

```
>>> ainv = inv(np.matrix(a))
>>> ainv
matrix([[-2. ,  1. ],
        [ 1.5, -0.5]])
```

Inverses of several matrices can be computed at once: ...

np.linalg.solve

```
np.linalg.solve(a, b)
```

Solve a linear matrix equation, or system of linear scalar equations.

Computes the ”exact” solution, x, of the well-determined, i.e., full rank, linear matrix equation ax = b.

Parameters

a : ***(..., M, M) array_like*** Coefficient matrix.

b : ***{(M,), (..., M, K)}, array_like*** Ordinate or ”dependent variable” values.

Returns

x : ***{(..., M,), (..., M, K)} ndarray*** Solution to the system a x = b. Returned shape is (..., M) if b is shape (M,) and (..., M, K) if b is (..., M, K), where the ”...” part is broadcasted between a and b.

Raises

LinAlgError If a is singular or not square.

See also: `scipy.linalg.solve`

Notes

Broadcasting rules apply, see the numpy.linalg documentation for details.

The solutions are computed using LAPACK routine `_gesv`.

a must be square and of full-rank, i.e., all rows (or, equivalently, columns) must be linearly independent; if either is not true, use lstsq for the least-squares best ”solution” of the system/equation.

Changed in version 2.0.

(Notes truncated; see the full reference page.)

Examples

Solve the system of equations: `x0 + 2 * x1 = 1` and `3 * x0 + 5 * x1 = 2`:

```
>>> a = np.array([[1, 2], [3, 5]])
>>> b = np.array([1, 2])
>>> x = np.linalg.solve(a, b)
>>> x
array([-1.,  1.])
```

Check that the solution is correct:

```
>>> np.allclose(np.dot(a, x), b)
True
```

np.linalg.eig

```
np.linalg.eig(a)
```

Compute the eigenvalues and right eigenvectors of a square array.

Parameters

a : *(..., M, M) array* Matrices for which the eigenvalues and right eigenvectors will be computed

Returns

A namedtuple with the following attributes:

eigenvalues : *(..., M) array* The eigenvalues, each repeated according to its multiplicity. The eigenvalues are not necessarily ordered. The resulting array will be of complex type, unless the imaginary part is zero in which case it will be cast to a real type. When a is real the resulting eigenvalues will be real (0 imaginary part) or occur in conjugate pairs

eigenvectors : *(..., M, M) array* The normalized (unit "length") eigenvectors, such that the column `eigenvectors[:,i]` is the eigenvector corresponding to the eigenvalue `eigenvalues[i]`.

Raises

LinAlgError If the eigenvalue computation does not converge.

See also: `eigvals`, `eigh`, `eigvalsh`, `scipy.linalg.eig`, `scipy.linalg.schur`

Notes

Broadcasting rules apply, see the numpy.linalg documentation for details.

This is implemented using the `_geev` LAPACK routines which compute the eigenvalues and eigenvectors of general square arrays.

The number w is an eigenvalue of a if there exists a vector v such that `a @ v = w * v`. Thus, the arrays a, eigenvalues, and eigenvectors satisfy the equations `a @ eigenvectors[:,i] = eigenvalues[i] * eigenvectors[:,i]` for $i \in \{0, ..., M-1\}$.

(Notes truncated; see the full reference page.)

Examples

```
>>> from numpy import linalg as LA
```

(Almost) trivial example with real eigenvalues and eigenvectors.

```
>>> eigenvalues, eigenvectors = LA.eig(np.diag((1, 2, 3)))
>>> eigenvalues
array([1. + 0j, 2. + 0j, 3. + 0j])
>>> eigenvectors.real
array([[1., 0., 0.],
       [0., 1., 0.],
       [0., 0., 1.]])
```

Real matrix possessing complex eigenvalues and eigenvectors; note that the eigenvalues are complex conjugates of each other.

```
>>> eigenvalues, eigenvectors = LA.eig(np.array([[1, -1], [1, 1]]))
>>> eigenvalues
array([1.+1.j, 1.-1.j])
>>> eigenvectors
array([[0.70710678+0.j        , 0.70710678-0.j        ],
       [0.        -0.70710678j, 0.        +0.70710678j]])
...
```

np.linalg.svd

```
np.linalg.svd(a, full_matrices=True, compute_uv=True, hermitian=False)
```

Singular Value Decomposition.

When a is a 2D array, and `full_matrices=False`, then it is factorized as `u @ np.diag(s) @ vh = (u * s) @ vh`, where u and the Hermitian transpose of vh are 2D arrays with orthonormal columns and s is a 1D array of a's singular values. When a is higher-dimensional, SVD is applied in stacked mode as explained below.

Parameters

`a` : ***(..., M, N) array_like*** A real or complex array with `a.ndim >= 2`.

`full_matrices` : ***bool, optional*** If True (default), u and vh have the shapes `(..., M, M)` and `(..., N, N)`, respectively. Otherwise, the shapes are `(..., M, K)` and `(..., K, N)`, respectively, where `K = min(M, N)`.

`compute_uv` : ***bool, optional*** Whether or not to compute u and vh in addition to s. True by default.

`hermitian` : ***bool, optional*** If True, a is assumed to be Hermitian (symmetric if real-valued), enabling a more efficient method for finding singular values. Defaults to False.

Returns

`U` : ***{ (..., M, M), (..., M, K) } array*** Unitary array(s). The first `a.ndim - 2` dimensions have the same size as those of the input a. The size of the last two dimensions depends on the value of full_matrices. Only returned when compute_uv is True.

`S` : ***(..., K) array*** Vector(s) with the singular values, within each vector sorted in descending order. The first `a.ndim - 2` dimensions have the same size as those of the input a.

`Vh` : ***{ (..., N, N), (..., K, N) } array*** Unitary array(s). The first `a.ndim - 2` dimensions have the same size as those of the input a. The size of the last two dimensions depends on the value of full_matrices. Only returned when compute_uv is True.

Raises

LinAlgError If SVD computation does not converge.

See also: `scipy.linalg.svd`, `scipy.linalg.svdvals`

Notes

When compute_uv is True, the result is a namedtuple with the following attribute names: U, S, and Vh.

The decomposition is performed using LAPACK routine `_gesdd`.

SVD is usually described for the factorization of a 2D matrix A. The higher-dimensional case will be discussed below. In the 2D case, SVD is written as $A = USV^H$, where $A = a$, $U = u$, $S = \mathtt{np.diag}(s)$ and $V^H = vh$. The 1D array s contains the singular values of a and u and vh are unitary. The rows of vh are the eigenvectors of $A^H A$ and the columns of u are the eigenvectors of AA^H. In both cases the corresponding (possibly non-zero) eigenvalues are given by `s**2`.

(Notes truncated; see the full reference page.)

Examples

```
>>> rng = np.random.default_rng()
>>> a = rng.normal(size=(9, 6)) + 1j*rng.normal(size=(9, 6))
>>> b = rng.normal(size=(2, 7, 8, 3)) + 1j*rng.normal(size=(2, 7, 8, 3))
```

Reconstruction based on full SVD, 2D case:

```
>>> U, S, Vh = np.linalg.svd(a, full_matrices=True)
>>> U.shape, S.shape, Vh.shape
((9, 9), (6,), (6, 6))
>>> np.allclose(a, np.dot(U[:, :6] * S, Vh))
```

```
True
>>> smat = np.zeros((9, 6), dtype=np.complex128)
>>> smat[:6, :6] = np.diag(S)
>>> np.allclose(a, np.dot(U, np.dot(smat, Vh)))
True
```

Reconstruction based on reduced SVD, 2D case: ...

np.linalg.norm

```
np.linalg.norm(x, ord=None, axis=None, keepdims=False)
```

Matrix or vector norm.

This function is able to return one of eight different matrix norms, or one of an infinite number of vector norms (described below), depending on the value of the `ord` parameter.

Parameters

`x` : ***array_like*** Input array. If axis is None, x must be 1-D or 2-D, unless ord is None. If both axis and ord are None, the 2-norm of `x.ravel` will be returned.

`ord` : ***{int, float, inf, -inf, 'fro', 'nuc'}, optional*** Order of the norm (see table under `Notes` for what values are supported for matrices and vectors respectively). inf means numpy's inf object. The default is None.

`axis` : ***{None, int, 2-tuple of ints}, optional.*** If axis is an integer, it specifies the axis of x along which to compute the vector norms. If axis is a 2-tuple, it specifies the axes that hold 2-D matrices, and the matrix norms of these matrices are computed. If axis is None then either a vector norm (when x is 1-D) or a matrix norm (when x is 2-D) is returned. The default is None.

`keepdims` : ***bool, optional*** If this is set to True, the axes which are normed over are left in the result as dimensions with size one. With this option the result will broadcast correctly against the original x.

Returns

`n` : ***float or ndarray*** Norm of the matrix or vector(s).

See also: `scipy.linalg.norm`

Notes

For values of `ord < 1`, the result is, strictly speaking, not a mathematical 'norm', but it may still be useful for various numerical purposes.

The following norms can be calculated:

(Notes truncated; see the full reference page.)

Examples

```
>>> from numpy import linalg as LA
>>> a = np.arange(9) - 4
>>> a
array([-4, -3, -2, ...,  2,  3,  4])
>>> b = a.reshape((3, 3))
>>> b
array([[-4, -3, -2],
       [-1,  0,  1],
       [ 2,  3,  4]])
```

```
>>> LA.norm(a)
7.745966692414834
>>> LA.norm(b)
7.745966692414834
>>> LA.norm(b, 'fro')
7.745966692414834
>>> LA.norm(a, np.inf)
4.0
>>> LA.norm(b, np.inf)
9.0
>>> LA.norm(a, -np.inf)
0.0
>>> LA.norm(b, -np.inf)
2.0
...
```

np.linalg.lstsq

```
np.linalg.lstsq(a, b, rcond=None)
```

Return the least-squares solution to a linear matrix equation.

Computes the vector x that approximately solves the equation `a @ x = b`. The equation may be under-, well-, or over-determined (i.e., the number of linearly independent rows of a can be less than, equal to, or greater than its number of linearly independent columns). If a is square and of full rank, then x (but for round-off error) is the "exact" solution of the equation. Else, x minimizes the Euclidean 2-norm $||b - ax||$. If there are multiple minimizing solutions, the one with the smallest 2-norm $||x||$ is returned.

Parameters

a : ***(M, N) array_like*** "Coefficient" matrix.

b : ***{(M,), (M, K)} array_like*** Ordinate or "dependent variable" values. If b is two-dimensional, the least-squares solution is calculated for each of the K columns of b.

rcond : ***float, optional*** Cut-off ratio for small singular values of a. For the purposes of rank determination, singular values are treated as zero if they are smaller than rcond times the largest singular value of a. The default uses the machine precision times `max(M, N)`. Passing `-1` will use machine precision.

Changed in version 2.0. Previously, the default was `-1`, but a warning was given that this would change.

Returns

x : ***{(N,), (N, K)} ndarray*** Least-squares solution. If b is two-dimensional, the solutions are in the K columns of x.

residuals : ***{(1,), (K,), (0,)} ndarray*** Sums of squared residuals: Squared Euclidean 2-norm for each column in `b - a @ x`. If the rank of a is < N or M <= N, this is an empty array. If b is 1-dimensional, this is a (1,) shape array. Otherwise the shape is (K,).

rank : ***int*** Rank of matrix a.

s : ***(min(M, N),) ndarray*** Singular values of a.

Raises

LinAlgError If computation does not converge.

See also: `scipy.linalg.lstsq`

Notes

If b is a matrix, then all array results are returned as matrices.

Examples

Fit a line, `y = mx + c`, through some noisy data-points:

```
>>> x = np.array([0, 1, 2, 3])
>>> y = np.array([-1, 0.2, 0.9, 2.1])
```

By examining the coefficients, we see that the line should have a gradient of roughly 1 and cut the y-axis at, more or less, -1.

We can rewrite the line equation as `y = Ap`, where `A = [[x 1]]` and `p = [[m], [c]]`. Now use lstsq to solve for 'p':

```
>>> A = np.vstack([x, np.ones(len(x))]).T
>>> A
array([[ 0.,  1.],
       [ 1.,  1.],
       [ 2.,  1.],
       [ 3.,  1.]])
...
```

Random sampling (Generator)

np.random.Generator.normal

```
np.random.Generator.normal(self, loc=0.0, scale=1.0, size=None)
```

Draw random samples from a normal (Gaussian) distribution.

The probability density function of the normal distribution, first derived by De Moivre and 200 years later by both Gauss and Laplace independently, is often called the bell curve because of its characteristic shape (see the example below).

The normal distributions occurs often in nature. For example, it describes the commonly occurring distribution of samples influenced by a large number of tiny, random disturbances, each with its own unique distribution.

Parameters

`loc` : *float or array_like of floats* Mean ("centre") of the distribution.

`scale` : *float or array_like of floats* Standard deviation (spread or "width") of the distribution. Must be non-negative.

`size` : *int or tuple of ints, optional* Output shape. If the given shape is, e.g., `(m, n, k)`, then `m * n * k` samples are drawn. If size is `None` (default), a single value is returned if `loc` and `scale` are both scalars. Otherwise, `np.broadcast(loc, scale).size` samples are drawn.

Returns

`out` : *ndarray or scalar* Drawn samples from the parameterized normal distribution.

See also: `scipy.stats.norm`

Notes

The probability density for the Gaussian distribution is

$$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}},$$

where μ is the mean and σ the standard deviation. The square of the standard deviation, σ^2, is called the variance.

The function has its peak at the mean, and its ”spread” increases with the standard deviation (the function reaches 0.607 times its maximum at $x + \sigma$ and $x - \sigma$). This implies that `normal` is more likely to return samples lying close to the mean, rather than those far away.

Examples

Draw samples from the distribution:

```
>>> mu, sigma = 0, 0.1 # mean and standard deviation
>>> rng = np.random.default_rng()
>>> s = rng.normal(mu, sigma, 1000)
```

Verify the mean and the standard deviation:

```
>>> abs(mu - np.mean(s))
0.0  # may vary
```

```
>>> abs(sigma - np.std(s, ddof=1))
0.0  # may vary
```

Display the histogram of the samples, along with the probability density function: ...

np.random.Generator.integers

```
np.random.Generator.integers(self, low, high=None, size=None, dtype=<class 'numpy.int64'>,
↪  endpoint=False)
```

Return random integers from low (inclusive) to high (exclusive), or if endpoint=True, low (inclusive) to high (inclusive). Replaces RandomState.randint (with endpoint=False) and RandomState.random_integers (with endpoint=True)

Return random integers from the ”discrete uniform” distribution of the specified dtype. If high is None (the default), then results are from 0 to low.

Parameters

`low` : *int or array-like of ints* Lowest (signed) integers to be drawn from the distribution (unless `high=None`, in which case this parameter is 0 and this value is used for high).

`high` : *int or array-like of ints, optional* If provided, one above the largest (signed) integer to be drawn from the distribution (see above for behavior if `high=None`). If array-like, must contain integer values

`size` : *int or tuple of ints, optional* Output shape. If the given shape is, e.g., `(m, n, k)`, then `m * n * k` samples are drawn. Default is None, in which case a single value is returned.

`dtype` : *dtype, optional* Desired dtype of the result. Byteorder must be native. The default value is np.int64.

`endpoint` : *bool, optional* If true, sample from the interval [low, high] instead of the default [low, high) Defaults to False

Returns

out : *int or ndarray of ints* size-shaped array of random integers from the appropriate distribution, or a single such random int if size not provided.

Notes

When using broadcasting with uint64 dtypes, the maximum value (2*64) cannot be represented as a standard integer type. The high array (or low if high is None) must have object dtype, e.g., array([2*64]).

Examples

```
>>> rng = np.random.default_rng()
>>> rng.integers(2, size=10)
array([1, 0, 0, 0, 1, 1, 0, 0, 1, 0])  # random
>>> rng.integers(1, size=10)
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

Generate a 2 x 4 array of ints between 0 and 4, inclusive:

```
>>> rng.integers(5, size=(2, 4))
array([[4, 0, 2, 1],
       [3, 2, 2, 0]])  # random
```

Generate a 1 x 3 array with 3 different upper bounds

```
>>> rng.integers(1, [3, 5, 10])
array([2, 2, 9])  # random
...
```

np.random.Generator.choice

```
np.random.Generator.choice(self, a, size=None, replace=True, p=None, axis=0, shuffle=True)
```

Generates a random sample from a given array

Parameters

a : *{array_like, int}* If an ndarray, a random sample is generated from its elements. If an int, the random sample is generated from np.arange(a).

size : *{int, tuple[int]}, optional* Output shape. If the given shape is, e.g., `(m, n, k)`, then `m * n * k` samples are drawn from the 1-d a. If a has more than one dimension, the size shape will be inserted into the axis dimension, so the output `ndim` will be `a.ndim - 1 + len(size)`. Default is None, in which case a single value is returned.

replace : *bool, optional* Whether the sample is with or without replacement. Default is True, meaning that a value of `a` can be selected multiple times.

p : *1-D array_like, optional* The probabilities associated with each entry in a. If not given, the sample assumes a uniform distribution over all entries in `a`.

axis : *int, optional* The axis along which the selection is performed. The default, 0, selects by row.

shuffle : *bool, optional* Whether the sample is shuffled when sampling without replacement. Default is True, False provides a speedup.

Returns

samples : *single item or ndarray* The generated random samples

Raises

ValueError If a is an int and less than zero, if p is not 1-dimensional, if a is array-like with a size 0, if p is not a vector of probabilities, if a and p have different lengths, or if replace=False and the sample size is greater than the population size.

See also: `integers`, `shuffle`, `permutation`

Notes

Setting user-specified probabilities through `p` uses a more general but less efficient sampler than the default. The general sampler produces a different sample than the optimized sampler even if each element of `p` is 1 / len(a).

`p` must sum to 1 when cast to `float64`. To ensure this, you may wish to normalize using `p = p / np.sum(p, dtype=np.float64)`.

When passing `a` as an integer type and `size` is not specified, the return type is a native Python `int`.

Examples

Generate a uniform random sample from np.arange(5) of size 3:

```
>>> rng = np.random.default_rng()
>>> rng.choice(5, 3)
array([0, 3, 4]) # random
>>> #This is equivalent to rng.integers(0,5,3)
```

Generate a non-uniform random sample from np.arange(5) of size 3:

```
>>> rng.choice(5, 3, p=[0.1, 0, 0.3, 0.6, 0])
array([3, 3, 0]) # random
```

Generate a uniform random sample from np.arange(5) of size 3 without replacement:

```
>>> rng.choice(5, 3, replace=False)
array([3,1,0]) # random
>>> #This is equivalent to rng.permutation(np.arange(5))[:3]
...
```

np.random.Generator.permutation

```
np.random.Generator.permutation(self, x, axis=0)
```

Randomly permute a sequence, or return a permuted range.

Parameters

`x` : *int or array_like* If x is an integer, randomly permute `np.arange(x)`. If x is an array, make a copy and shuffle the elements randomly.

`axis` : *int, optional* The axis which x is shuffled along. Default is 0.

Returns

`out` : *ndarray* Permuted sequence or array range.

Examples

```
>>> rng = np.random.default_rng()
>>> rng.permutation(10)
array([1, 7, 4, 3, 0, 9, 2, 5, 8, 6]) # random
```

```
>>> rng.permutation([1, 4, 9, 12, 15])
array([15,  1,  9,  4, 12]) # random
```

```
>>> arr = np.arange(9).reshape((3, 3))
>>> rng.permutation(arr)
array([[6, 7, 8], # random
       [0, 1, 2],
       [3, 4, 5]])
```

```
>>> rng.permutation("abc")
Traceback (most recent call last):
    ...
numpy.exceptions.AxisError: axis 0 is out of bounds for array of dimension 0
...
```

I/O

np.save

```
np.save(file, arr, allow_pickle=True)
```

Save an array to a binary file in NumPy `.npy` format.

Parameters

`file` : *file, str, or pathlib.Path* File or filename to which the data is saved. If file is a file-object, then the filename is unchanged. If file is a string or Path, a `.npy` extension will be appended to the filename if it does not already have one.

`arr` : *array_like* Array data to be saved.

`allow_pickle` : *bool, optional* Allow saving object arrays using Python pickles. Reasons for disallowing pickles include security (loading pickled data can execute arbitrary code) and portability (pickled objects may not be loadable on different Python installations, for example if the stored objects require libraries that are not available, and not all pickled data is compatible between different versions of Python). Default: True

See also: `savez`, `savetxt`, `load`

Notes

For a description of the `.npy` format, see `numpy.lib.format`.

Any data saved to the file is appended to the end of the file.

Examples

```
>>> from tempfile import TemporaryFile
>>> outfile = TemporaryFile()
```

```
>>> x = np.arange(10)
>>> np.save(outfile, x)
```

```
>>> _ = outfile.seek(0) # Only needed to simulate closing & reopening file
>>> np.load(outfile)
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
>>> with open('test.npy', 'wb') as f:
...     np.save(f, np.array([1, 2]))
...     np.save(f, np.array([1, 3]))
>>> with open('test.npy', 'rb') as f:
```

```
...     a = np.load(f)
...     b = np.load(f)
>>> print(a, b)
**[1 2] [1 3]
...
```

np.load

```
np.load(file, mmap_mode=None, allow_pickle=False, fix_imports=True, encoding='ASCII', *,
↪  max_header_size=10000)
```

Load arrays or pickled objects from `.npy`, `.npz` or pickled files.

> Warning
> Loading files that contain object arrays uses the `pickle` module, which is not secure against erroneous or maliciously constructed data. Consider passing `allow_pickle=False` to load data that is known not to contain object arrays for the safer handling of untrusted sources.

Parameters

`file` : ***file-like object, string, or pathlib.Path*** The file to read. File-like objects must support the `seek()` and `read()` methods and must always be opened in binary mode. Pickled files require that the file-like object support the `readline()` method as well.

`mmap_mode` : ***{None, 'r+', 'r', 'w+', 'c'}, optional*** If not None, then memory-map the file, using the given mode (see numpy.memmap for a detailed description of the modes). A memory-mapped array is kept on disk. However, it can be accessed and sliced like any ndarray. Memory mapping is especially useful for accessing small fragments of large files without reading the entire file into memory.

`allow_pickle` : ***bool, optional*** Allow loading pickled object arrays stored in npy files. Reasons for disallowing pickles include security, as loading pickled data can execute arbitrary code. If pickles are disallowed, loading object arrays will fail. Default: False

`fix_imports` : ***bool, optional*** Only useful when loading Python 2 generated pickled files, which includes npy/npz files containing object arrays. If fix_imports is True, pickle will try to map the old Python 2 names to the new names used in Python 3.

`encoding` : ***str, optional*** What encoding to use when reading Python 2 strings. Only useful when loading Python 2 generated pickled files, which includes npy/npz files containing object arrays. Values other than 'latin1', 'ASCII', and 'bytes' are not allowed, as they can corrupt numerical data. Default: 'ASCII'

`max_header_size` : ***int, optional*** Maximum allowed size of the header. Large headers may not be safe to load securely and thus require explicitly passing a larger value. See `ast.literal_eval()` for details. This option is ignored when allow_pickle is passed. In that case the file is by definition trusted and the limit is unnecessary.

Returns

`result` : ***array, tuple, dict, etc.*** Data stored in the file. For `.npz` files, the returned instance of NpzFile class must be closed to avoid leaking file descriptors.

Raises

OSError If the input file does not exist or cannot be read.

UnpicklingError If `allow_pickle=True`, but the file cannot be loaded as a pickle.

ValueError The file contains an object array, but `allow_pickle=False` given.

EOFError When calling `np.load` multiple times on the same file handle, if all data has already been read

See also: `save`, `savez`, `savez_compressed`, `loadtxt`, `memmap`, `lib.format.open_memmap`

Notes

- If the file contains pickle data, then whatever object is stored in the pickle is returned.
- If the file is a `.npy` file, then a single array is returned.
- If the file is a `.npz` file, then a dictionary-like object is returned, containing `{filename: array}` key-value pairs, one for each file in the archive.
- If the file is a `.npz` file, the returned value supports the context manager protocol in a similar fashion to the open function:

  ```
  with load('foo.npz') as data:
      a = data['a']
  ```

  The underlying file descriptor is closed when exiting the 'with' block.

Examples

Store data to disk, and load it again:

```
>>> np.save('/tmp/123', np.array([[1, 2, 3], [4, 5, 6]]))
>>> np.load('/tmp/123.npy')
array([[1, 2, 3],
       [4, 5, 6]])
```

Store compressed data to disk, and load it again:

```
>>> a=np.array([[1, 2, 3], [4, 5, 6]])
>>> b=np.array([1, 2])
>>> np.savez('/tmp/123.npz', a=a, b=b)
>>> data = np.load('/tmp/123.npz')
>>> data['a']
array([[1, 2, 3],
       [4, 5, 6]])
>>> data['b']
array([1, 2])
>>> data.close()
...
```

np.savez

```
np.savez(file, *args, allow_pickle=True, **kwds)
```

Save several arrays into a single file in uncompressed `.npz` format.

Provide arrays as keyword arguments to store them under the corresponding name in the output file: `savez(fn, x=x, y=y)`.

If arrays are specified as positional arguments, i.e., `savez(fn, x, y)`, their names will be arr_0, arr_1, etc.

Parameters

`file` : ***file, str, or pathlib.Path*** Either the filename (string) or an open file (file-like object) where the data will be saved. If file is a string or a Path, the `.npz` extension will be appended to the filename if it is not already there.

`args` : *Arguments, optional* Arrays to save to the file. Please use keyword arguments (see kwds below) to assign names to arrays. Arrays specified as args will be named ”arr_0”, ”arr_1”, and so on.

`allow_pickle` : *bool, optional* Allow saving object arrays using Python pickles. Reasons for disallowing pickles include security (loading pickled data can execute arbitrary code) and portability (pickled objects may not be loadable on different Python installations, for example if the stored objects require libraries that are not available, and not all pickled data is compatible between different versions of Python). Default: True

`kwds` : *Keyword arguments, optional* Arrays to save to the file. Each array will be saved to the output file with its corresponding keyword name.

Returns

None

See also: `save`, `savetxt`, `savez_compressed`

Notes

The `.npz` file format is a zipped archive of files named after the variables they contain. The archive is not compressed and each file in the archive contains one variable in `.npy` format. For a description of the `.npy` format, see `numpy.lib.format`.

When opening the saved `.npz` file with load a ~lib.npyio.NpzFile object is returned. This is a dictionary-like object which can be queried for its list of arrays (with the `.files` attribute), and for the arrays themselves.

Keys passed in kwds are used as filenames inside the ZIP archive. Therefore, keys should be valid filenames; e.g., avoid keys that begin with `/` or contain `..`

(Notes truncated; see the full reference page.)

Examples

```
>>> from tempfile import TemporaryFile
>>> outfile = TemporaryFile()
>>> x = np.arange(10)
>>> y = np.sin(x)
```

Using savez with *args, the arrays are saved with default names.

```
>>> np.savez(outfile, x, y)
>>> _ = outfile.seek(0) # Only needed to simulate closing & reopening file
>>> npzfile = np.load(outfile)
>>> npzfile.files
['arr_0', 'arr_1']
>>> npzfile['arr_0']
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

Using savez with **kwds, the arrays are saved with the keyword names. ...

np.loadtxt

```
np.loadtxt(fname, dtype=<class 'float'>, comments='#', delimiter=None, converters=None, skiprows=0,
↪  usecols=None, unpack=False, ndmin=0, encoding=None, max_rows=None, *, quotechar=None, like=None)
```

Load data from a text file.

Parameters

fname : *file, str, pathlib.Path, list of str, generator* File, filename, list, or generator to read. If the filename extension is `.gz` or `.bz2`, the file is first decompressed. Note that generators must return bytes or strings. The strings in a list or produced by a generator are treated as lines.

dtype : *data-type, optional* Data-type of the resulting array; default: float. If this is a structured data-type, the resulting array will be 1-dimensional, and each row will be interpreted as an element of the array. In this case, the number of columns used must match the number of fields in the data-type.

comments : *str or sequence of str or None, optional* The characters or list of characters used to indicate the start of a comment. None implies no comments. For backwards compatibility, byte strings will be decoded as 'latin1'. The default is '#'.

delimiter : *str, optional* The character used to separate the values. For backwards compatibility, byte strings will be decoded as 'latin1'. The default is whitespace.

Changed in version 1.23.0. Only single character delimiters are supported. Newline characters cannot be used as the delimiter.

converters : *dict or callable, optional* Converter functions to customize value parsing. If converters is callable, the function is applied to all columns, else it must be a dict that maps column number to a parser function. See examples for further details. Default: None.

Changed in version 1.23.0. The ability to pass a single callable to be applied to all columns was added.

skiprows : *int, optional* Skip the first skiprows lines, including comments; default: 0.

usecols : *int or sequence, optional* Which columns to read, with 0 being the first. For example, `usecols = (1,4,5)` will extract the 2nd, 5th and 6th columns. The default, None, results in all columns being read.

unpack : *bool, optional* If True, the returned array is transposed, so that arguments may be unpacked using `x, y, z = loadtxt(...)`. When used with a structured data-type, arrays are returned for each field. Default is False.

ndmin : *int, optional* The returned array will have at least ndmin dimensions. Otherwise mono-dimensional axes will be squeezed. Legal values: 0 (default), 1 or 2.

encoding : *str, optional* Encoding used to decode the inputfile. Does not apply to input streams. The special value 'bytes' enables backward compatibility workarounds that ensures you receive byte arrays as results if possible and passes 'latin1' encoded strings to converters. Override this value to receive unicode arrays and pass strings as input to converters. If set to None the system default is used. The default value is None.

Changed in version 2.0. Before NumPy 2, the default was `'bytes'` for Python 2 compatibility. The default is now `None`.

max_rows : *int, optional* Read max_rows rows of content after skiprows lines. The default is to read all the rows. Note that empty rows containing no data such as empty lines and comment lines are not counted towards max_rows, while such lines are counted in skiprows.

Changed in version 1.23.0. Lines containing no data, including comment lines (e.g., lines starting with '#' or as specified via comments) are not counted towards max_rows.

quotechar : *unicode character or None, optional* The character used to denote the start and end of a quoted item. Occurrences of the delimiter or comment characters are ignored within a quoted item. The default value is `quotechar=None`, which means quoting support is disabled.

If two consecutive instances of quotechar are found within a quoted field, the first is treated as an escape character. See examples.

New in version 1.23.0.

like : *array_like, optional* **Reference object to allow the creation of arrays which are not** NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

out : *ndarray* Data read from the text file.

See also: `load`, `fromstring`, `fromregex`, `genfromtxt`, `scipy.io.loadmat`

Notes

This function aims to be a fast reader for simply formatted files. The genfromtxt function provides more sophisticated handling of, e.g., lines with missing values.

Each row in the input text file must have the same number of values to be able to read all values. If all rows do not have same number of values, a subset of up to n columns (where n is the least number of values present in all rows) can be read by specifying the columns via usecols.

The strings produced by the Python float.hex method can be used as input for floats.

Examples

```
>>> from io import StringIO   # StringIO behaves like a file object
>>> c = StringIO("0 1\n2 3")
>>> np.loadtxt(c)
array([[0., 1.],
       [2., 3.]])
```

```
>>> d = StringIO("M 21 72\nF 35 58")
>>> np.loadtxt(d, dtype={'names': ('gender', 'age', 'weight'),
...                      'formats': ('S1', 'i4', 'f4')})
array([(b'M', 21, 72.), (b'F', 35, 58.)],
      dtype=[('gender', 'S1'), ('age', '<i4'), ('weight', '<f4')])
```

```
>>> c = StringIO("1,0,2\n3,0,4")
>>> x, y = np.loadtxt(c, delimiter=',', usecols=(0, 2), unpack=True)
>>> x
array([1., 3.])
>>> y
array([2., 4.])
...
```

np.savetxt

```
np.savetxt(fname, X, fmt='%.18e', delimiter=' ', newline='\n', header='', footer='', comments='# ',
↪  encoding=None)
```

Save an array to a text file.

Parameters

fname : *filename, file handle or pathlib.Path* If the filename ends in `.gz`, the file is automatically saved in compressed gzip format. loadtxt understands gzipped files transparently.

X : *1D or 2D array_like* Data to be saved to a text file.

fmt : *str or sequence of strs, optional* A single format (%10.5f), a sequence of formats, or a multi-format string, e.g. 'Iteration %d -- %10.5f', in which case delimiter is ignored. For complex X, the legal options for fmt are:

- a single specifier, `fmt='%.4e'`, resulting in numbers formatted like `' (%s+%sj)' % (fmt, fmt)`
- a full string specifying every real and imaginary part, e.g. `' %.4e %+.4ej %.4e %+.4ej %.4e %+.4ej'` for 3 columns
- a list of specifiers, one per column - in this case, the real and imaginary part must have separate specifiers, e.g. `['%.3e + %.3ej', '(%.15e%+.15ej)']` for 2 columns

delimiter : *str, optional* String or character separating columns.

newline : *str, optional* String or character separating lines.

header : *str, optional* String that will be written at the beginning of the file.

footer : *str, optional* String that will be written at the end of the file.

comments : *str, optional* String that will be prepended to the `header` and `footer` strings, to mark them as comments. Default: '# ', as expected by e.g. `numpy.loadtxt`.

encoding : *{None, str}, optional* Encoding used to encode the outputfile. Does not apply to output streams. If the encoding is something other than 'bytes' or 'latin1' you will not be able to load the file in NumPy versions < 1.14. Default is 'latin1'.

See also: `save`, `savez`, `savez_compressed`

Notes

Further explanation of the fmt parameter (`%[flag]width[.precision]specifier`):

flags: `-` : left justify

`+` : Forces to precede result with + or -.

`0` : Left pad the number with zeros instead of space (see width).

width: Minimum number of characters to be printed. The value is not truncated if it has more characters.

(Notes truncated; see the full reference page.)

Examples

```
>>> x = y = z = np.arange(0.0,5.0,1.0)
>>> np.savetxt('test.out', x, delimiter=',')   # X is an array
>>> np.savetxt('test.out', (x,y,z))   # x,y,z equal sized 1D arrays
>>> np.savetxt('test.out', x, fmt='%1.4e')   # use exponential notation
```

np.genfromtxt

```
np.genfromtxt(fname, dtype=<class 'float'>, comments='#', delimiter=None, skip_header=0, skip_footer=0,
 ↪  converters=None, missing_values=None, filling_values=None, usecols=None, names=None,
 ↪  excludelist=None, deletechars=" !#$%&'()*+,-./:;<=>?@[\\]^{|}~", replace_space='_', autostrip=False,
 ↪  case_sensitive=True, defaultfmt='f%i', unpack=None, usemask=False, loose=True, invalid_raise=True,
 ↪  max_rows=None, encoding= ...)
```

Load data from a text file, with missing values handled as specified.

Each line past the first skip_header lines is split at the delimiter character, and characters following the comments character are discarded.

Parameters

fname : *file, str, pathlib.Path, list of str, generator* File, filename, list, or generator to read. If the filename extension is `.gz` or `.bz2`, the file is first decompressed. Note that generators must return bytes or strings. The strings in a list or produced by a generator are treated as lines.

dtype : *dtype, optional* Data type of the resulting array. If a structured dtype, the output array will be 1D and structured where each field corresponds to one column. If None, the dtype of each column will be inferred automatically, and the output array will be structured only if either the dtypes are not all the same or if names is not None.

comments : *str, optional* The character used to indicate the start of a comment. All the characters occurring on a line after a comment are discarded.

delimiter : *str, int, or sequence, optional* The string used to separate values. By default, any consecutive whitespaces act as delimiter. An integer or sequence of integers can also be provided as width(s) of each field.

skiprows : *int, optional* skiprows was removed in numpy 1.10. Please use skip_header instead.

skip_header : *int, optional* The number of lines to skip at the beginning of the file.

skip_footer : *int, optional* The number of lines to skip at the end of the file.

converters : *variable, optional* The set of functions that convert the data of a column to a value. The converters can also be used to provide a default value for missing data: `converters = {3: lambda s: float(s or 0)}`.

missing : *variable, optional* missing was removed in numpy 1.10. Please use missing_values instead.

missing_values : *variable, optional* The set of strings corresponding to missing data.

filling_values : *variable, optional* The set of values to be used as default when the data are missing.

usecols : *sequence, optional* Which columns to read, with 0 being the first. For example, `usecols = (1, 4, 5)` will extract the 2nd, 5th and 6th columns.

names : *{None, True, str, sequence}, optional* If names is True, the output will be a structured array whose field names are read from the first line after the first skip_header lines. This line can optionally be preceded by a comment delimiter. Any content before the comment delimiter is discarded. If names is a sequence or a single string of comma-separated names, the output is a structured array whose field names are taken from names. If names is None, the output is structured only if dtype is structured, in which case the field names are taken from dtype.

excludelist : *sequence, optional* A list of names to exclude. This list is appended to the default list ['return','file','print']. Excluded names are appended with an underscore: for example, file would become file_.

deletechars : *str, optional* A string combining invalid characters that must be deleted from the names.

defaultfmt : *str, optional* A format used to define default field names, such as ”f%i” or ”f%02i”.

autostrip : *bool, optional* Whether to automatically strip white spaces from the variables.

replace_space : *char, optional* Character(s) used in replacement of white spaces in the variable names. By default, use a '_'.

case_sensitive : *{True, False, 'upper', 'lower'}, optional* If True, field names are case sensitive. If False or 'upper', field names are converted to upper case. If 'lower', field names are converted to lower case.

unpack : *bool, optional* If True, the returned array is transposed, so that arguments may be unpacked using `x, y, z = genfromtxt(...)`. When used with a structured data-type, arrays are returned for each field. Default is False.

usemask : *bool, optional* If True, return a masked array. If False, return a regular array.

loose : *bool, optional* If True, do not raise errors for invalid values.

invalid_raise : *bool, optional* If True, an exception is raised if an inconsistency is detected in the number of columns. If False, a warning is emitted and the offending lines are skipped.

max_rows : *int, optional* The maximum number of rows to read. Must not be used with skip_footer at the same time. If given, the value must be at least 1. Default is to read the entire file.

encoding : *str, optional* Encoding used to decode the inputfile. Does not apply when fname is a file object. The special value 'bytes' enables backward compatibility workarounds that ensure that you receive byte arrays when possible and passes latin1 encoded strings to converters. Override this value to receive unicode arrays and pass strings as input to converters. If set to None the system default is used. The default value is 'bytes'.

Changed in version 2.0. Before NumPy 2, the default was `'bytes'` for Python 2 compatibility. The default is now `None`.

ndmin : *int, optional* Same parameter as loadtxt

New in version 1.23.0.

like : *array_like, optional* **Reference object to allow the creation of arrays which are not** NumPy arrays. If an array-like passed in as `like` supports the `__array_function__` protocol, the result will be defined by it. In this case, it ensures the creation of an array object compatible with that passed in via this argument.

New in version 1.20.0.

Returns

out : *ndarray* Data read from the text file. If usemask is True, this is a masked array.

See also: `numpy.loadtxt`

Notes

- When spaces are used as delimiters, or when no delimiter has been given as input, there should not be any missing data between two fields.
- When variables are named (either by a flexible dtype or with a names sequence), there must not be any header in the file (else a ValueError exception is raised).
- Individual values are not stripped of spaces by default. When using a custom converter, make sure the function does remove spaces.
- Custom converters may receive unexpected values due to dtype discovery.

Examples

```
>>> from io import StringIO
```

Comma delimited file with mixed dtype

```
>>> s = StringIO("1,1.3,abcde")
>>> data = np.genfromtxt(s, dtype=[('myint','i8'),('myfloat','f8'),
... ('mystring','S5')], delimiter=",")
>>> data
array((1, 1.3, b'abcde'),
      dtype=[('myint', '<i8'), ('myfloat', '<f8'), ('mystring', 'S5')])
```

Using dtype = None

```
>>> _ = s.seek(0) # needed for StringIO example only
>>> data = np.genfromtxt(s, dtype=None,
... names = ['myint','myfloat','mystring'], delimiter=",")
>>> data
array((1, 1.3, 'abcde'),
      dtype=[('myint', '<i8'), ('myfloat', '<f8'), ('mystring', '<U5')])
...
```